

Arquiteturas de Deep Learning

REDES NEURAIS RECORRENTES

Zenilton K. G. Patrocínio Jr

zenilton@pucminas.br

Arquitetura de Redes Neurais

Redes Neurais típicas (como as “*Feed-Forward*”) são grafos acíclicos direcionados, o que significa a existência de uma ordenação topológica

Arquitetura de Redes Neurais

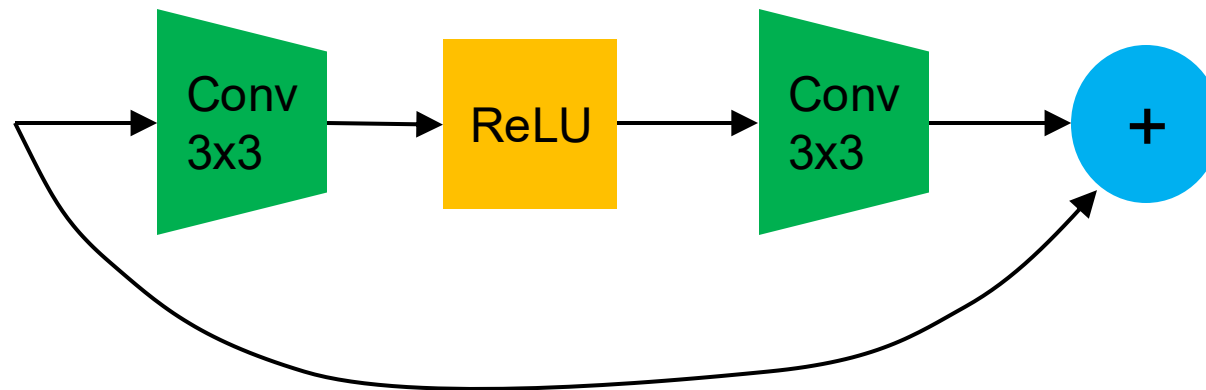
Redes Neurais típicas (como as “*Feed-Forward*”) são grafos acíclicos direcionados, o que significa a existência de uma ordenação topológica

Essa ordenação topológica pode ser usada para cálculo de ativações (no “*forward pass*”) e de gradientes (no “*backward pass*”).

Arquitetura de Redes Neurais

Redes Neurais típicas (como as “*Feed-Forward*”) são grafos acíclicos direcionados, o que significa a existência de uma ordenação topológica

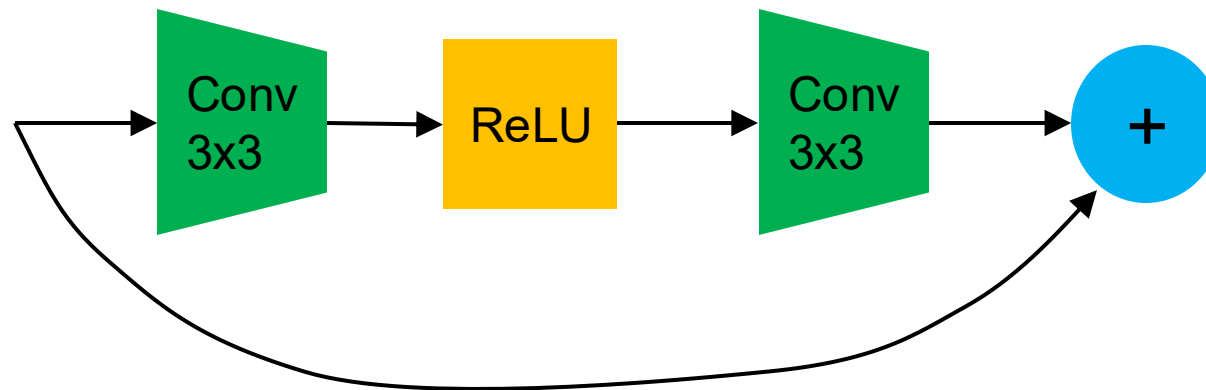
Essa ordenação topológica pode ser usada para cálculo de ativações (no “*forward pass*”) e de gradientes (no “*backward pass*”).



Arquitetura de Redes Neurais

Redes Neurais típicas (como as “*Feed-Forward*”) são grafos acíclicos direcionados, o que significa a existência de uma ordenação topológica

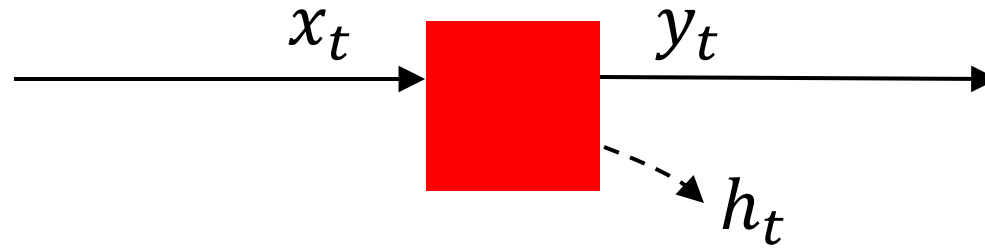
Essa ordenação topológica pode ser usada para cálculo de ativações (no “*forward pass*”) e de gradientes (no “*backward pass*”).



Essas redes processam um lote de entrada (ou ***minibatch***) a cada vez

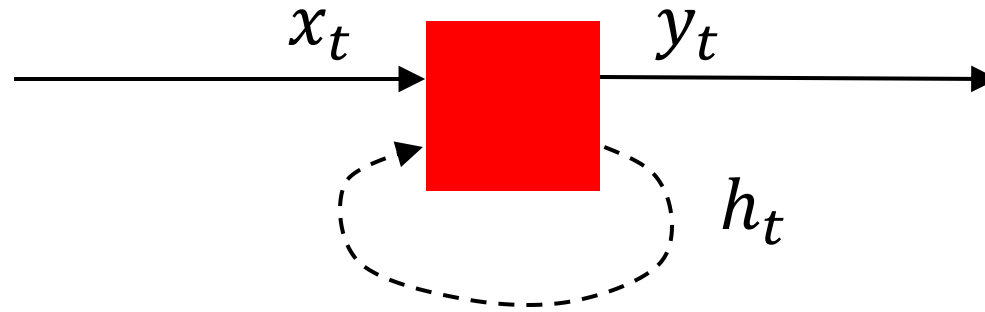
Redes Neurais Recorrentes

Redes recorrentes introduzem a presença de ciclos e a noção de tempo



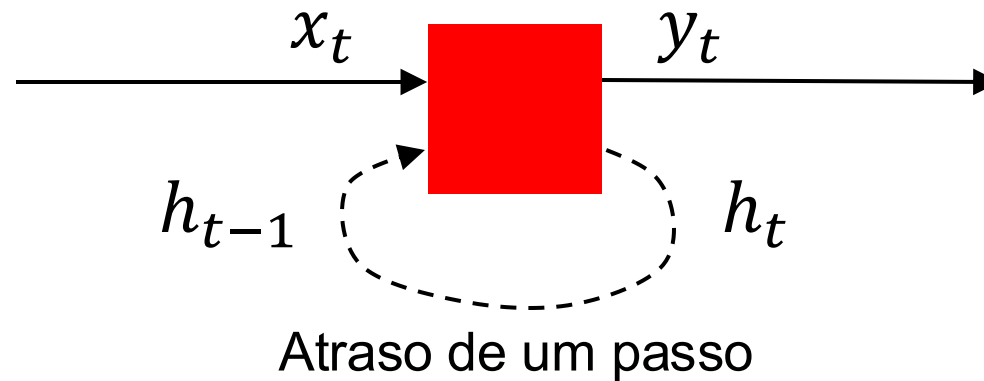
Redes Neurais Recorrentes

Redes recorrentes introduzem a presença de ciclos e a noção de tempo



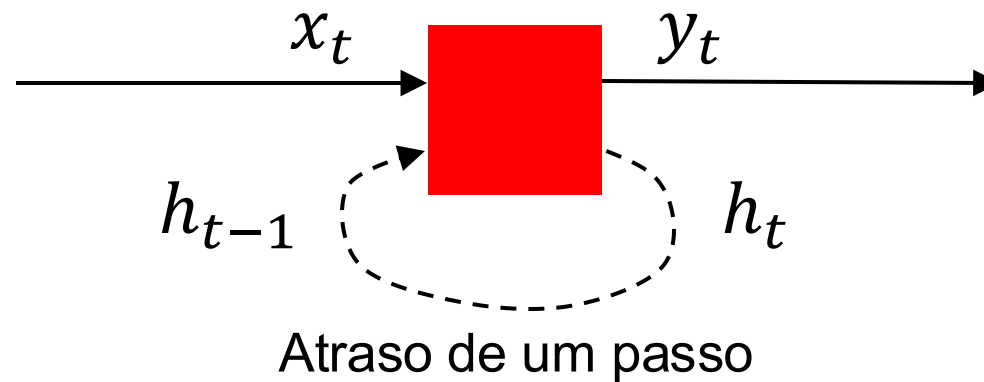
Redes Neurais Recorrentes

Redes recorrentes introduzem a presença de ciclos e a noção de tempo



Redes Neurais Recorrentes

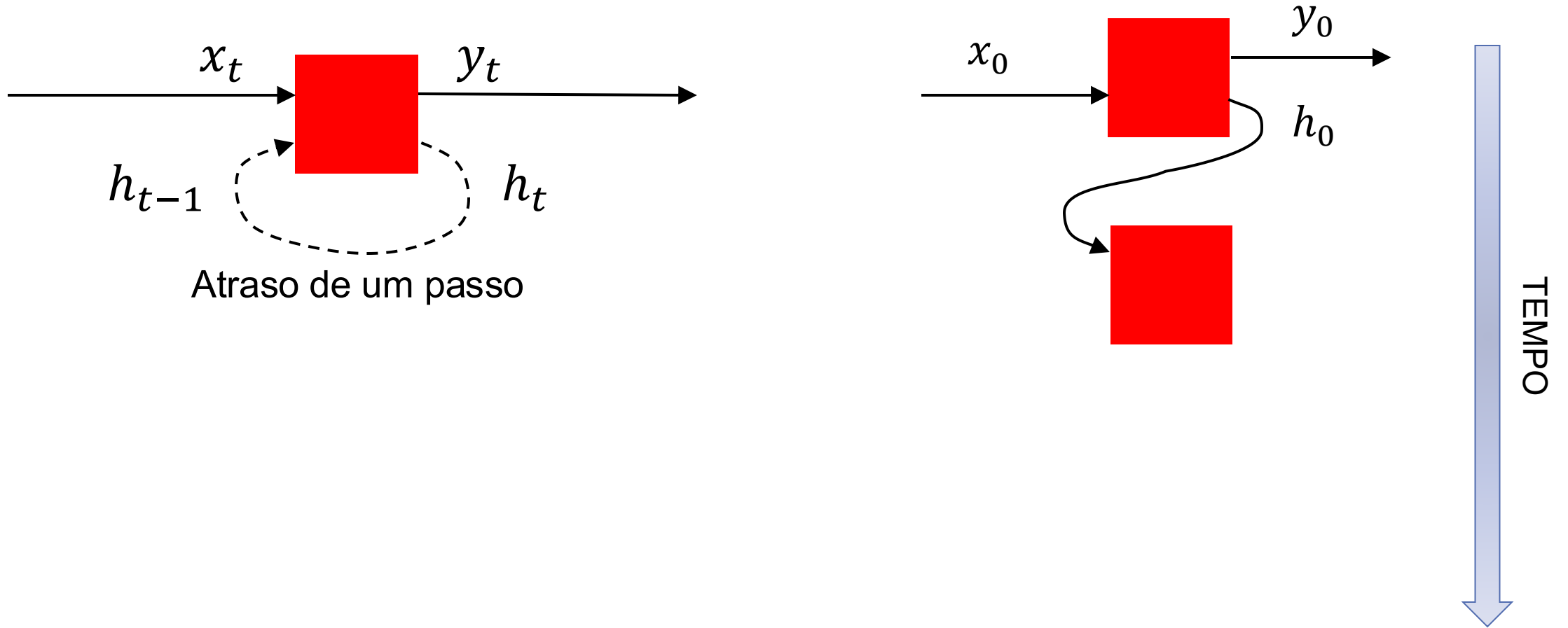
Redes recorrentes introduzem a presença de ciclos e a noção de tempo



Elas são projetadas para processar sequências de dados $x_1, \dots, x_n$ e podem produzir sequências de saídas $y_1, \dots, y_m$

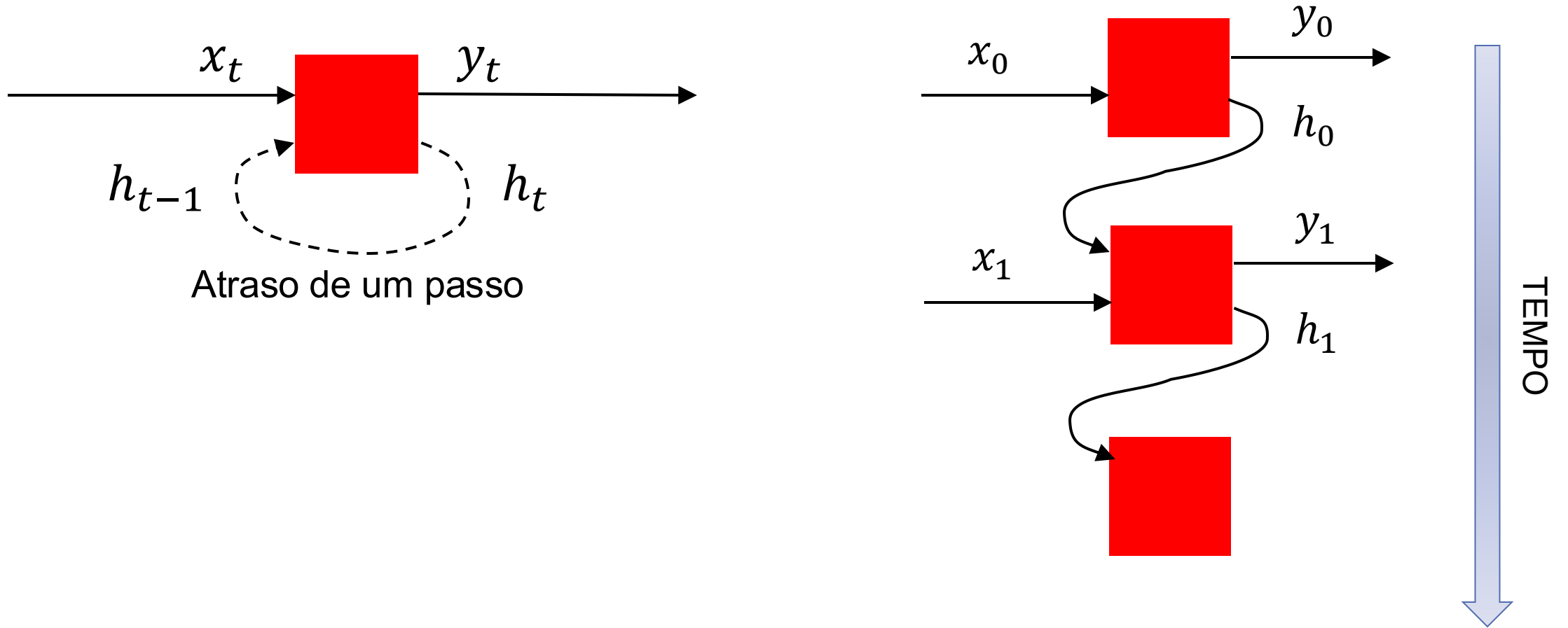
Redes Neurais Recorrentes

Rede recorrente pode ser “estendida” em várias etapas de tempo



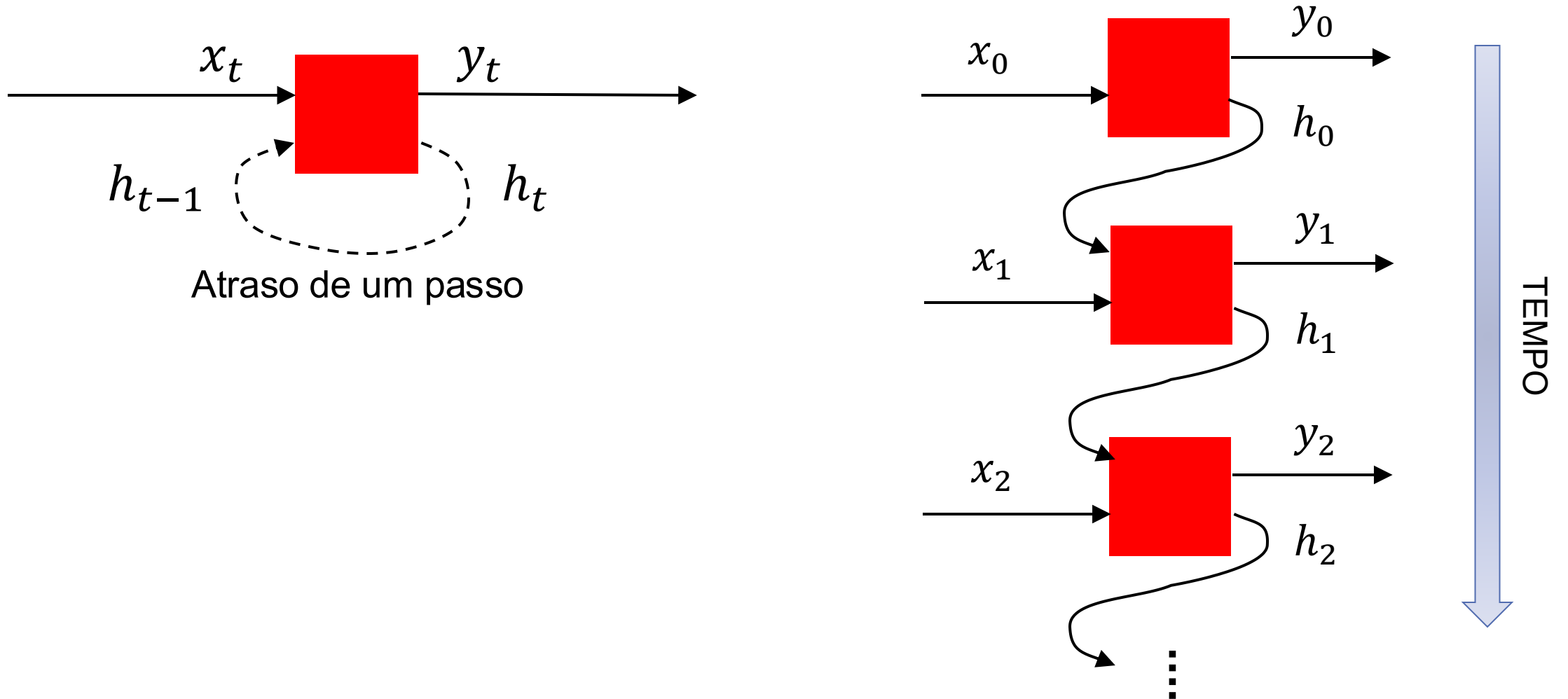
Redes Neurais Recorrentes

Rede recorrente pode ser “estendida” em várias etapas de tempo



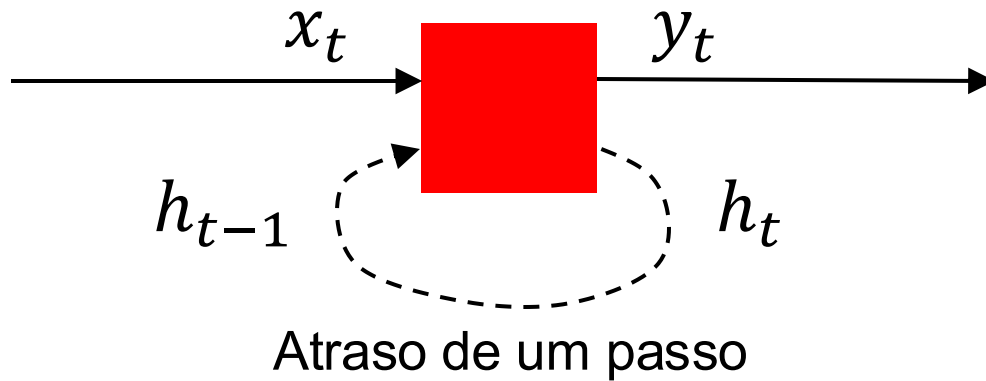
Redes Neurais Recorrentes

Rede recorrente pode ser “estendida” em várias etapas de tempo

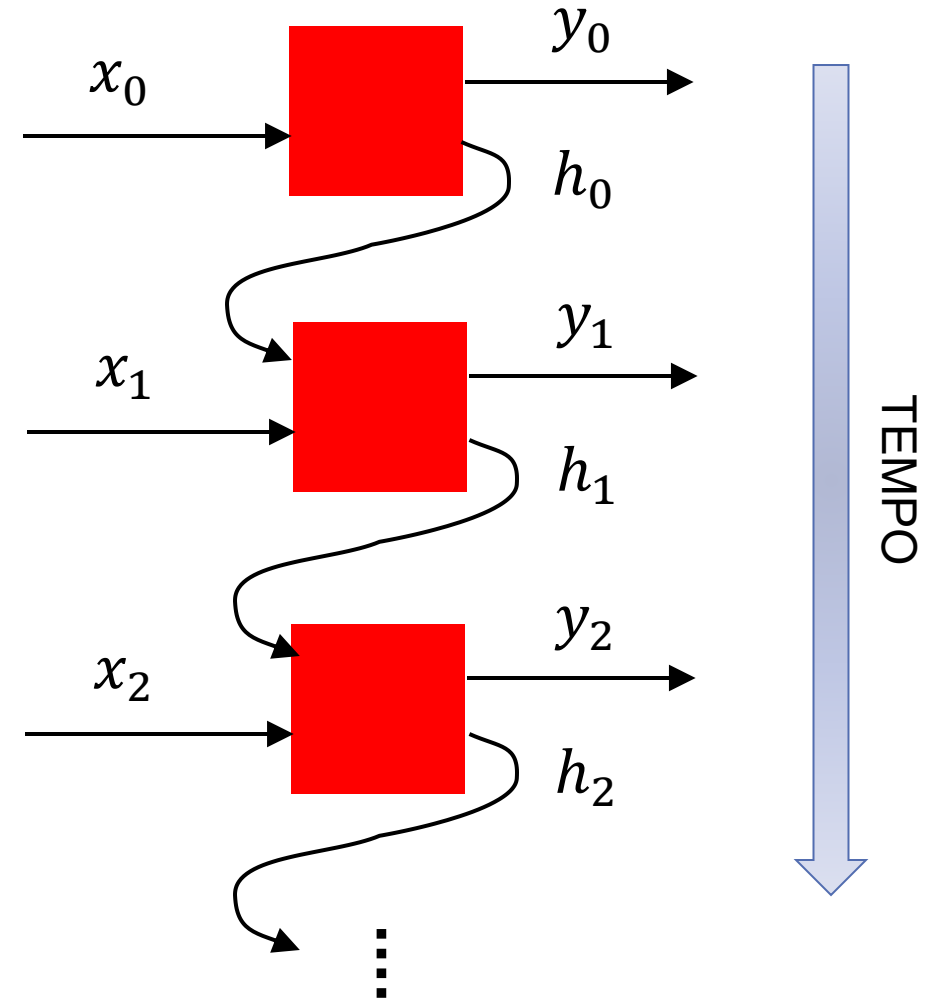


Redes Neurais Recorrentes

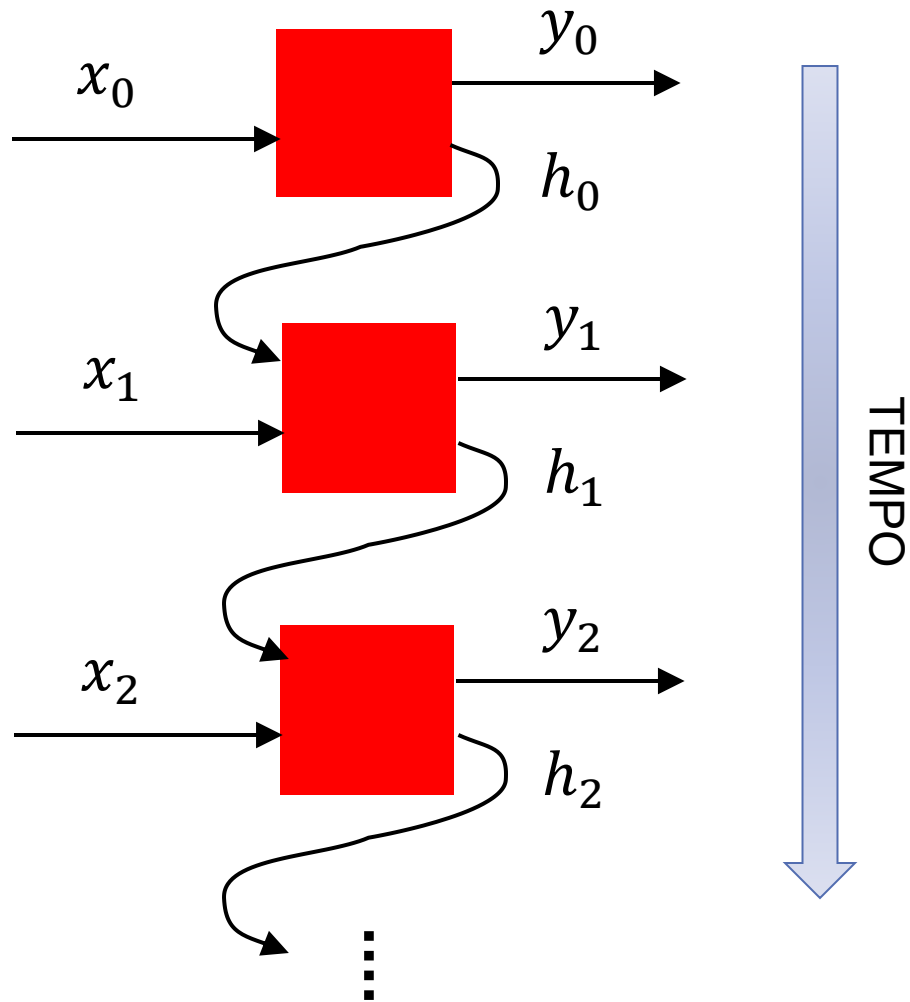
Rede recorrente pode ser “estendida” em várias etapas de tempo



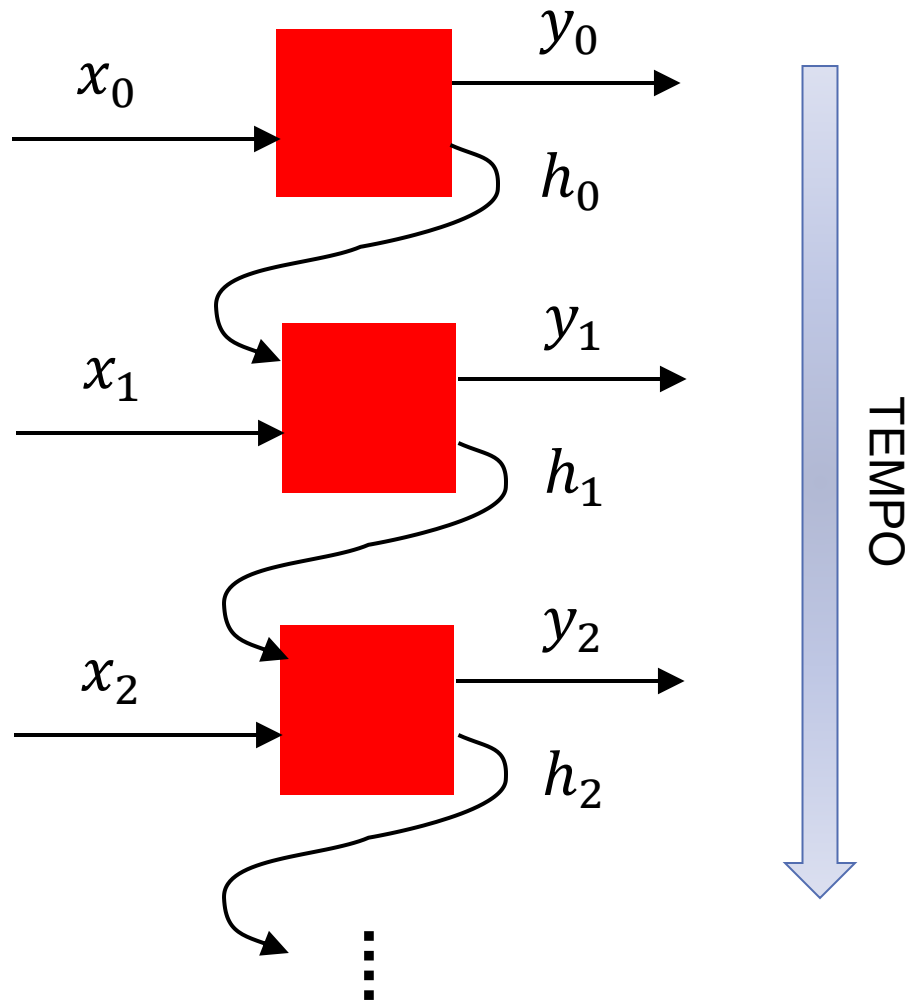
- Assim se produz um grafo acíclico direcionado que da suporte à propagação retrógrada
- Porém seu tamanho dependerá do comprimento da sequência de entrada



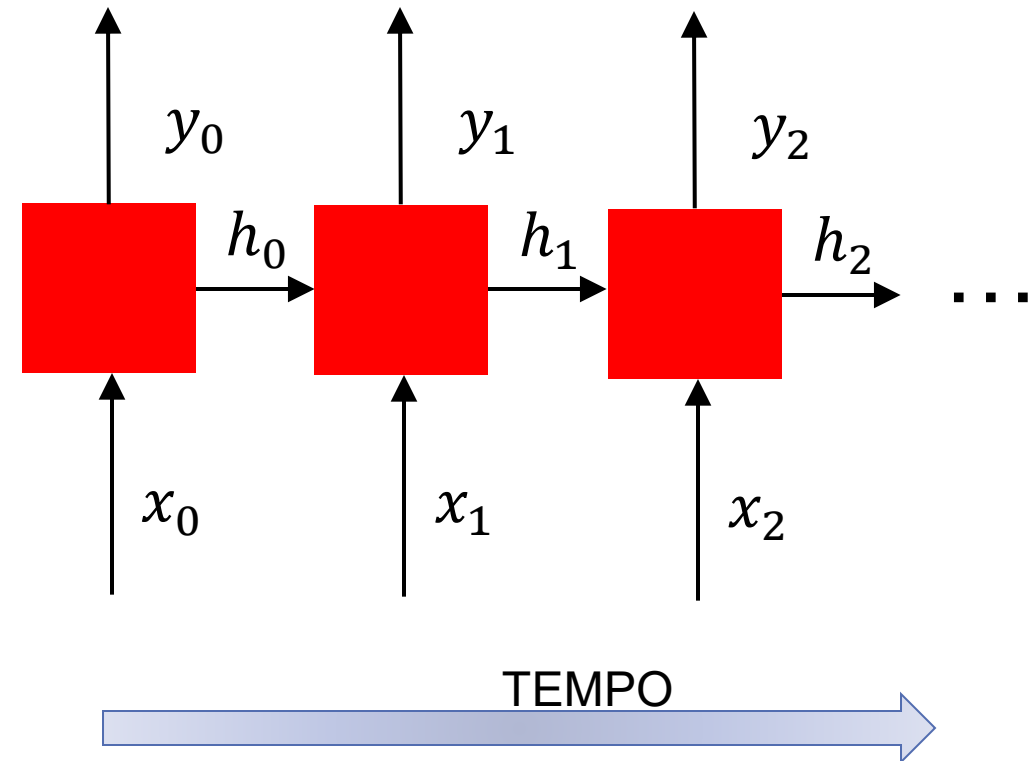
Redes Neurais Recorrentes



Redes Neurais Recorrentes

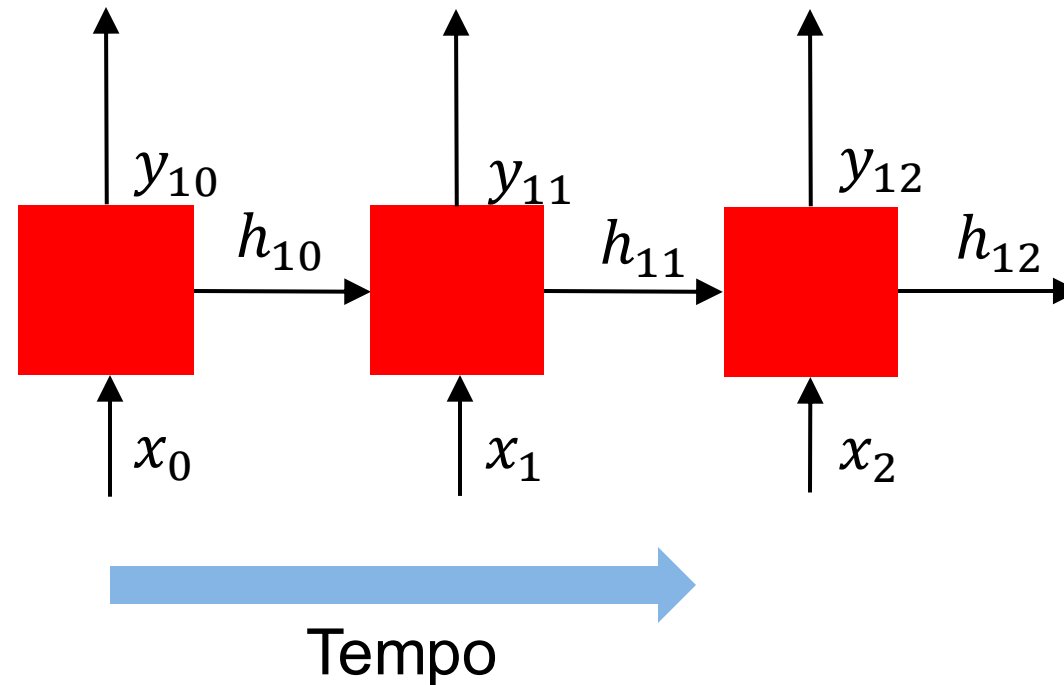


Rede “estendida” é geralmente representada na horizontal:



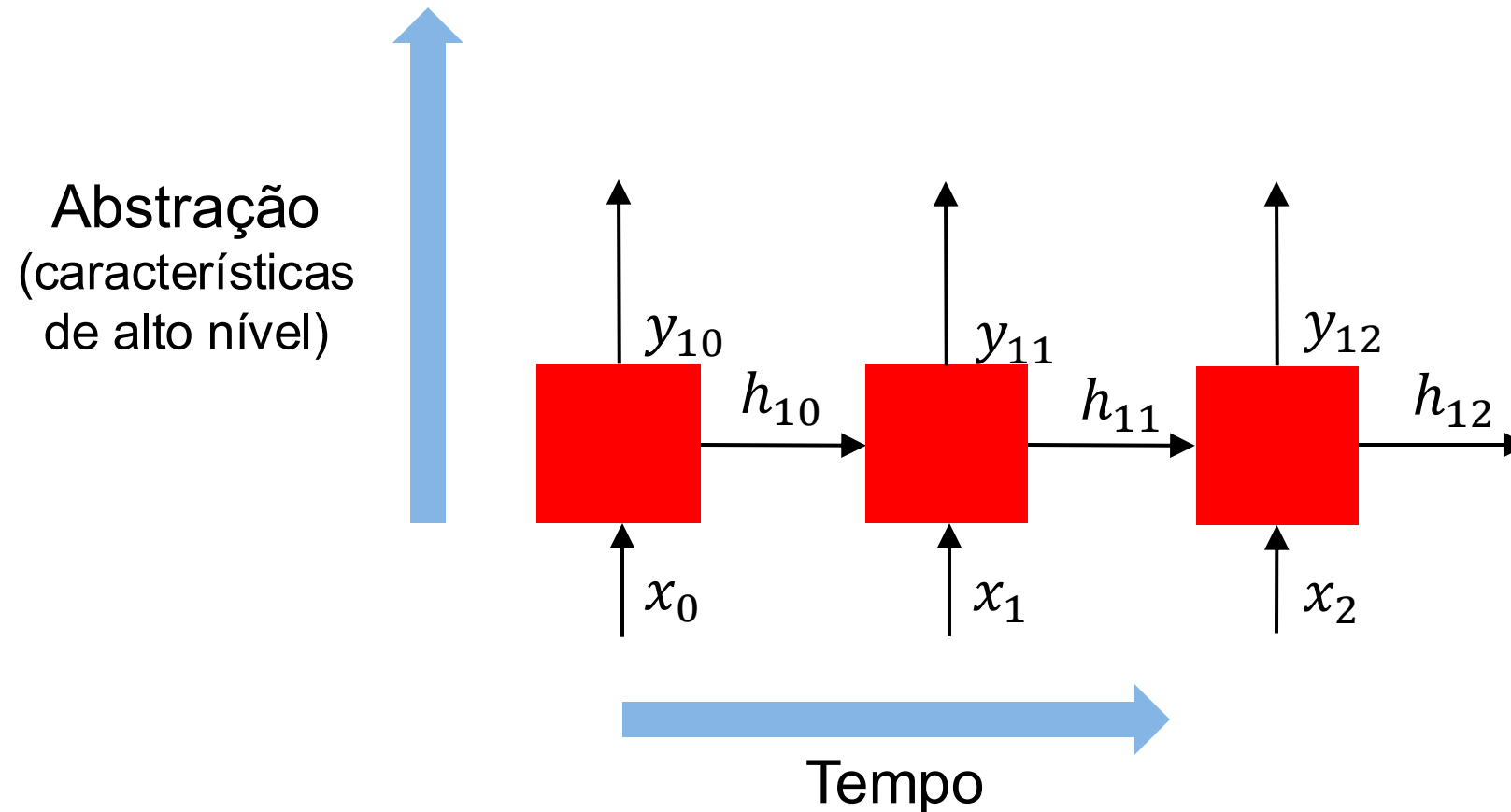
Estrutura de Redes Recorrentes

Camadas pode ser empilhadas (verticalmente) produzindo versões profundas



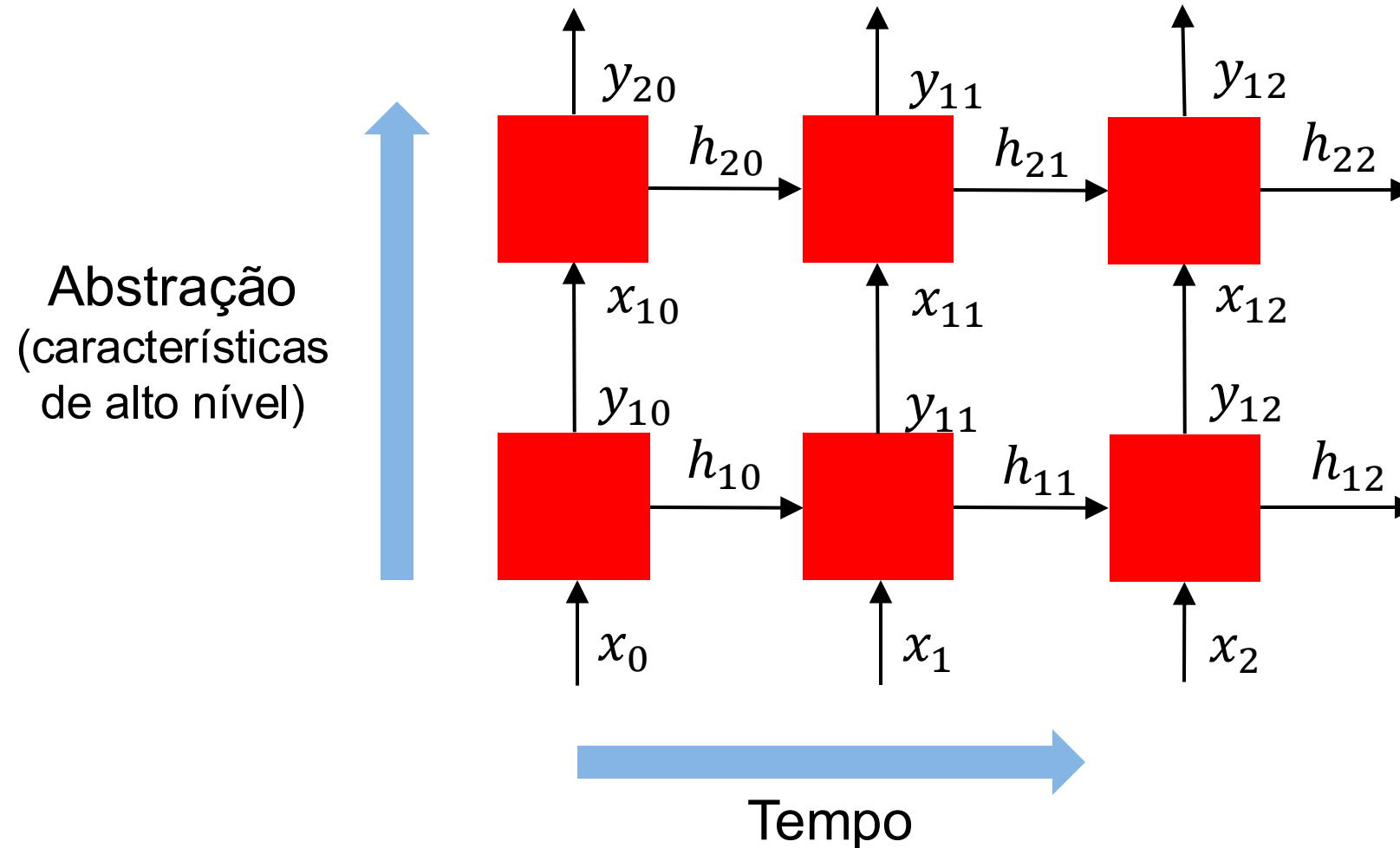
Estrutura de Redes Recorrentes

Camadas pode ser empilhadas (verticalmente) produzindo versões profundas



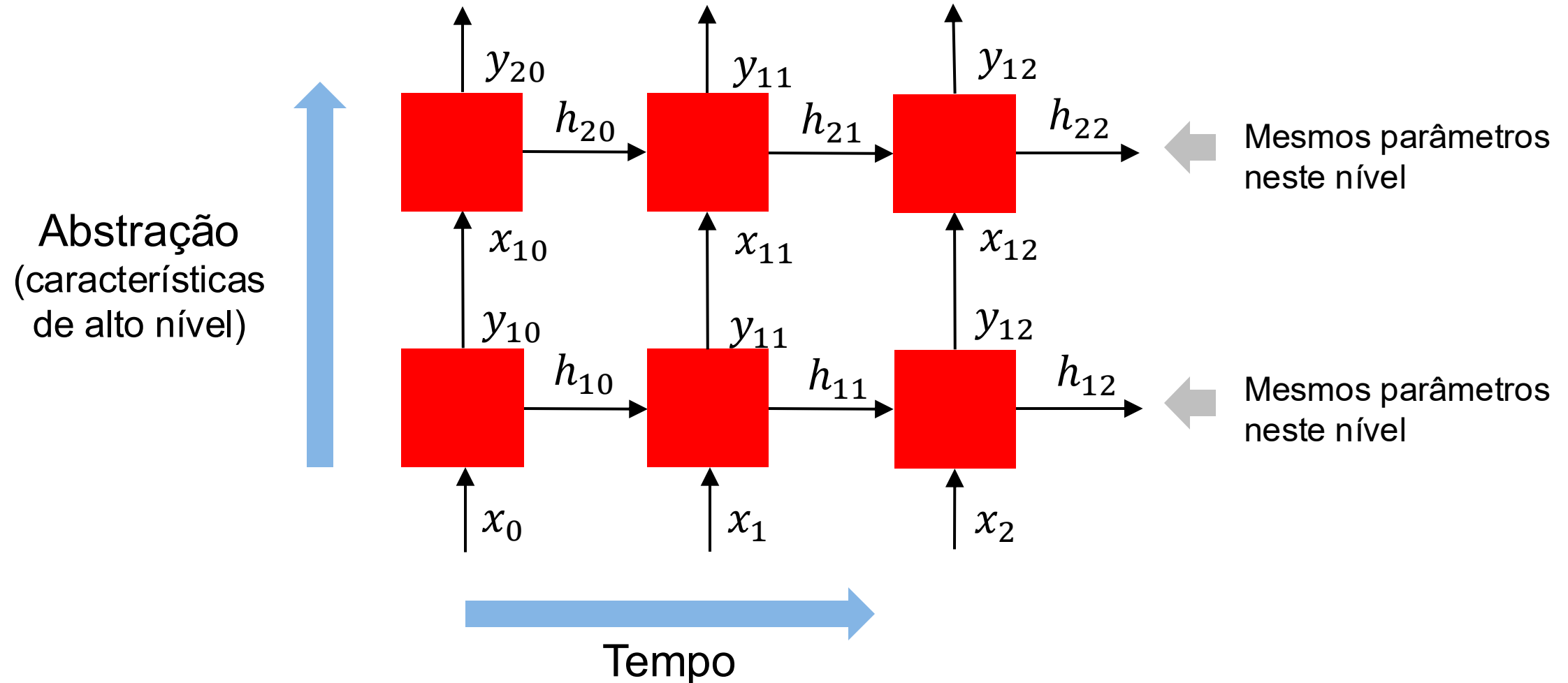
Estrutura de Redes Recorrentes

Camadas pode ser empilhadas (verticalmente) produzindo versões profundas



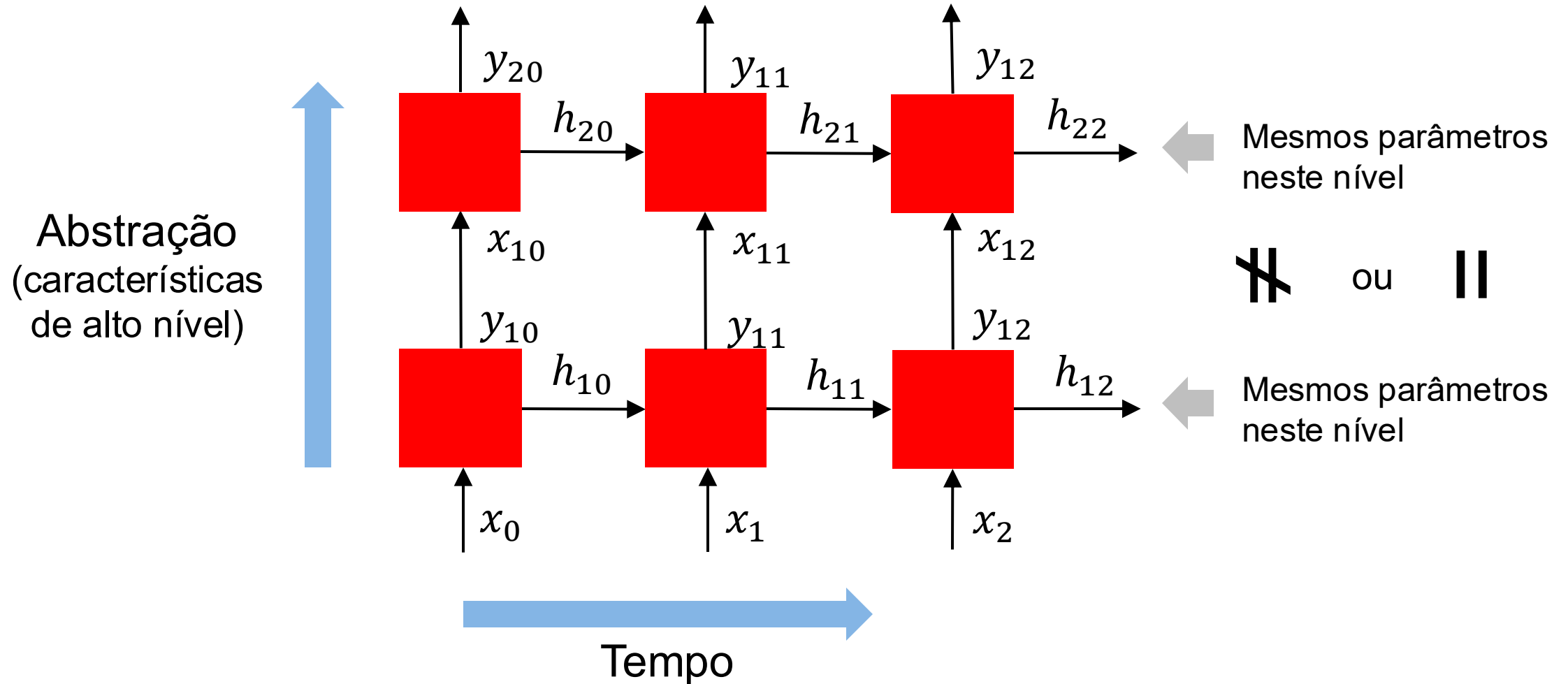
Estrutura de Redes Recorrentes

Camadas pode ser empilhadas (verticalmente) produzindo versões profundas



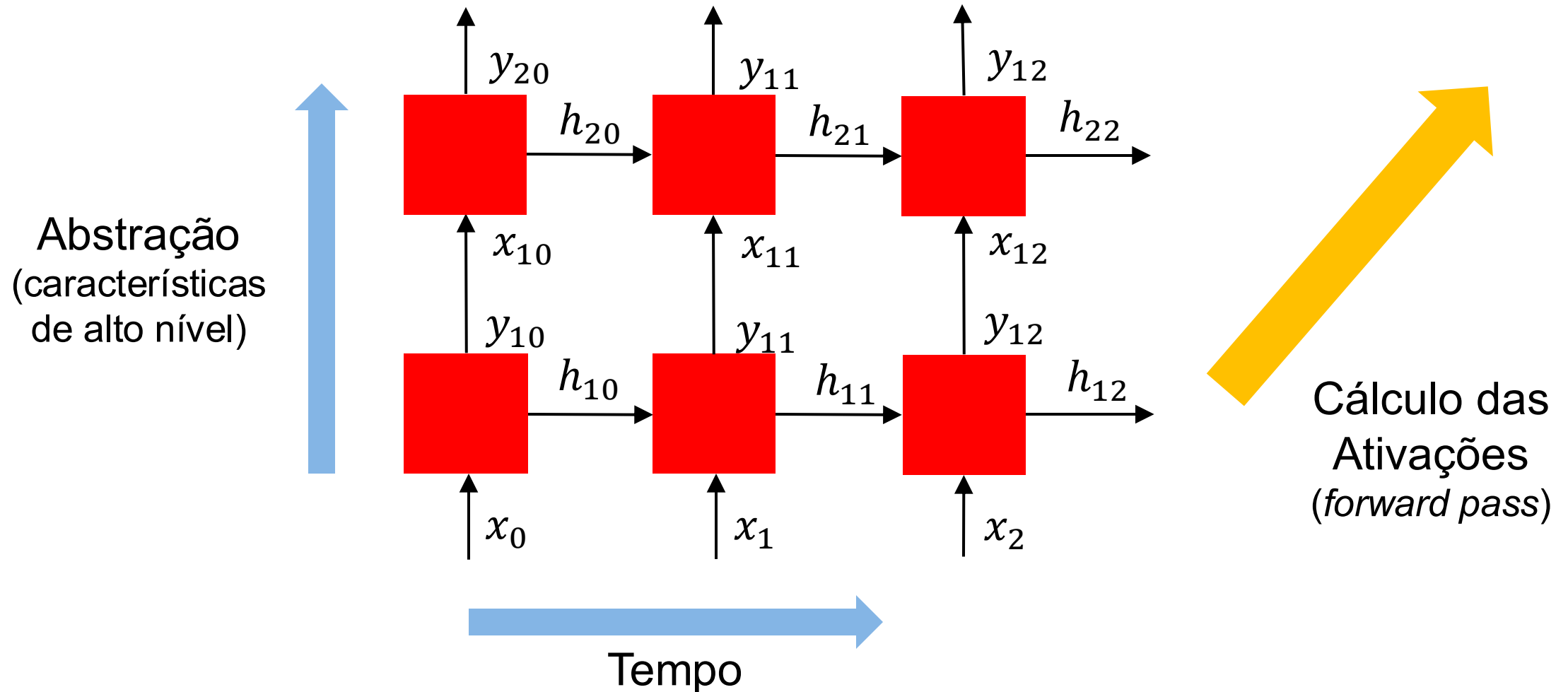
Estrutura de Redes Recorrentes

Camadas pode ser empilhadas (verticalmente) produzindo versões profundas



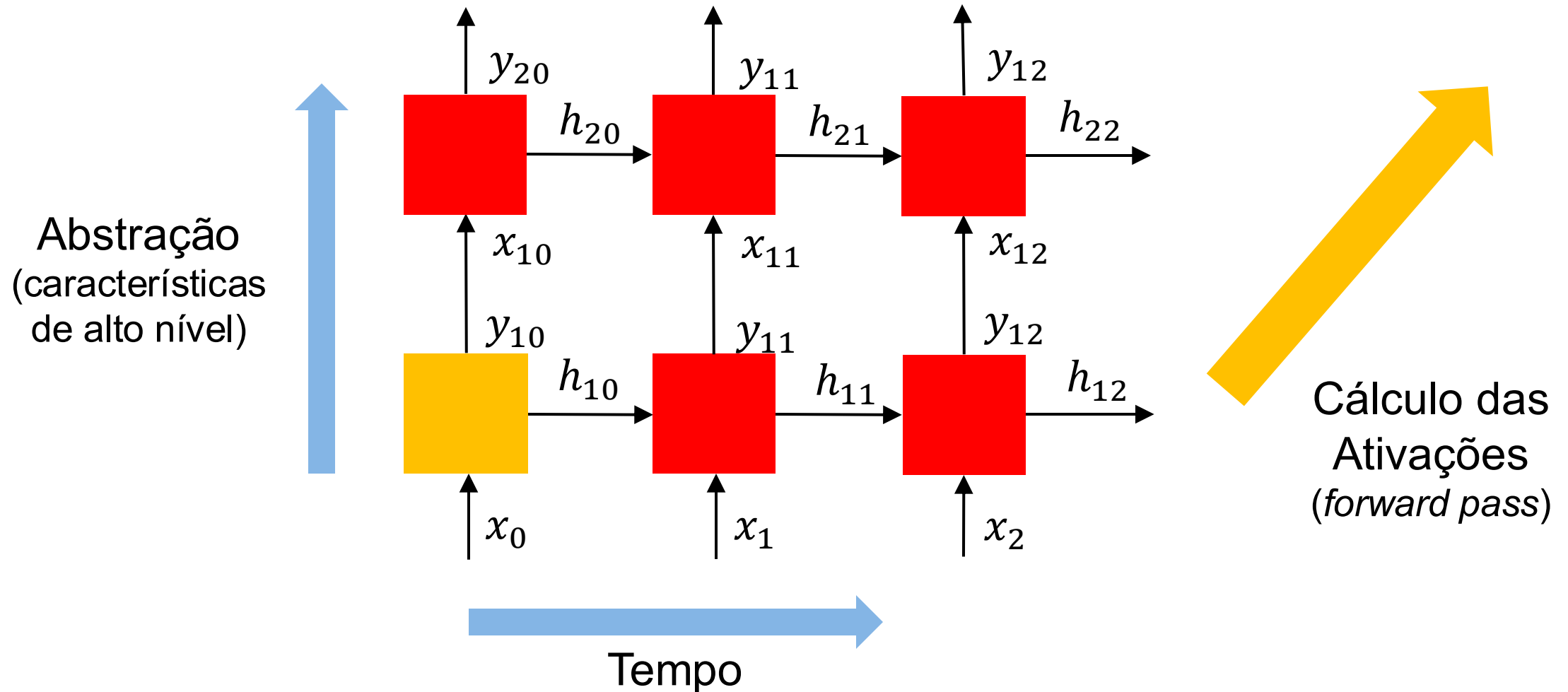
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



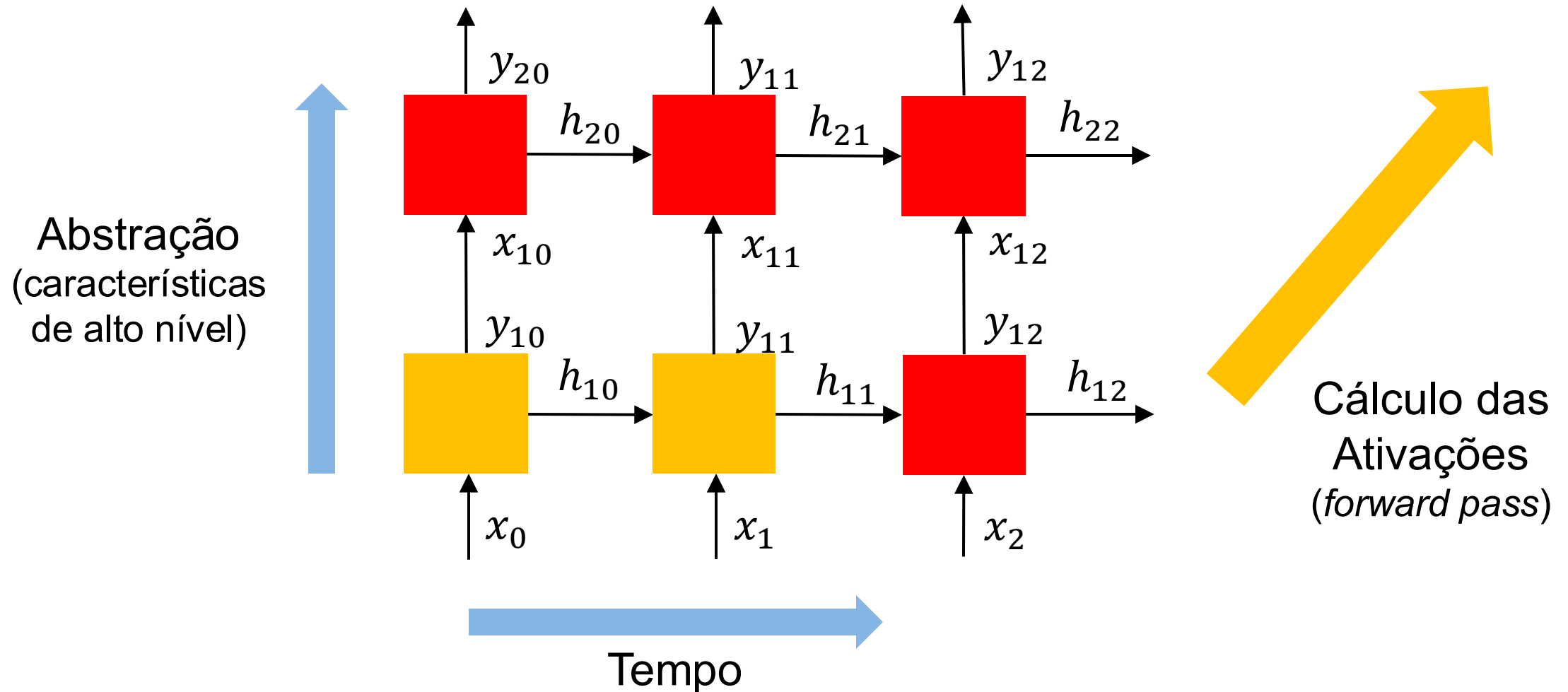
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



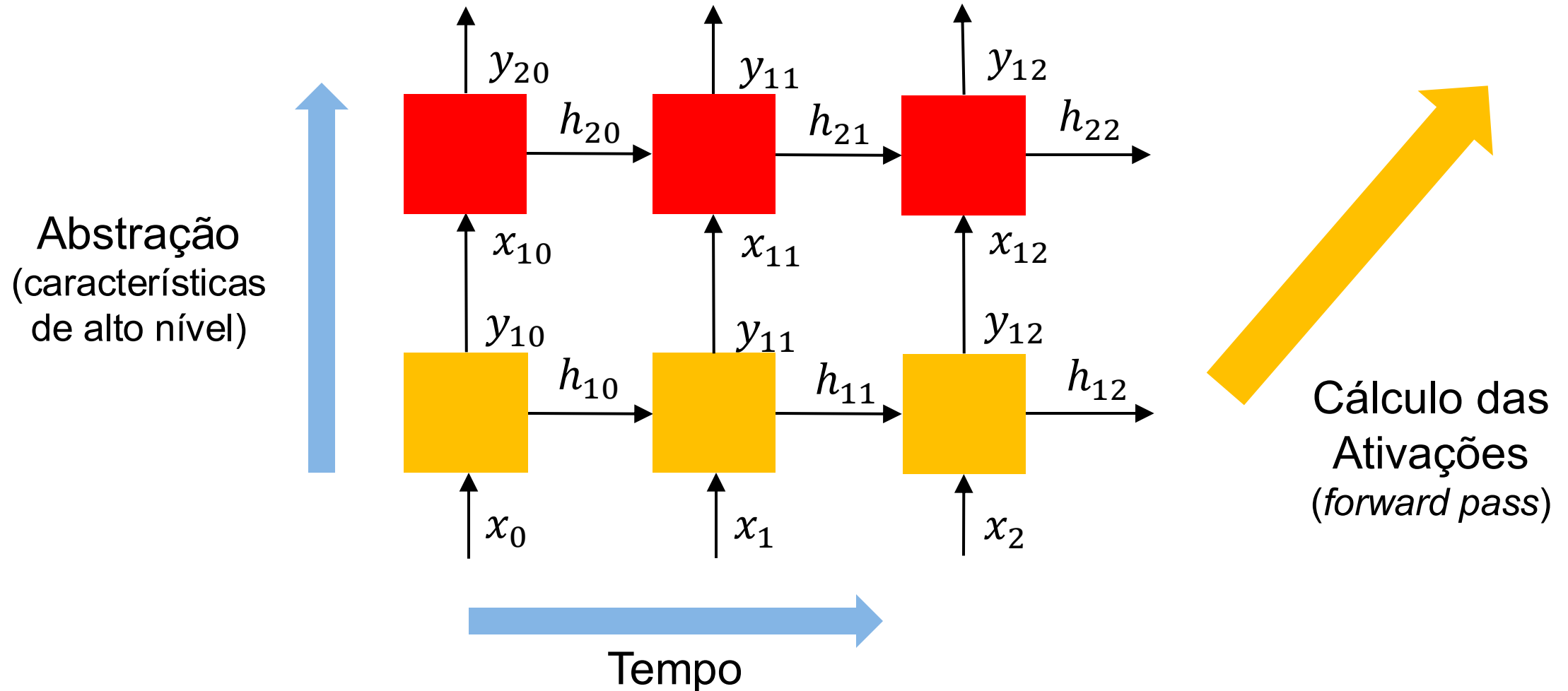
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



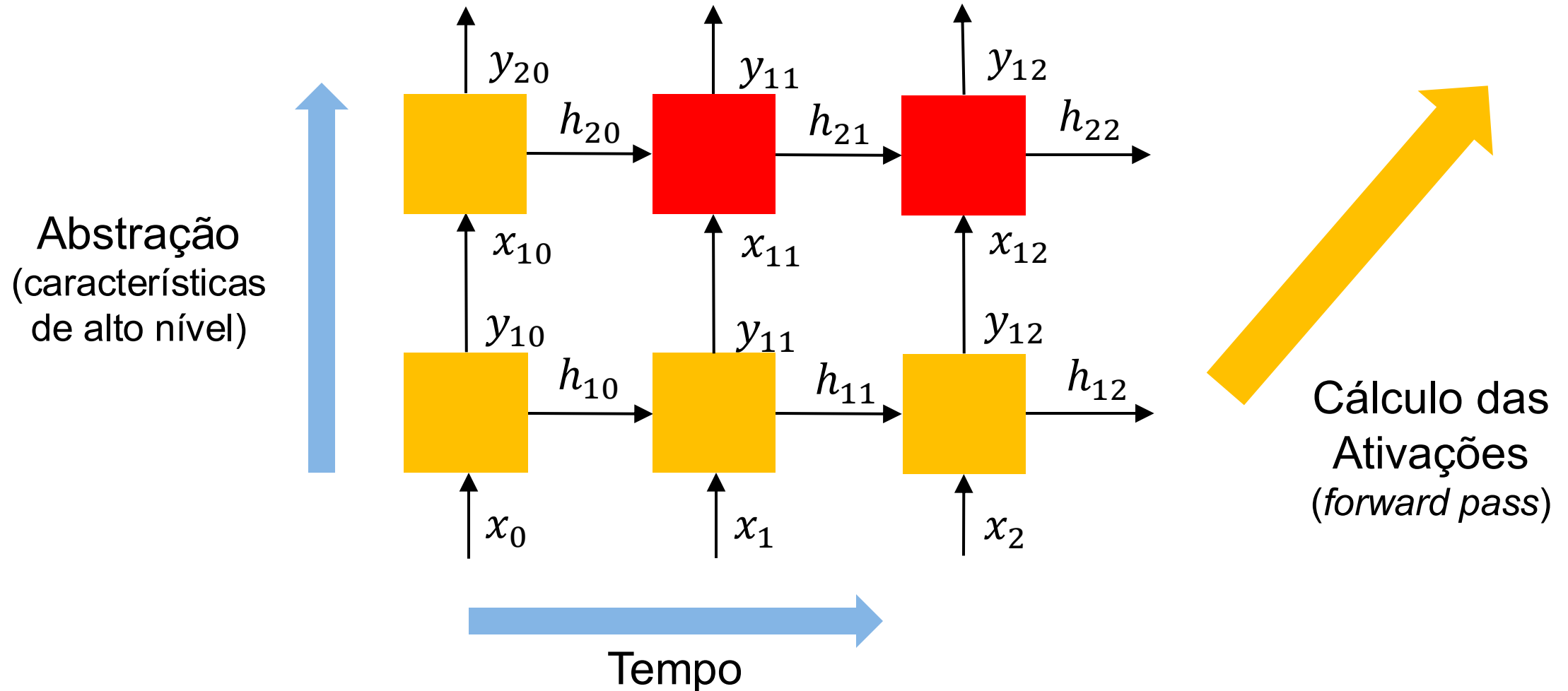
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



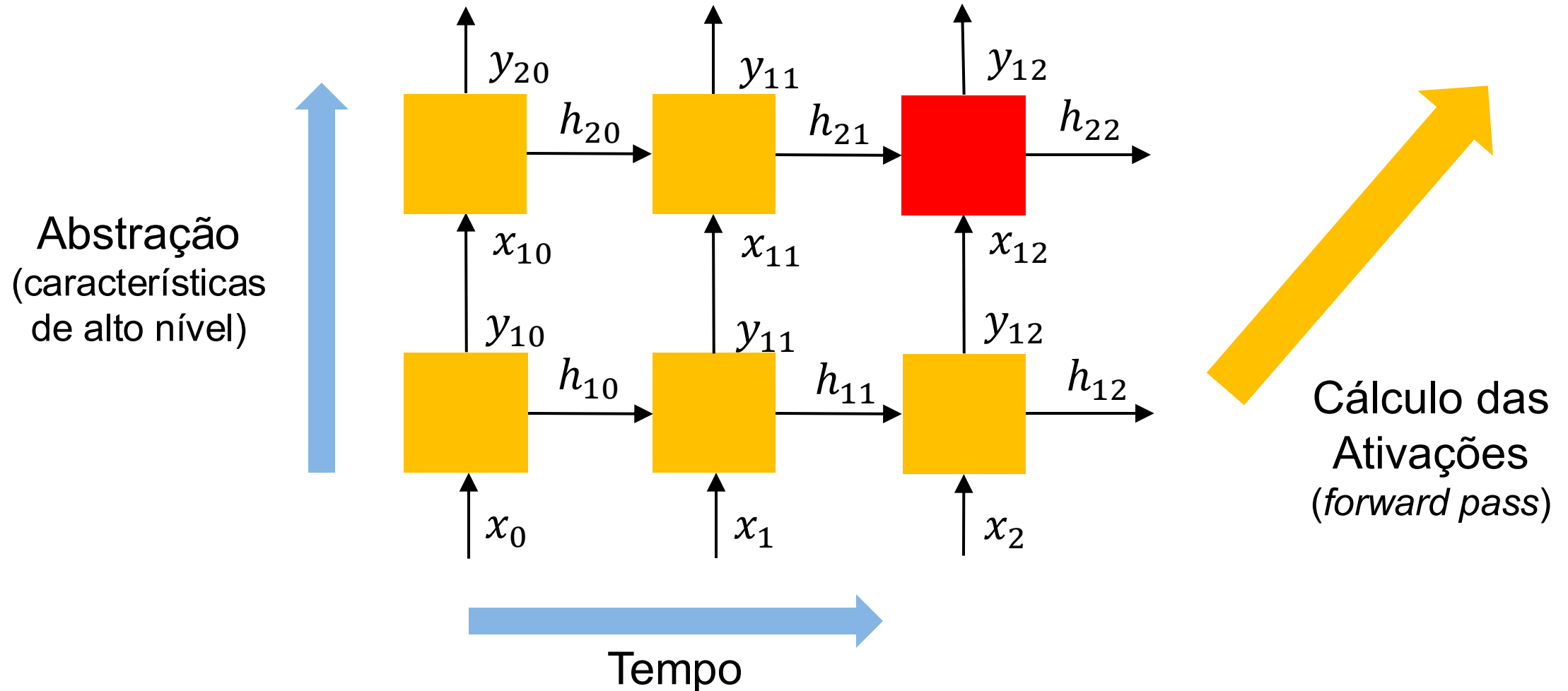
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



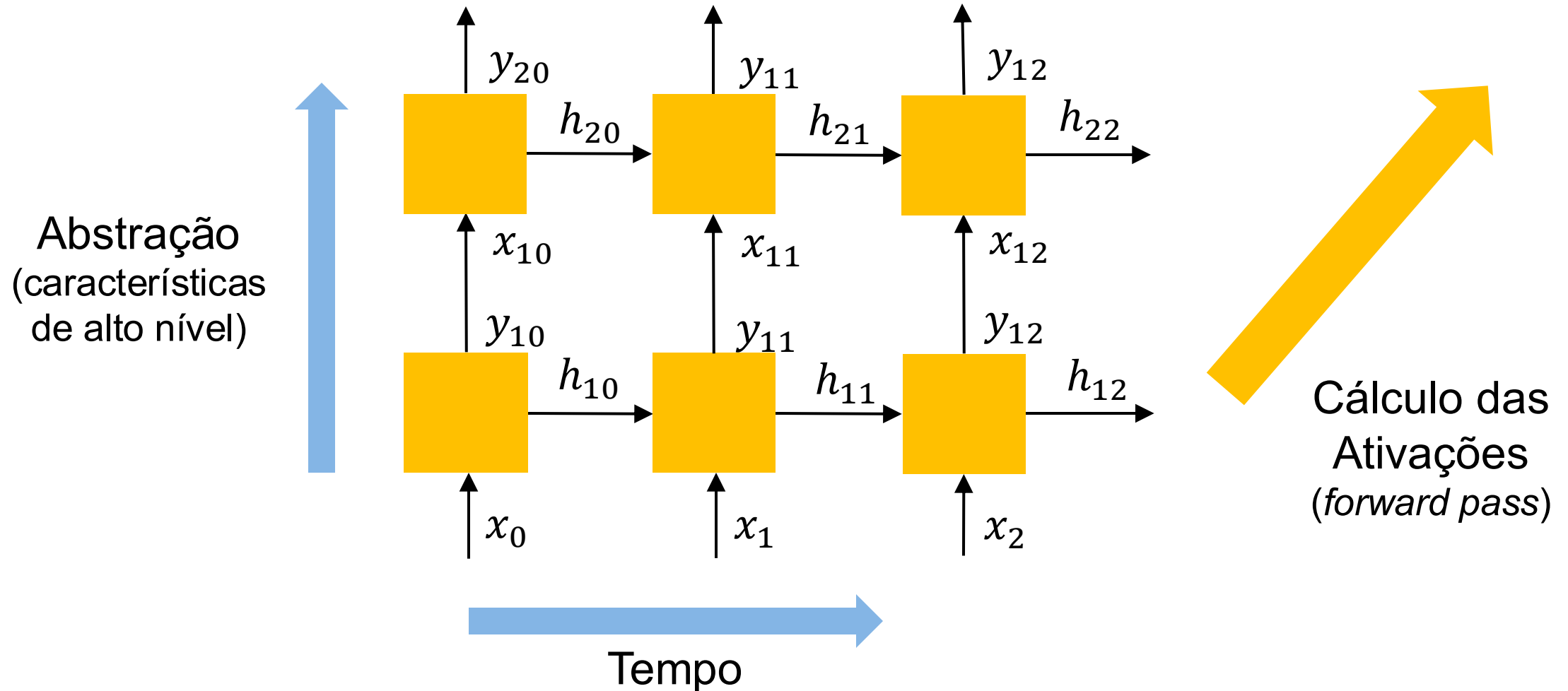
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



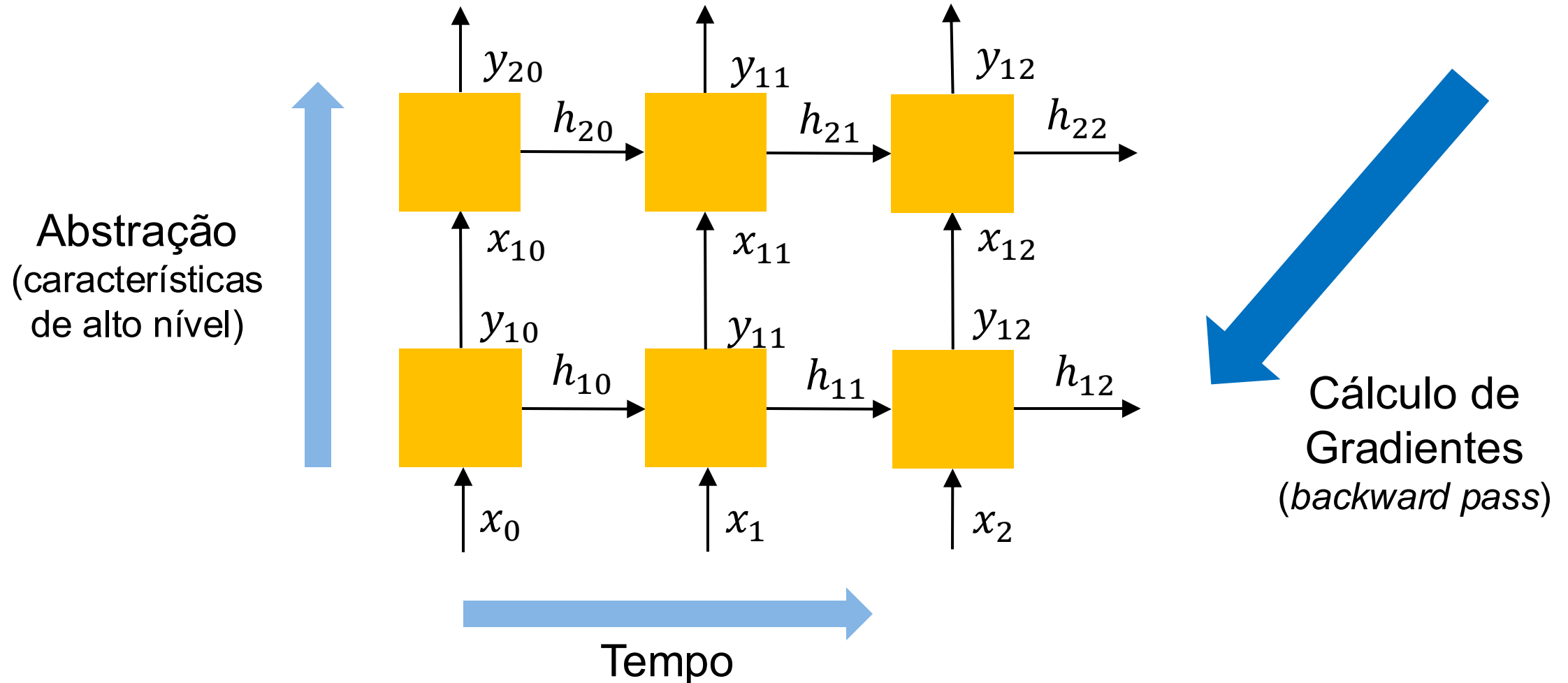
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



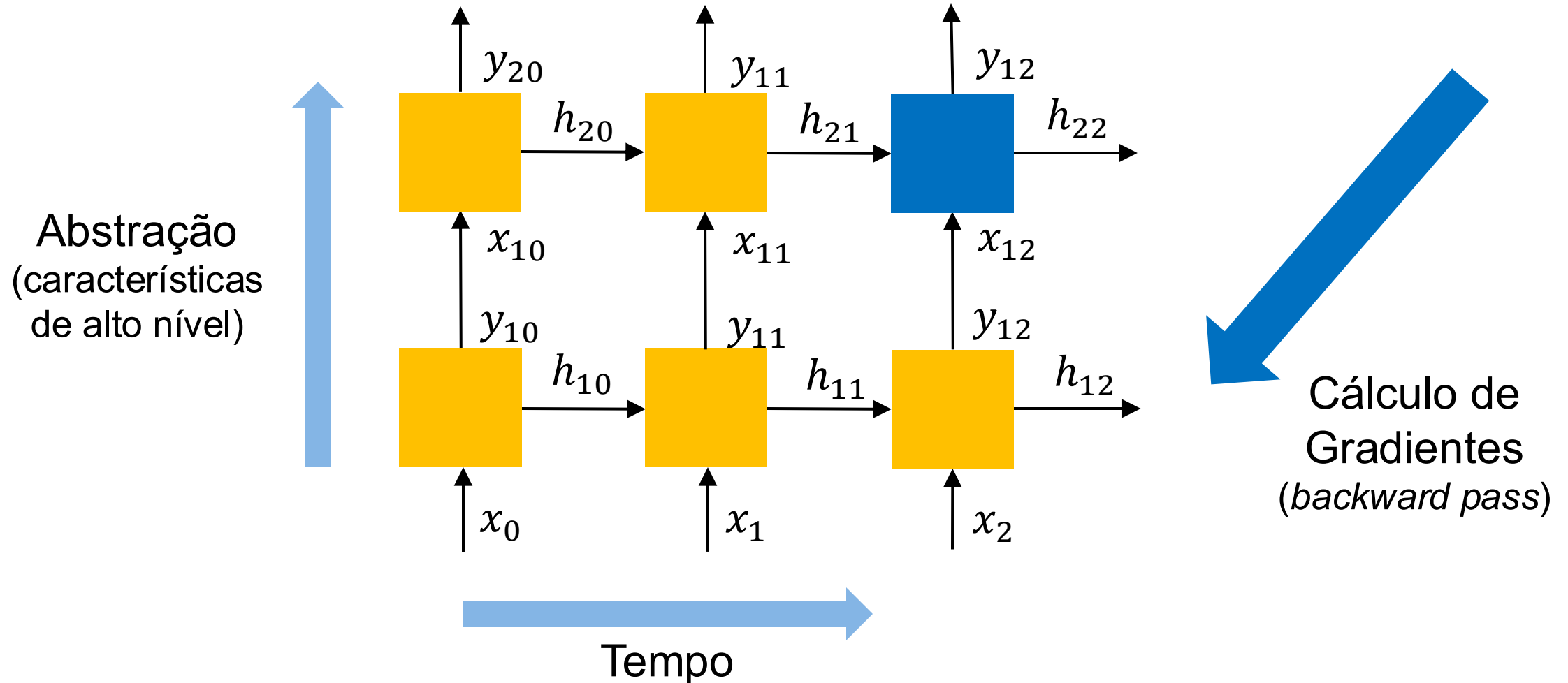
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



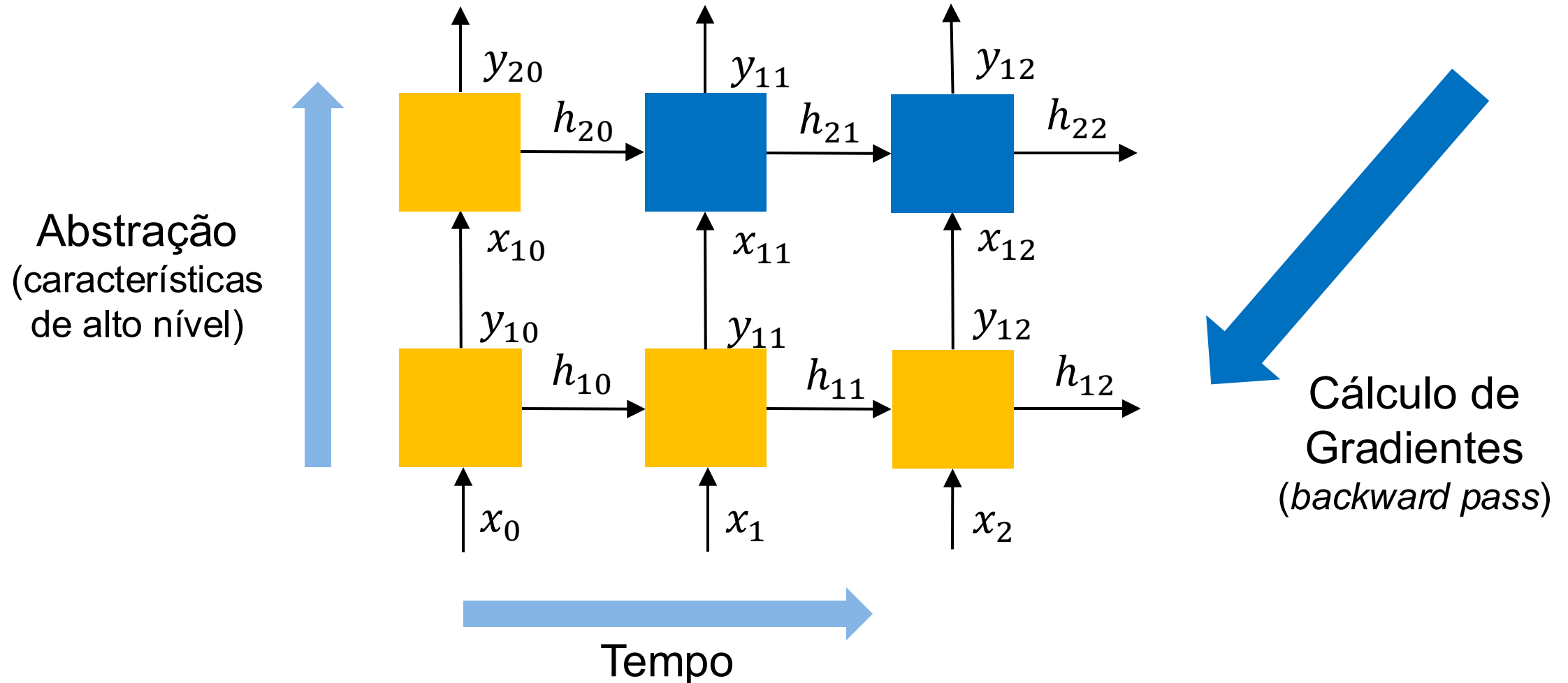
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



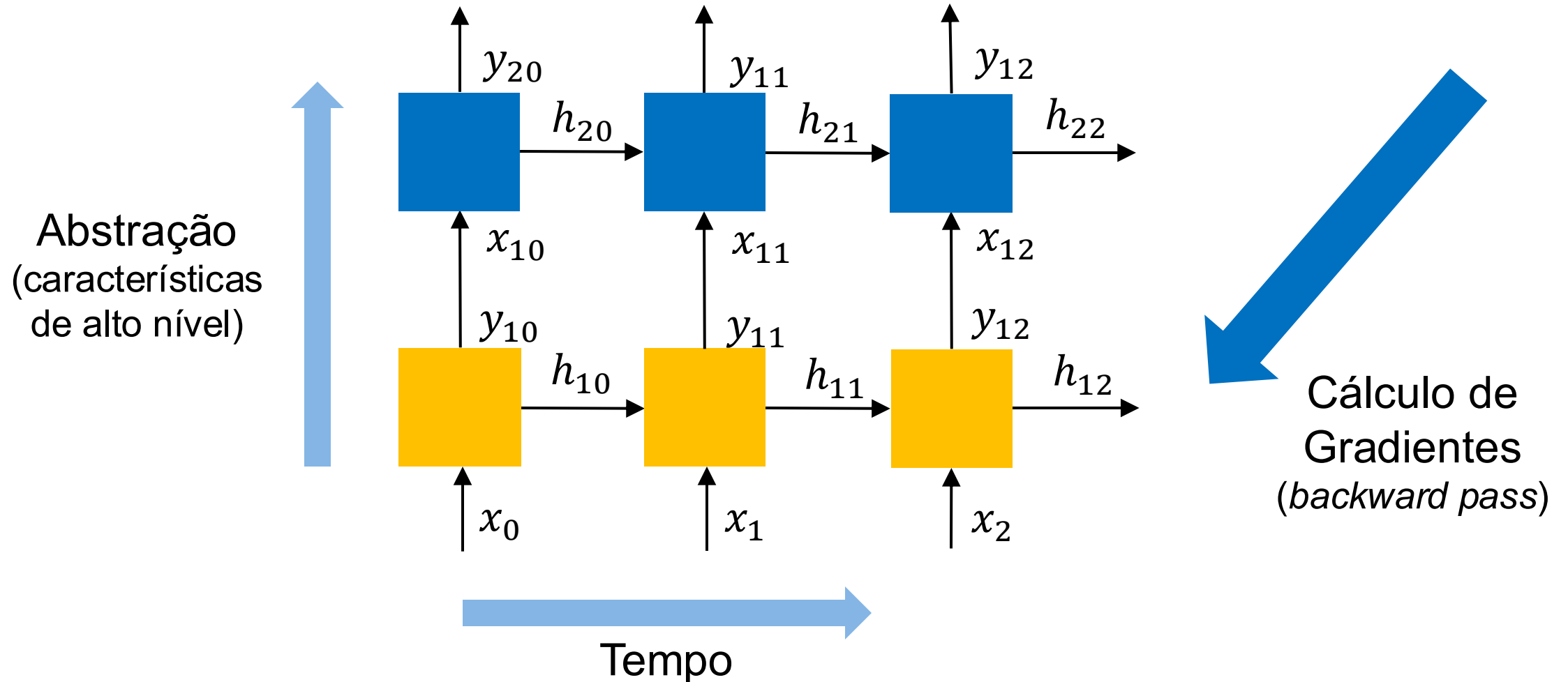
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



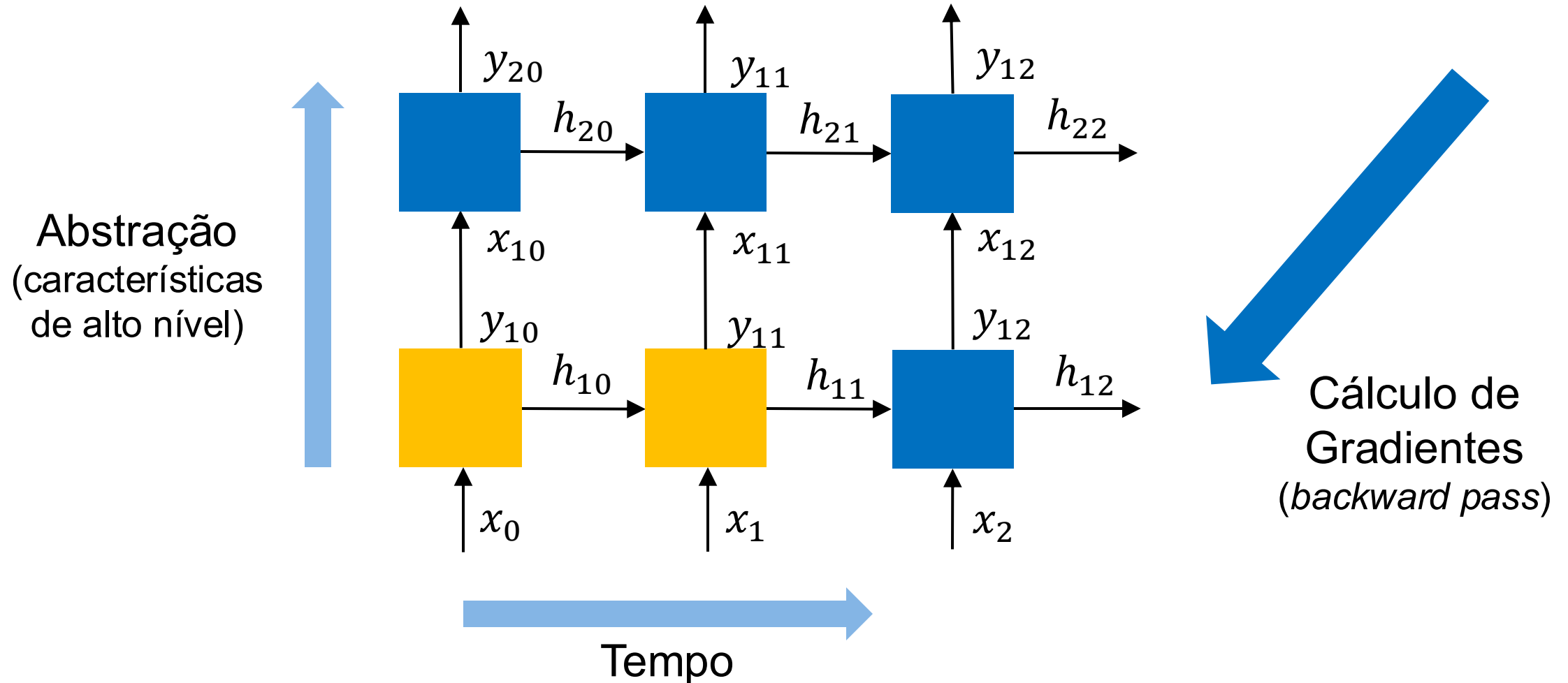
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



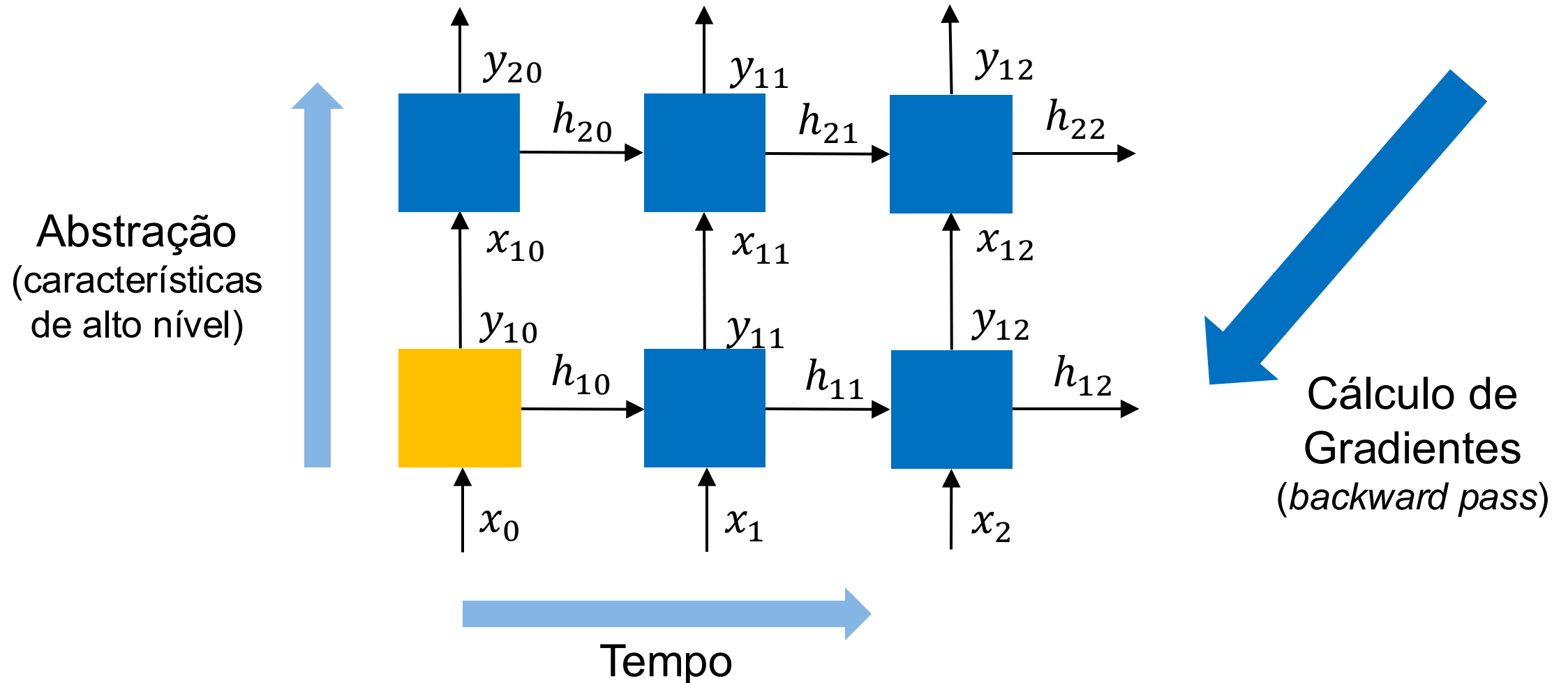
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



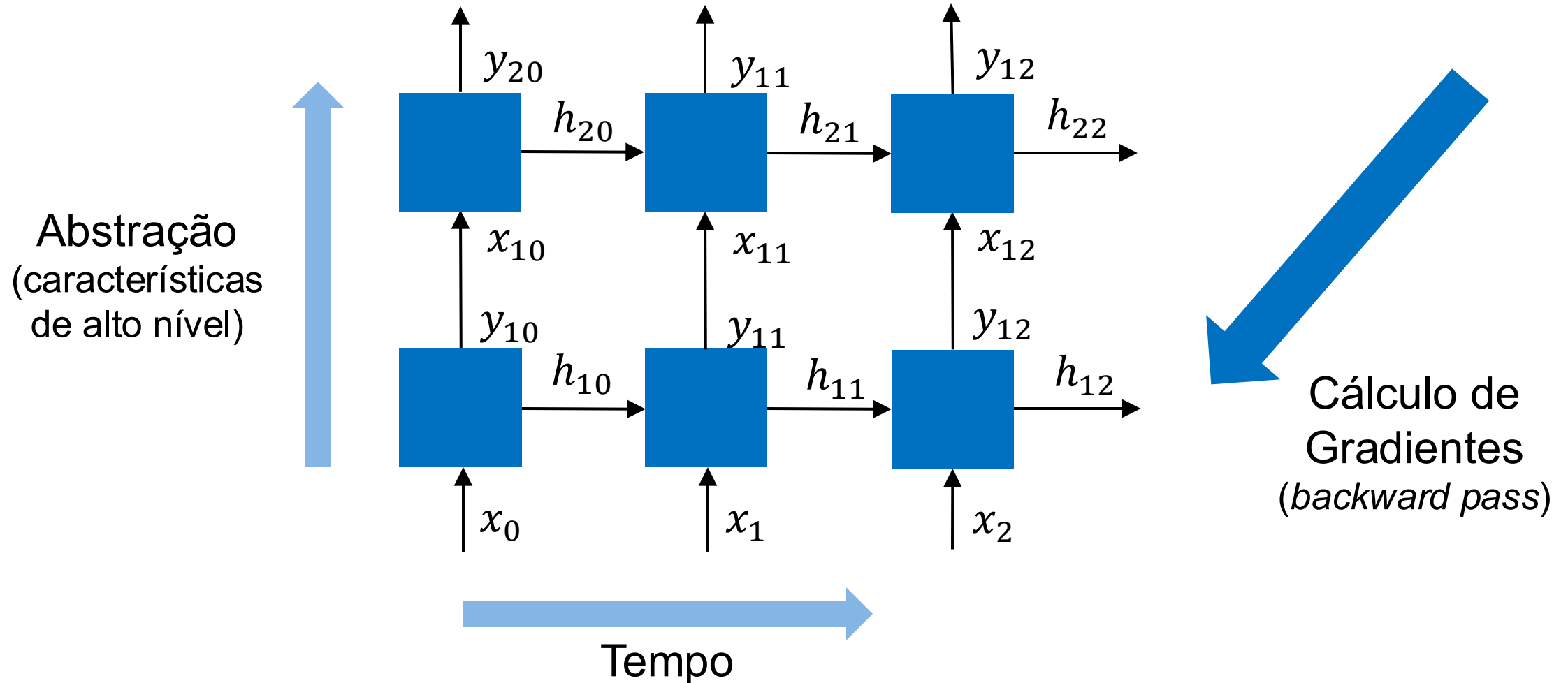
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



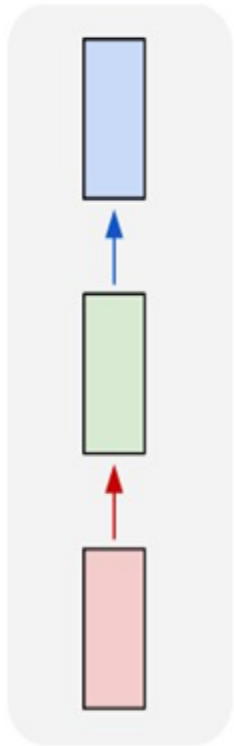
Estrutura de Redes Recorrentes

Método de propagação (*backpropagation*) retrógrada continua funcionando...



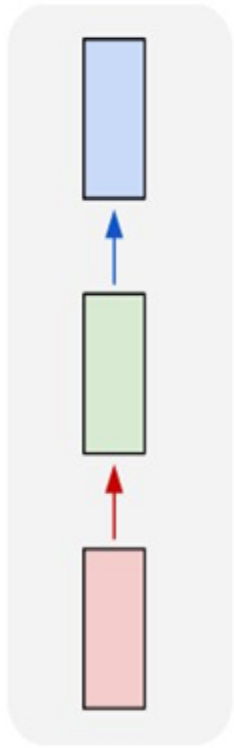
Flexibilidade das Redes Recorrentes

1 – 1



Flexibilidade das Redes Recorrentes

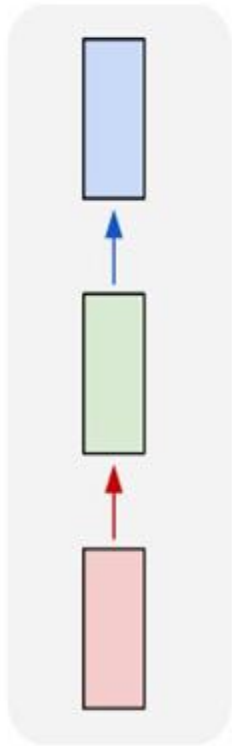
1 – 1



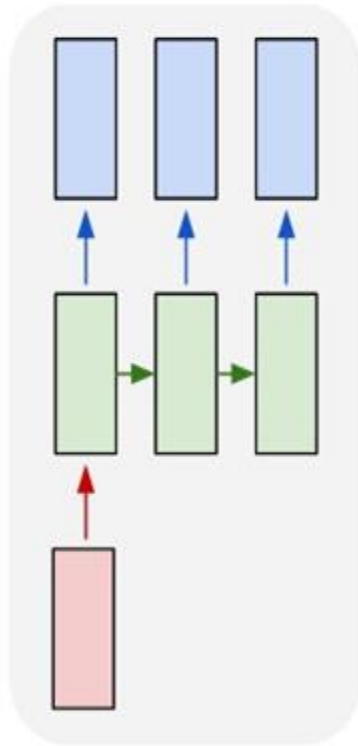
Redes Neurais "Simples"

Flexibilidade das Redes Recorrentes

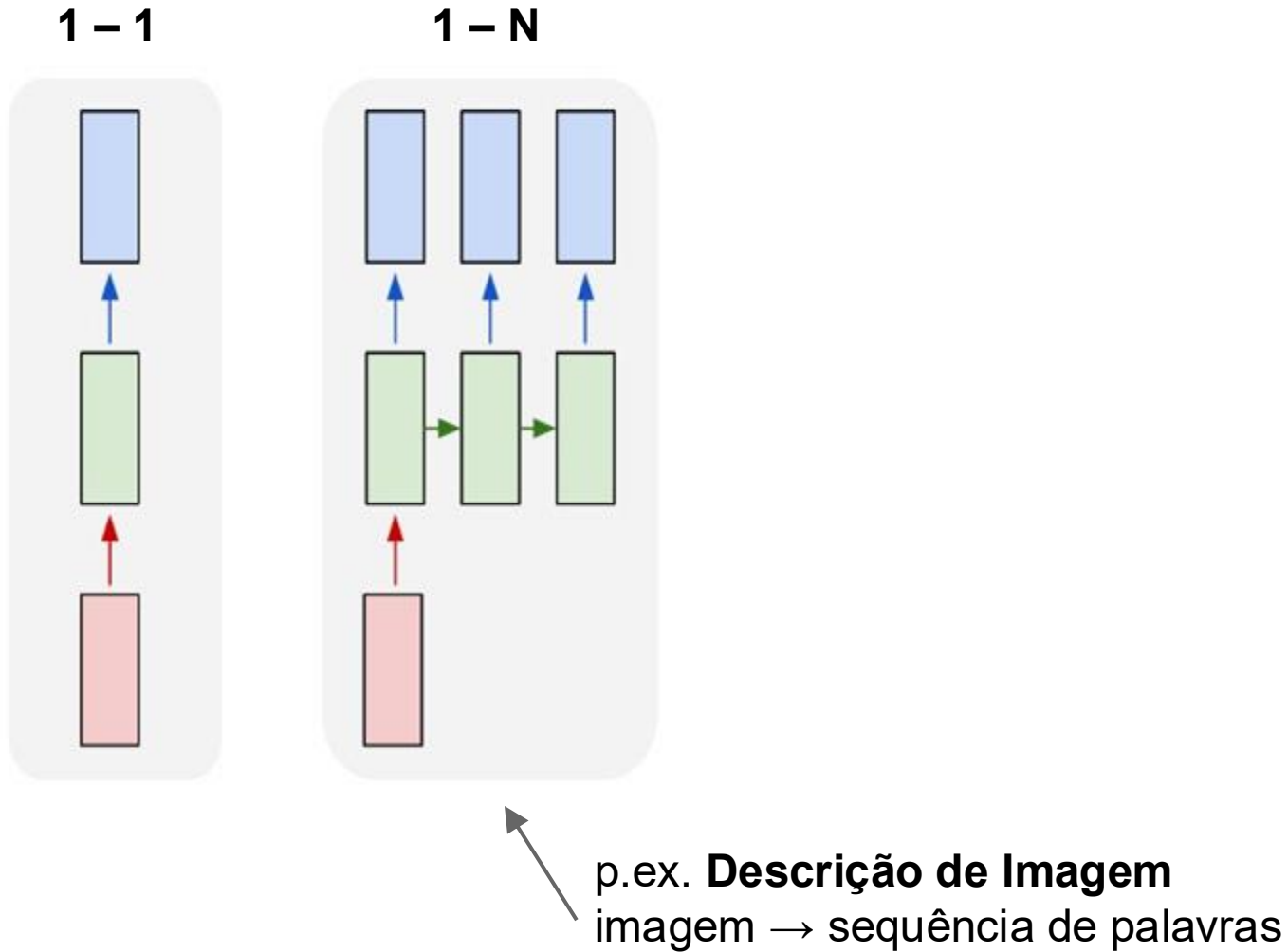
1 – 1



1 – N

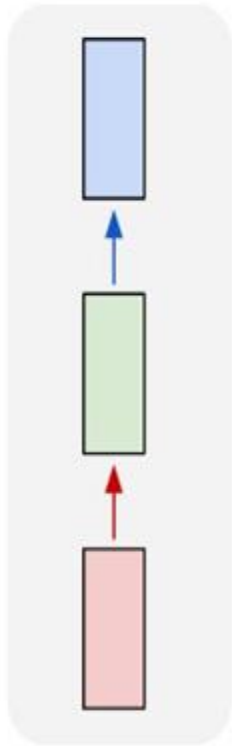


Flexibilidade das Redes Recorrentes

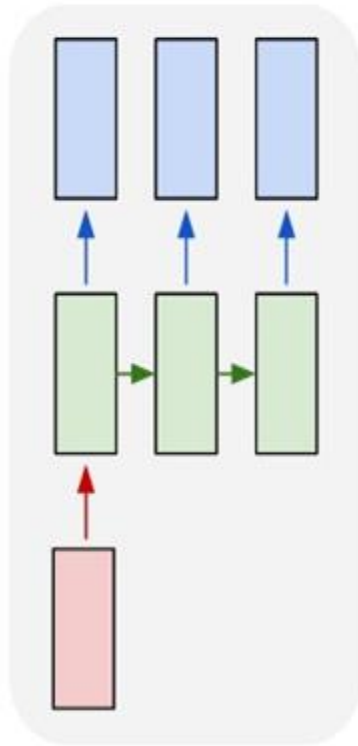


Flexibilidade das Redes Recorrentes

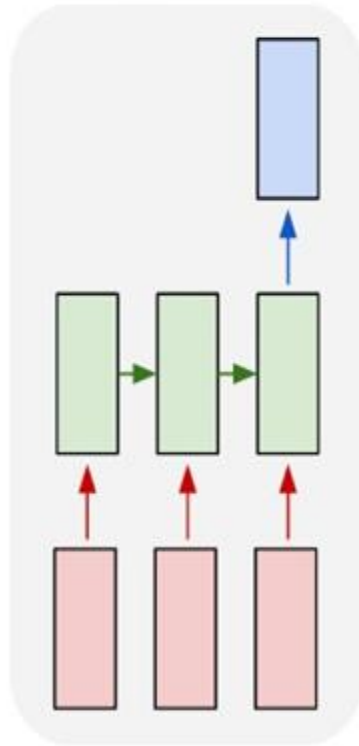
1 – 1



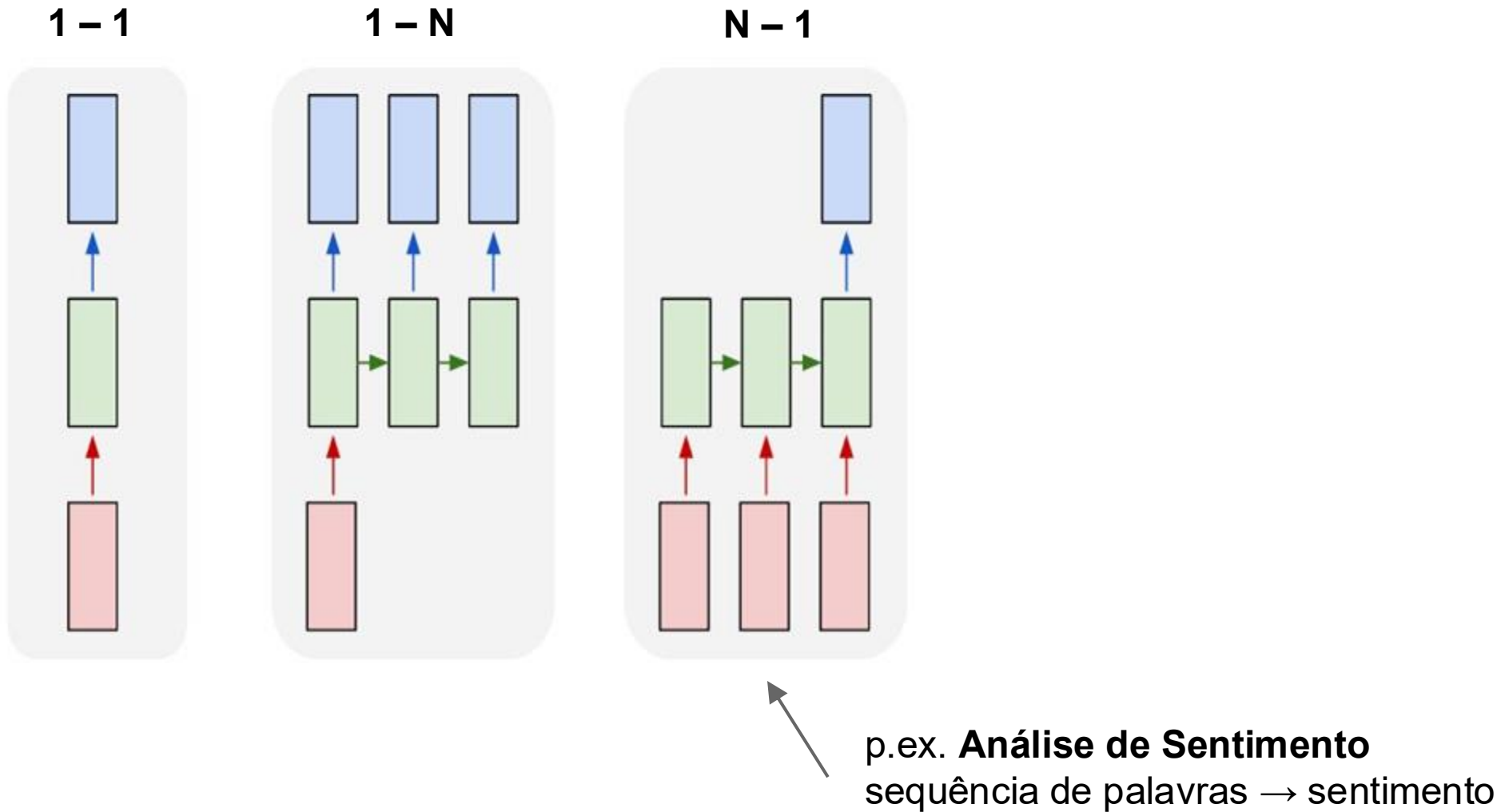
1 – N



N – 1

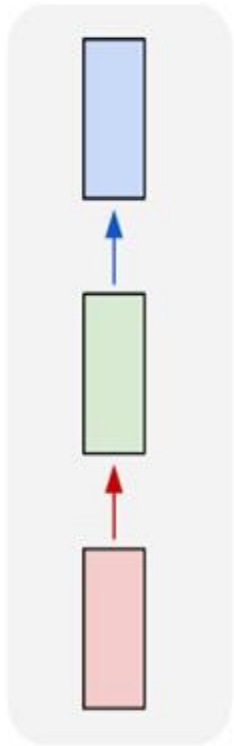


Flexibilidade das Redes Recorrentes

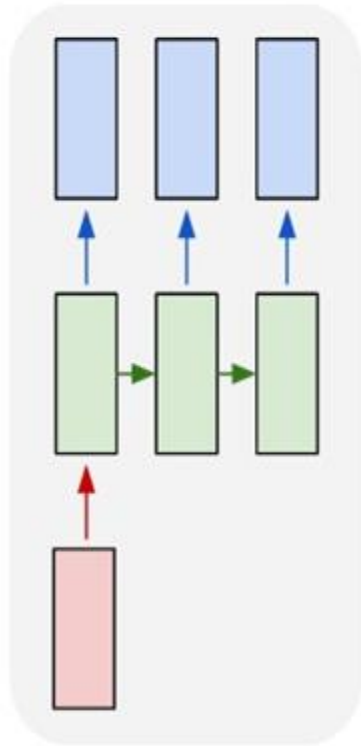


Flexibilidade das Redes Recorrentes

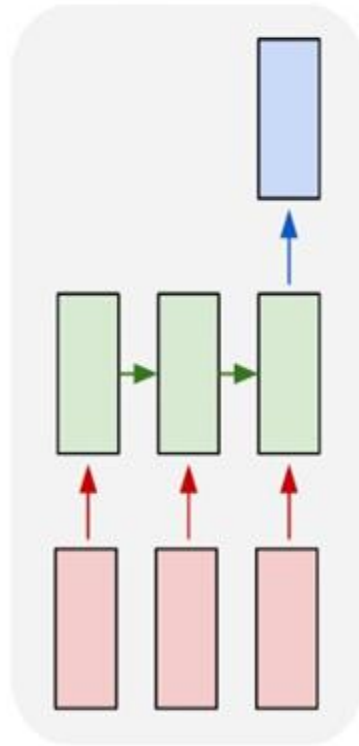
1 – 1



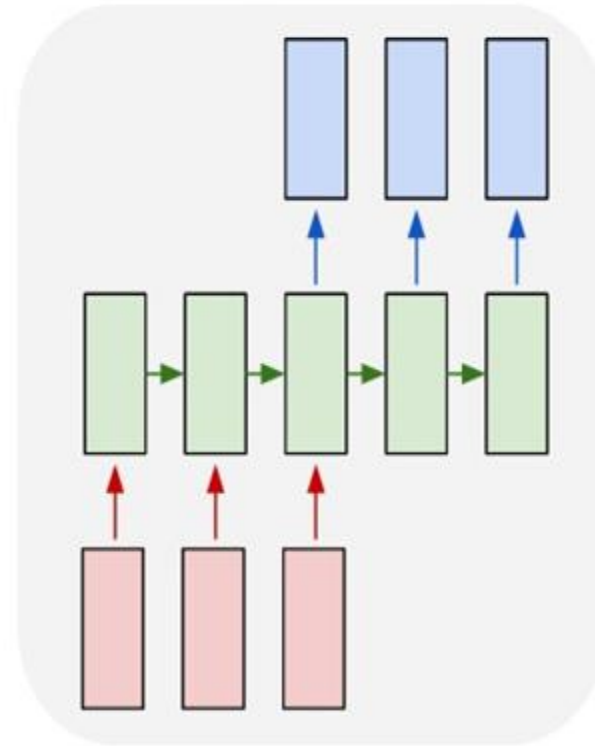
1 – N



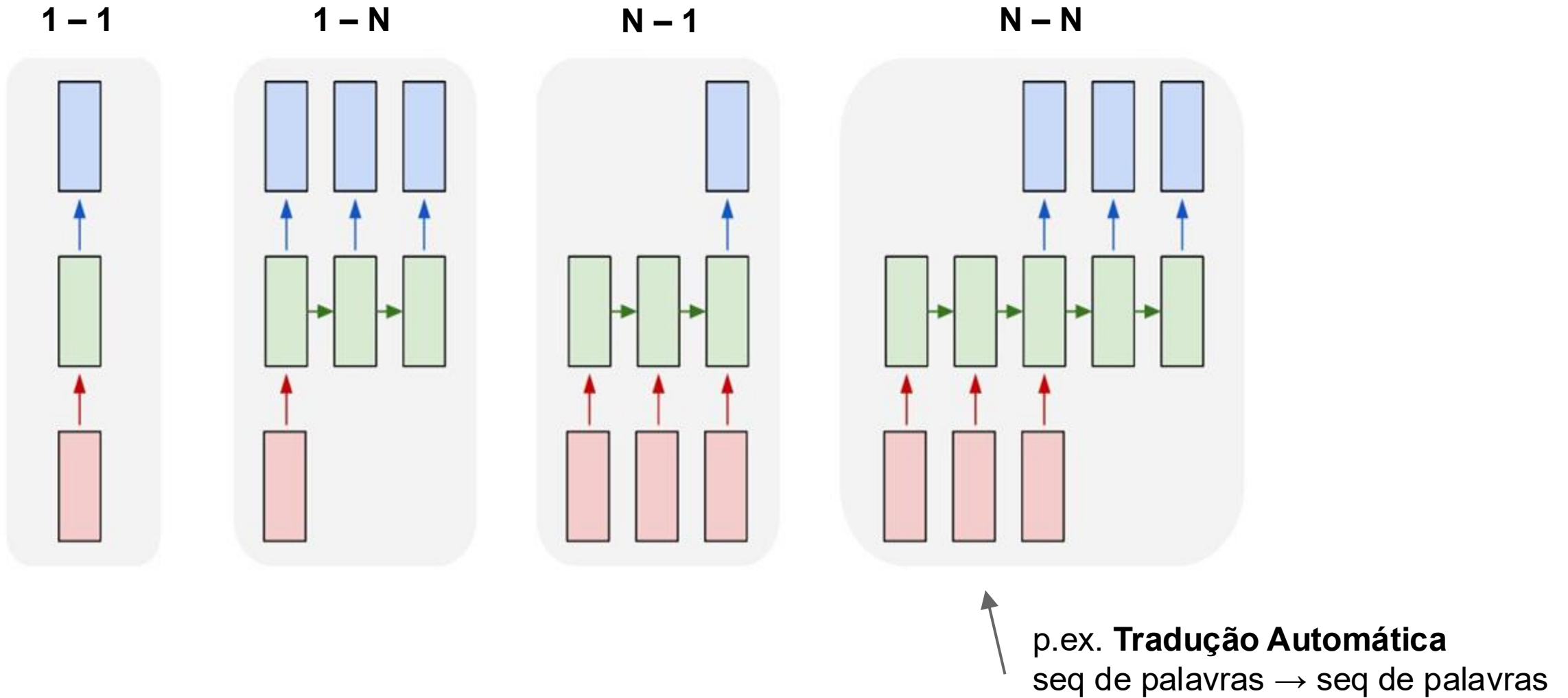
N – 1



N – N

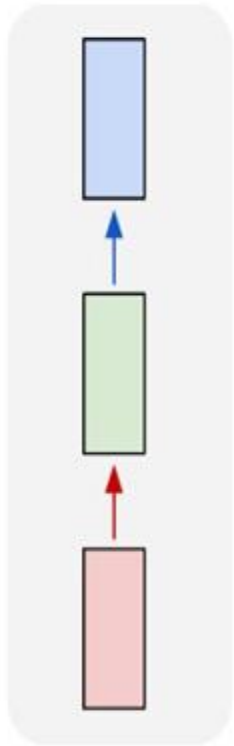


Flexibilidade das Redes Recorrentes

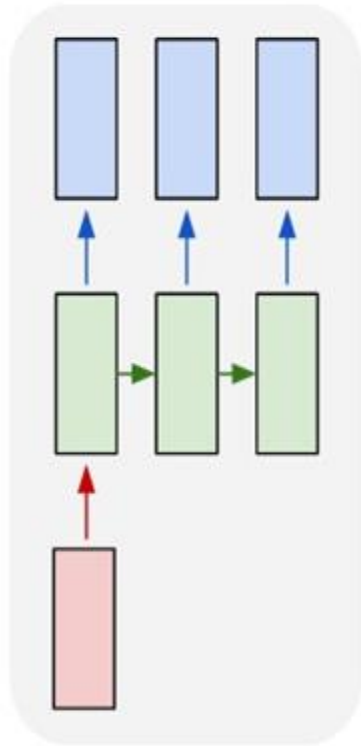


Flexibilidade das Redes Recorrentes

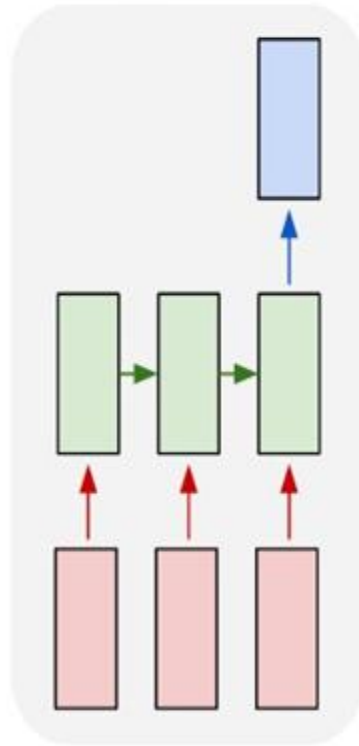
1 – 1



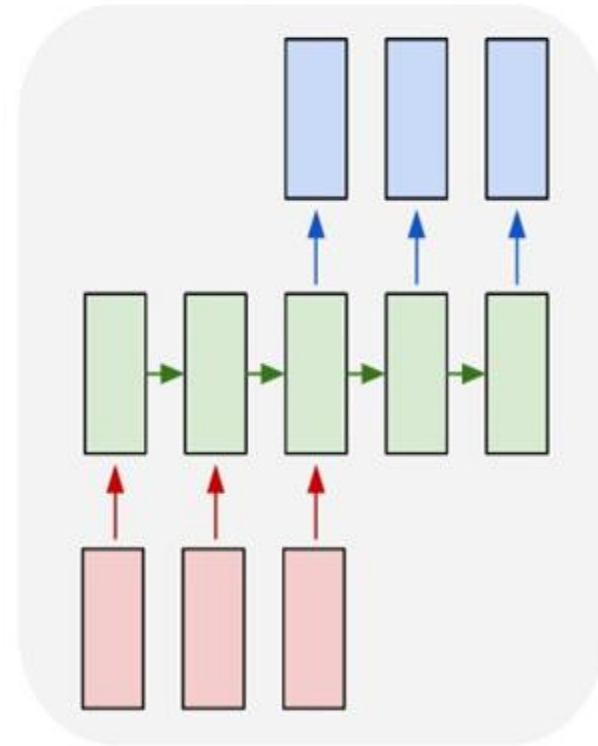
1 – N



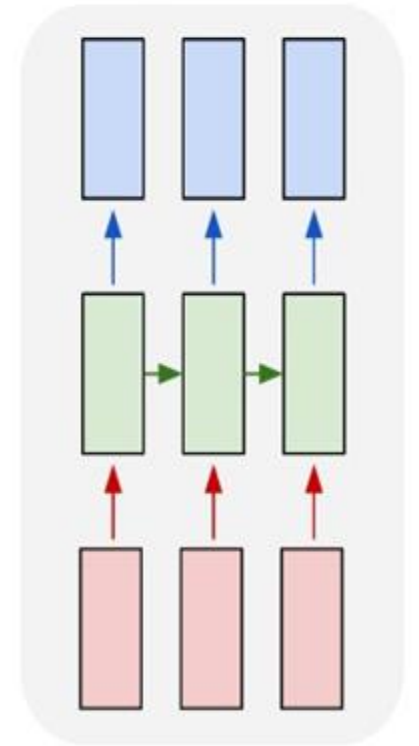
N – 1



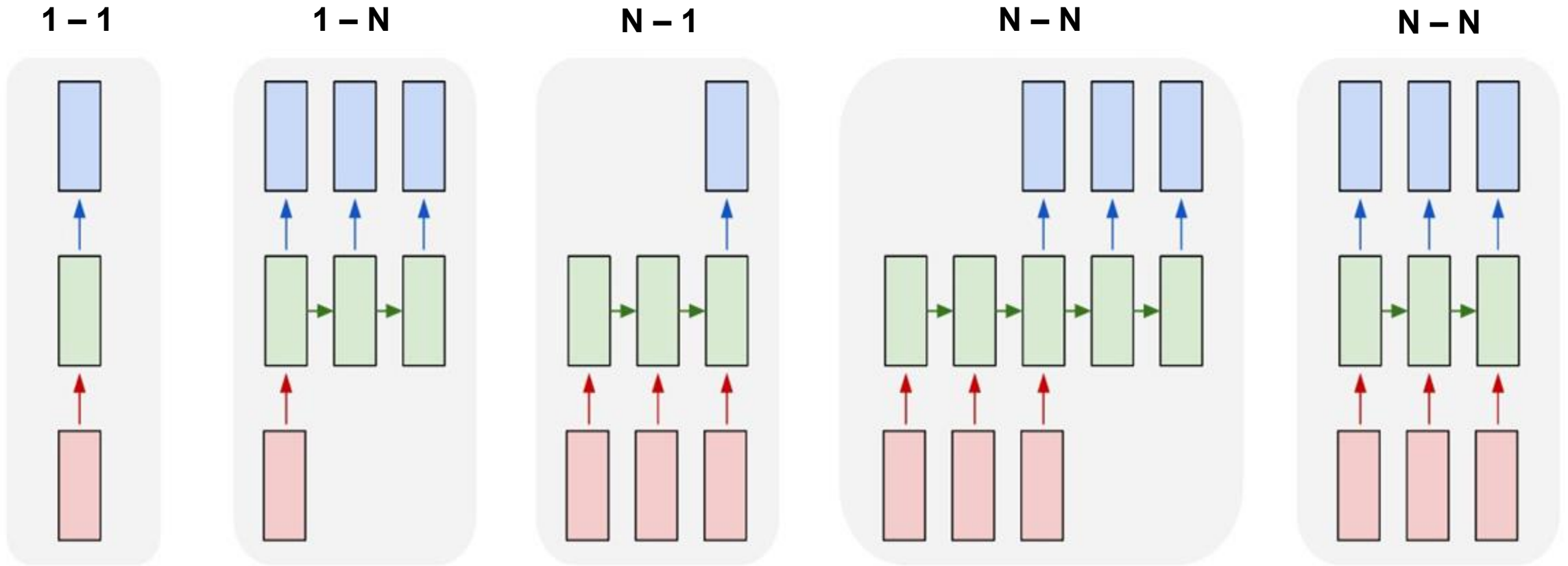
N – N



N – N



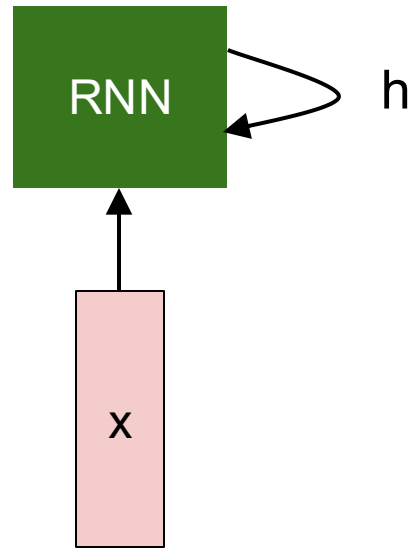
Flexibilidade das Redes Recorrentes



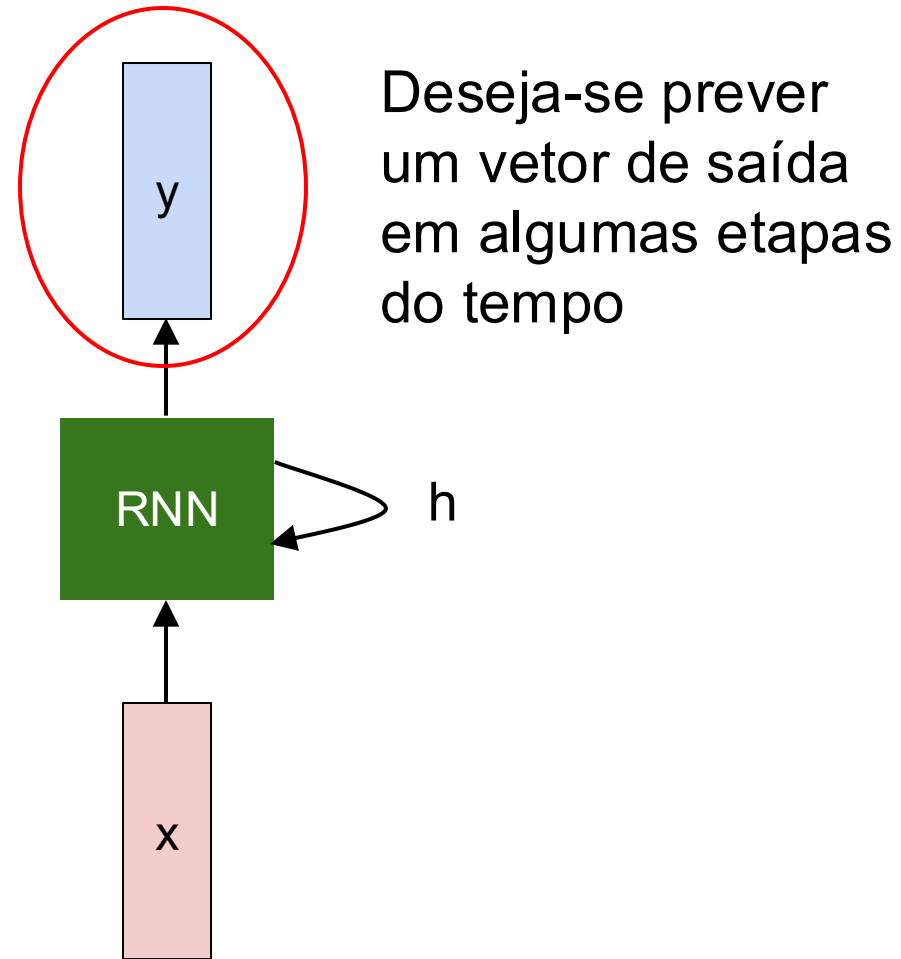
p.ex. **Classificação de vídeo quadro a quadro**
seq de quadros → seq de classificações



Funcionamento de Rede Neural Recorrente (RNN)

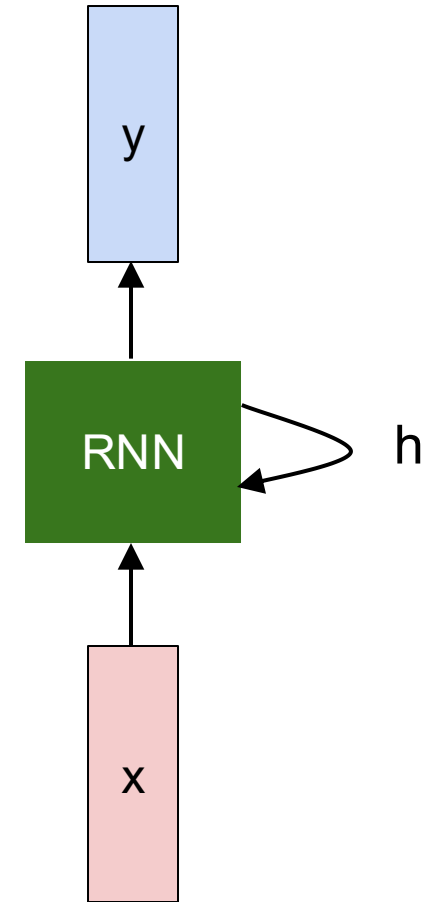


Funcionamento de Rede Neural Recorrente (RNN)



Funcionamento de Rede Neural Recorrente (RNN)

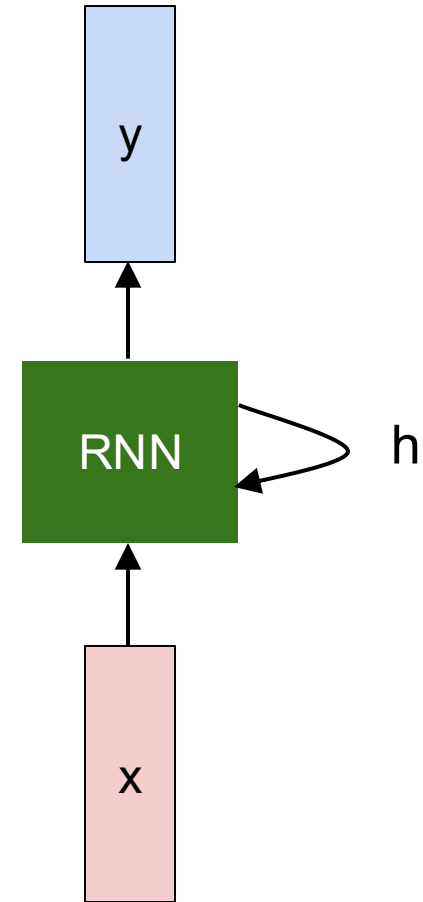
Pode-se processar a sequência de vetores de entrada x pela aplicação de uma fórmula de recorrência a cada passo (ou instante de tempo)



Funcionamento de Rede Neural Recorrente (RNN)

Pode-se processar a sequência de vetores de entrada $\mathbf{x}$ pela aplicação de uma fórmula de recorrência a cada passo (ou instante de tempo)

$$h_t = f_W(h_{t-1}, x_t)$$

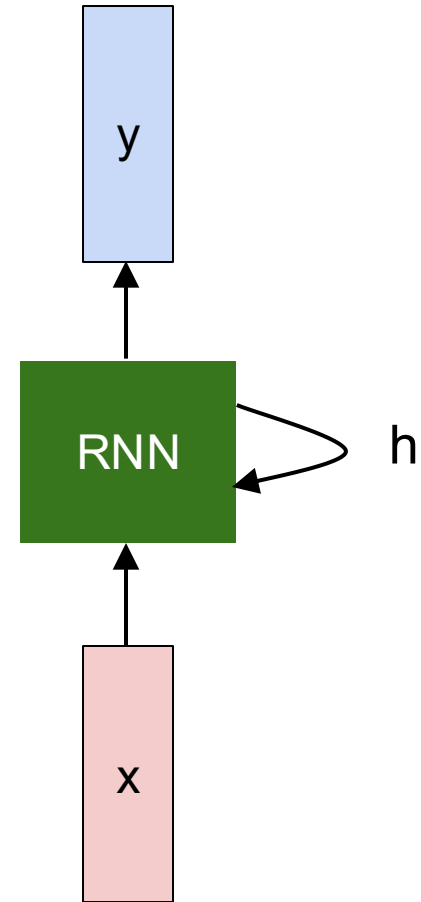


Funcionamento de Rede Neural Recorrente (RNN)

Pode-se processar a sequência de vetores de entrada $\mathbf{x}$ pela aplicação de uma fórmula de recorrência a cada passo (ou instante de tempo)

$$h_t = f_W(h_{t-1}, x_t)$$

Vetor de entrada
em um passo



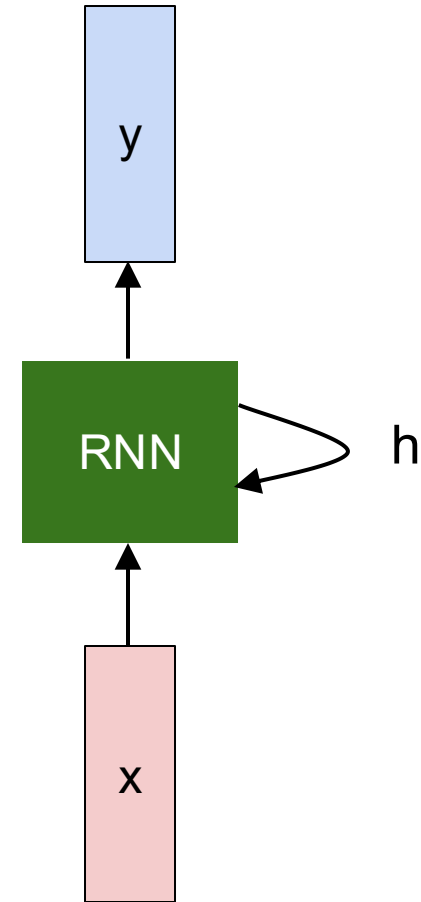
Funcionamento de Rede Neural Recorrente (RNN)

Pode-se processar a sequência de vetores de entrada $\mathbf{x}$ pela aplicação de uma fórmula de recorrência a cada passo (ou instante de tempo)

$$h_t = f_W(h_{t-1}, x_t)$$

Estado
Anterior

Vetor de entrada
em um passo



Funcionamento de Rede Neural Recorrente (RNN)

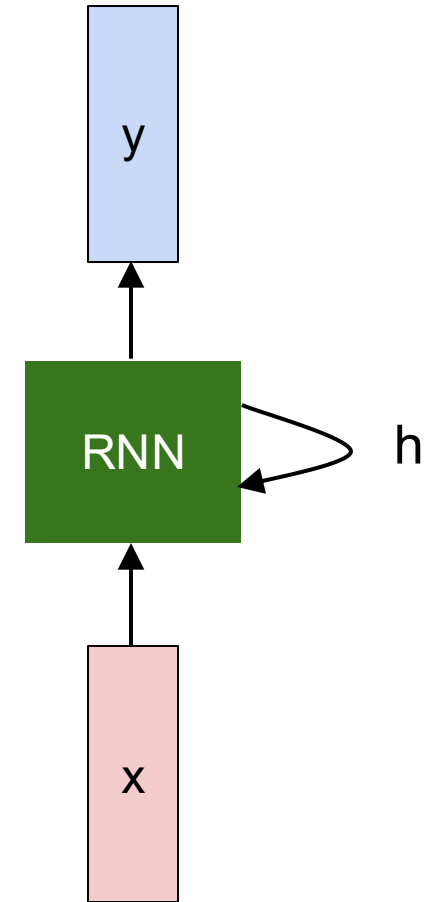
Pode-se processar a sequência de vetores de entrada $\mathbf{x}$ pela aplicação de uma fórmula de recorrência a cada passo (ou instante de tempo)

$$h_t = f_W(h_{t-1}, x_t)$$

Estado
Anterior

Vetor de entrada
em um passo

Alguma função com
parâmetros W



Funcionamento de Rede Neural Recorrente (RNN)

Pode-se processar a sequência de vetores de entrada $\mathbf{x}$ pela aplicação de uma fórmula de recorrência a cada passo (ou instante de tempo)

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

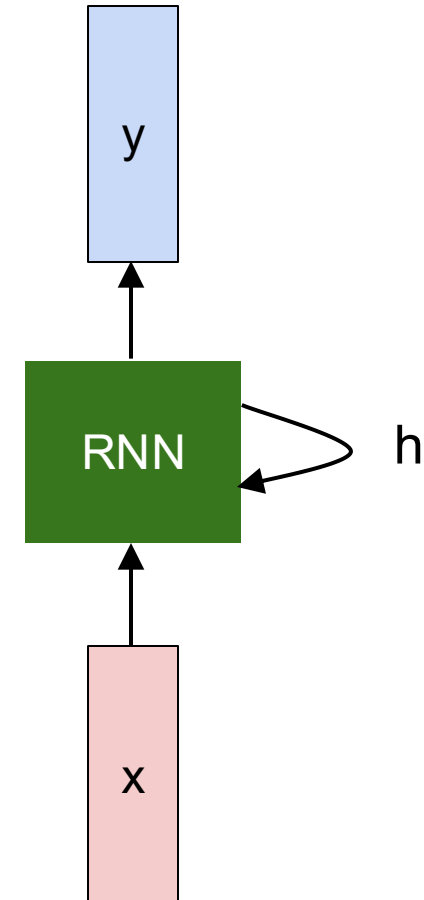
Estado Novo

Alguma função com parâmetros W

Estado Anterior

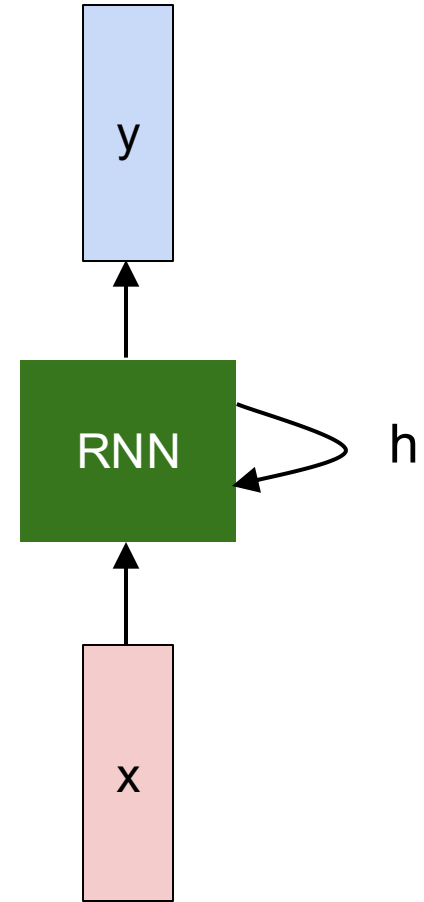
Vetor de entrada em um passo

OBS: a mesma função e o mesmo conjunto de parâmetros são usados em todos os passos



Funcionamento de Rede Neural Recorrente (RNN)

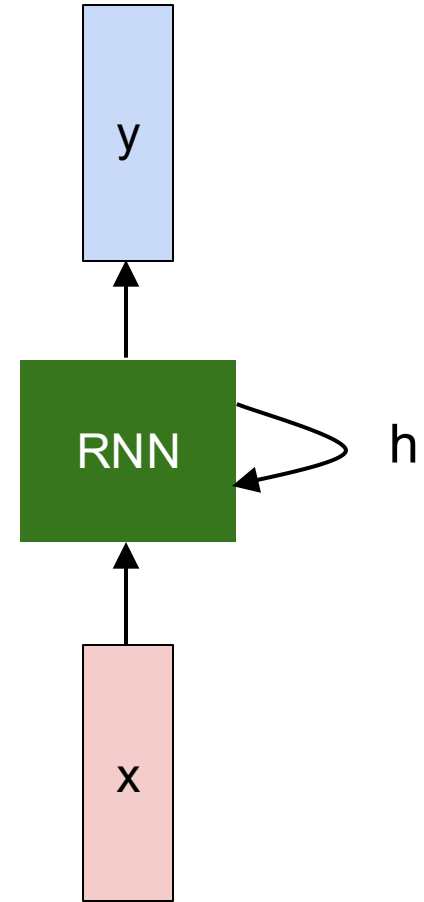
A informação de estado consiste de um único vetor
“escondido” h



Funcionamento de Rede Neural Recorrente (RNN)

A informação de estado consiste de um único vetor “escondido” **h**

$$h_t = f_W(h_{t-1}, x_t)$$



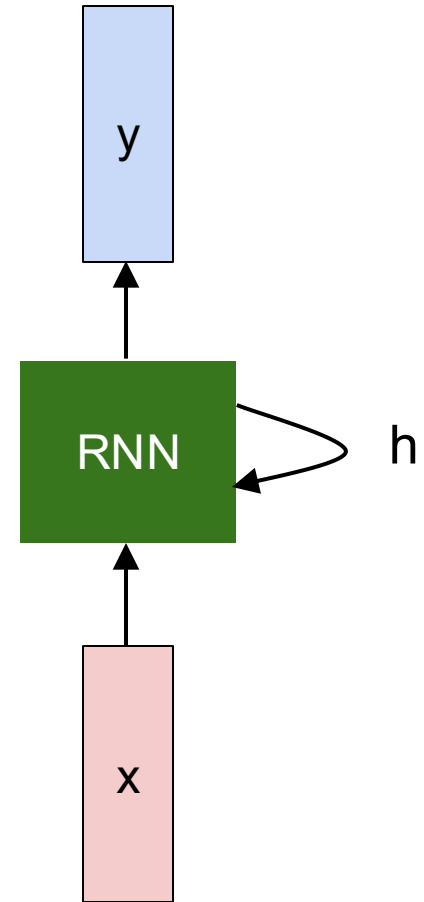
Funcionamento de Rede Neural Recorrente (RNN)

A informação de estado consiste de um único vetor “escondido” h

$$h_t = f_W(h_{t-1}, x_t)$$



$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



Funcionamento de Rede Neural Recorrente (RNN)

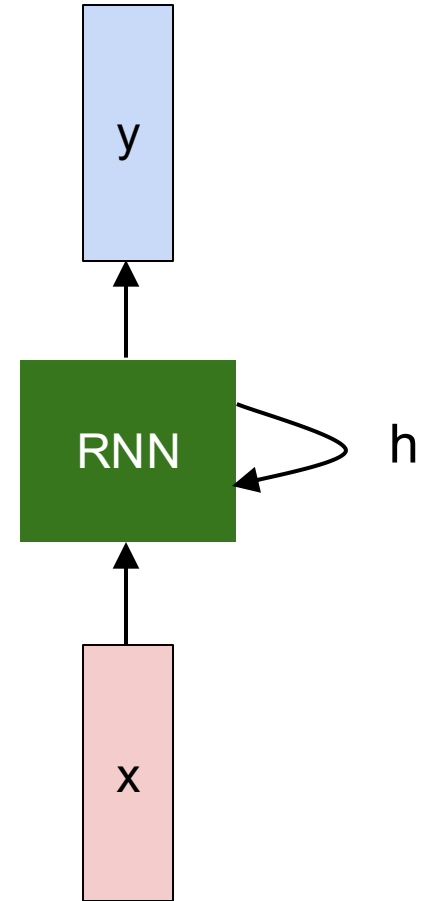
A informação de estado consiste de um único vetor “escondido” h

$$h_t = f_W(h_{t-1}, x_t)$$



$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

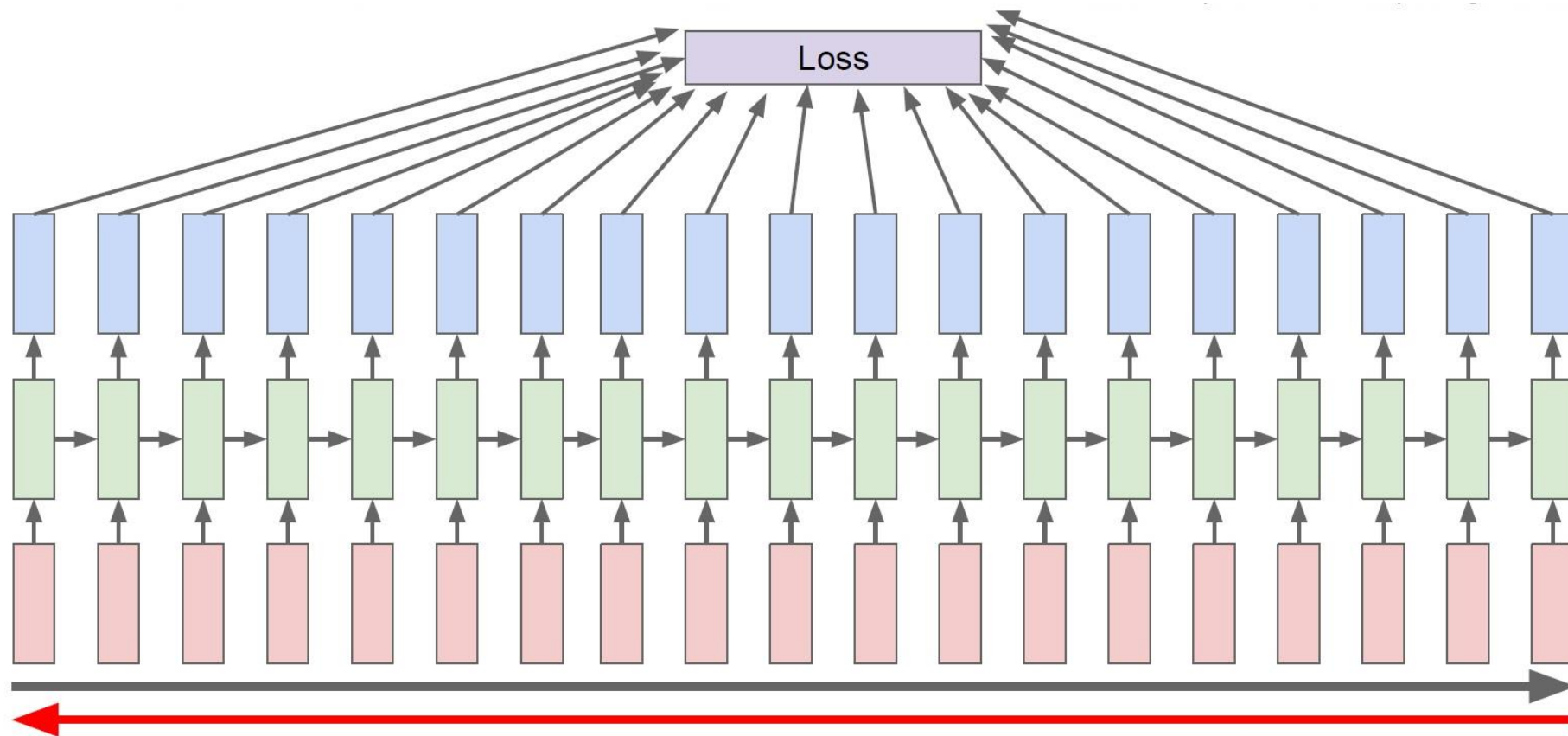
$$y_t = W_{hy}h_t$$



BPTT – *Backpropagation Through Time*

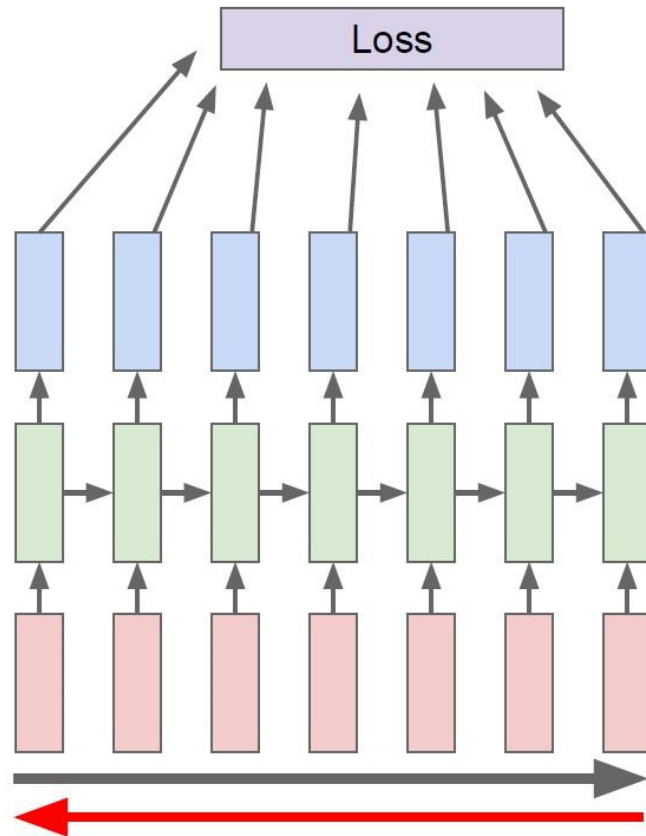
Forward pass → Toda a sequência

Backward pass → Toda a sequência



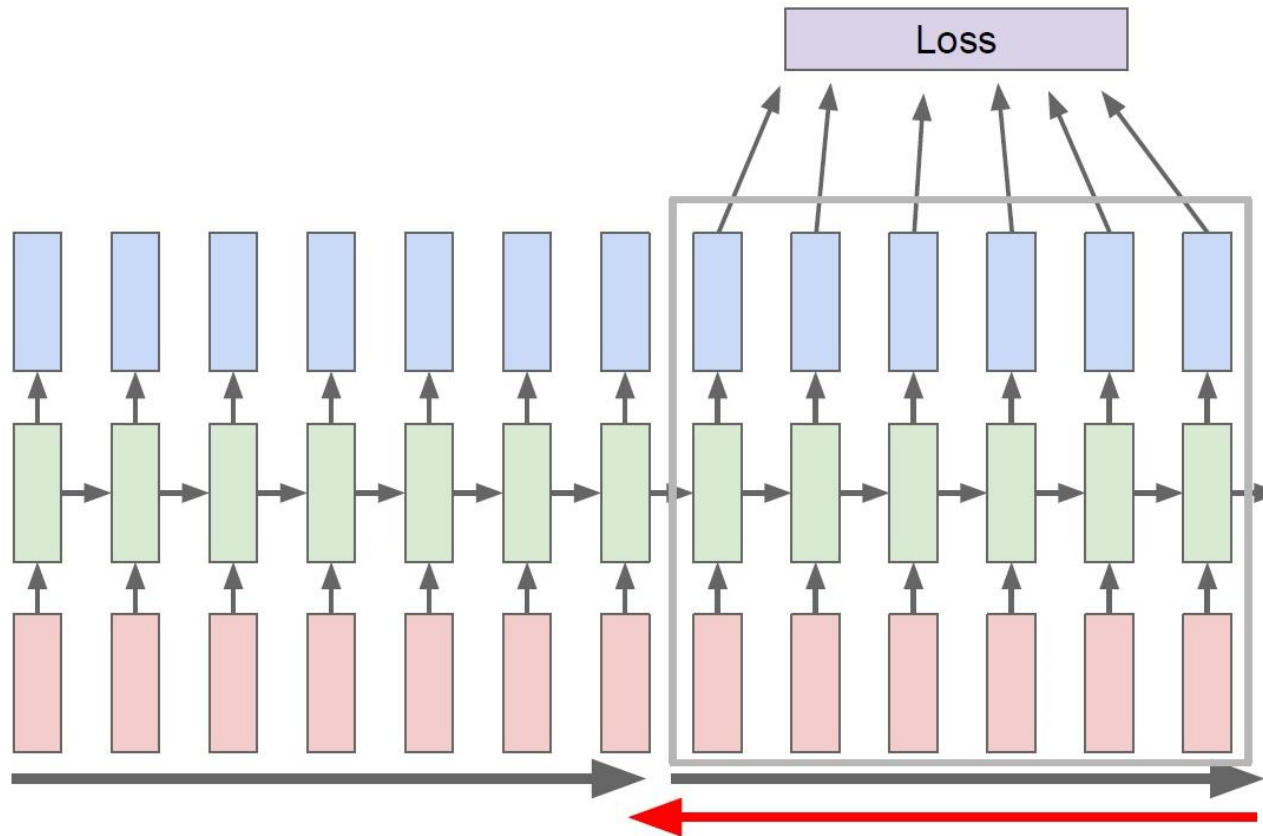
TBTT – *Truncated Backpropagation Through Time*

Forward pass → Parte a sequência
Backward pass → Parte a sequência



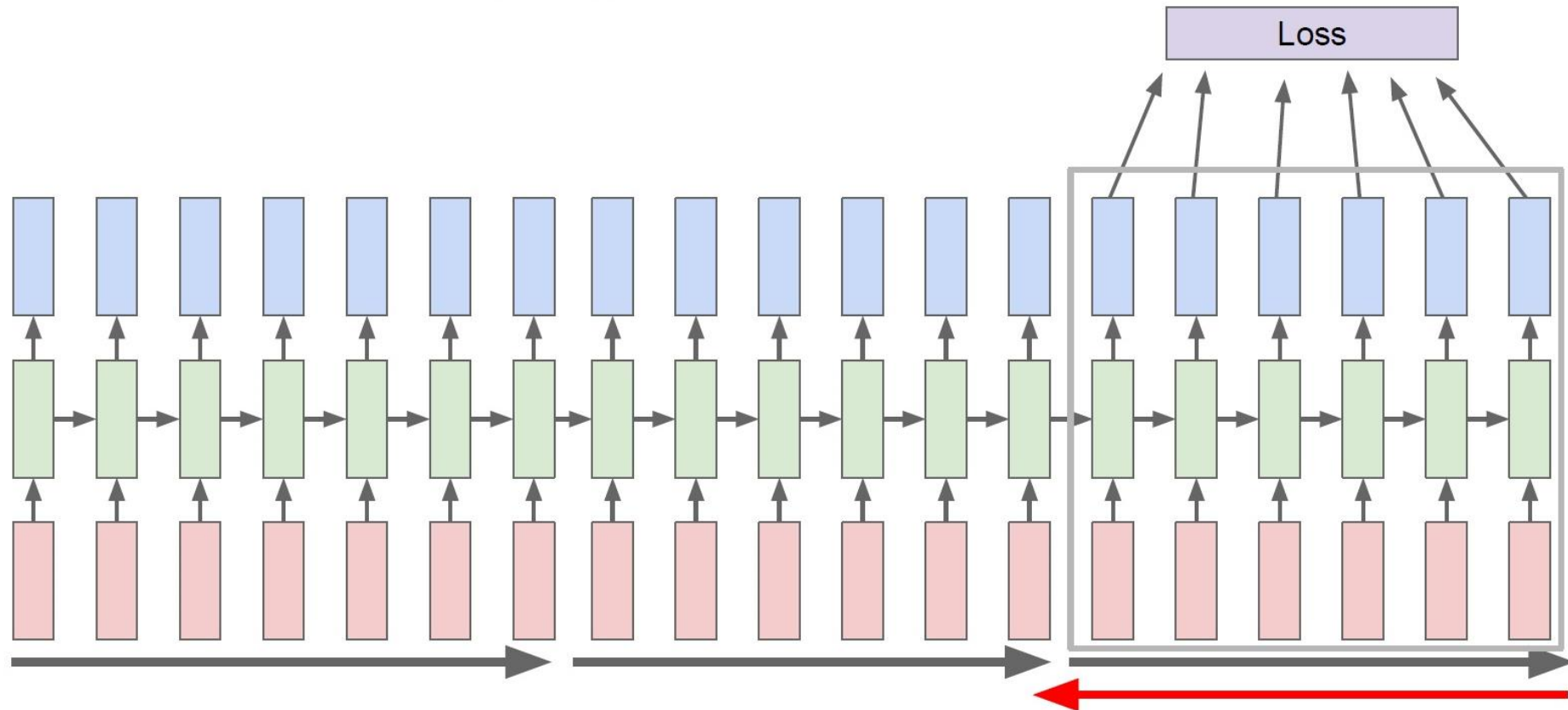
TBTT – *Truncated Backpropagation Through Time*

- Forward pass* → Parte a sequência (consegue propagar estado adiante)
- Backward pass* → Parte a sequência (retropropagar apenas alguns passos)



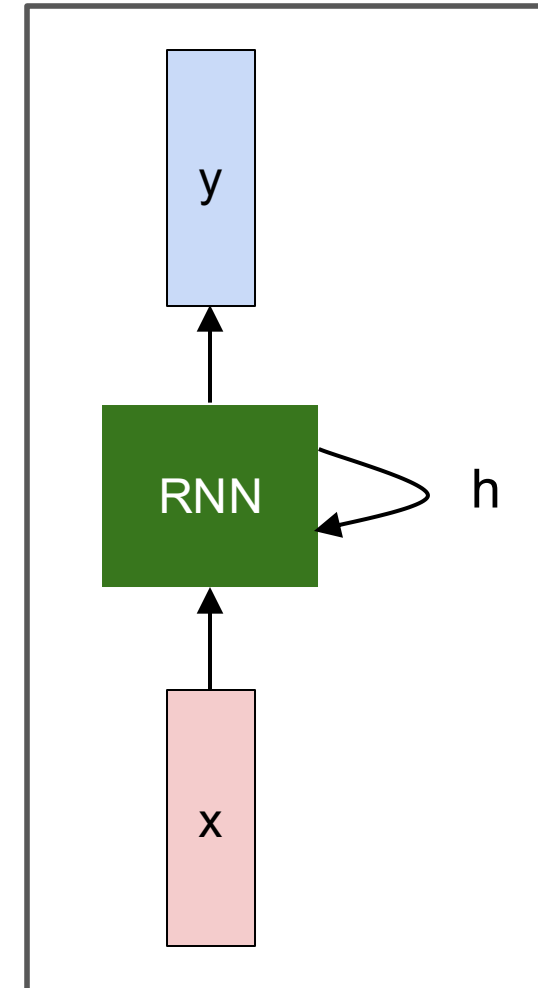
TBTT – *Truncated Backpropagation Through Time*

- Forward pass* → Parte a sequência (consegue propagar estado adiante)
- Backward pass* → Parte a sequência (retropropagar apenas alguns passos)



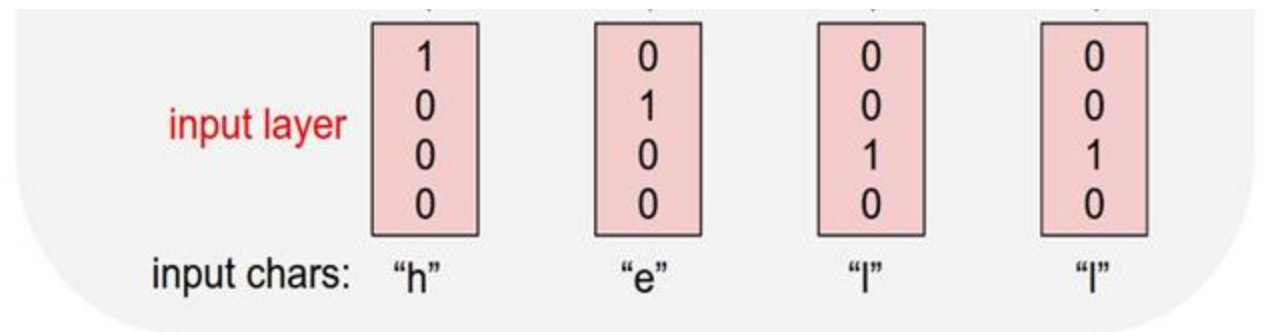
Exemplo de Modelo de Linguagem (Simples)

- Modelo baseado em caracteres
- Vocabulário:
[h, e, l o]
- Exemplo de treinamento:
“hello”



Exemplo de Modelo de Linguagem (Simple)

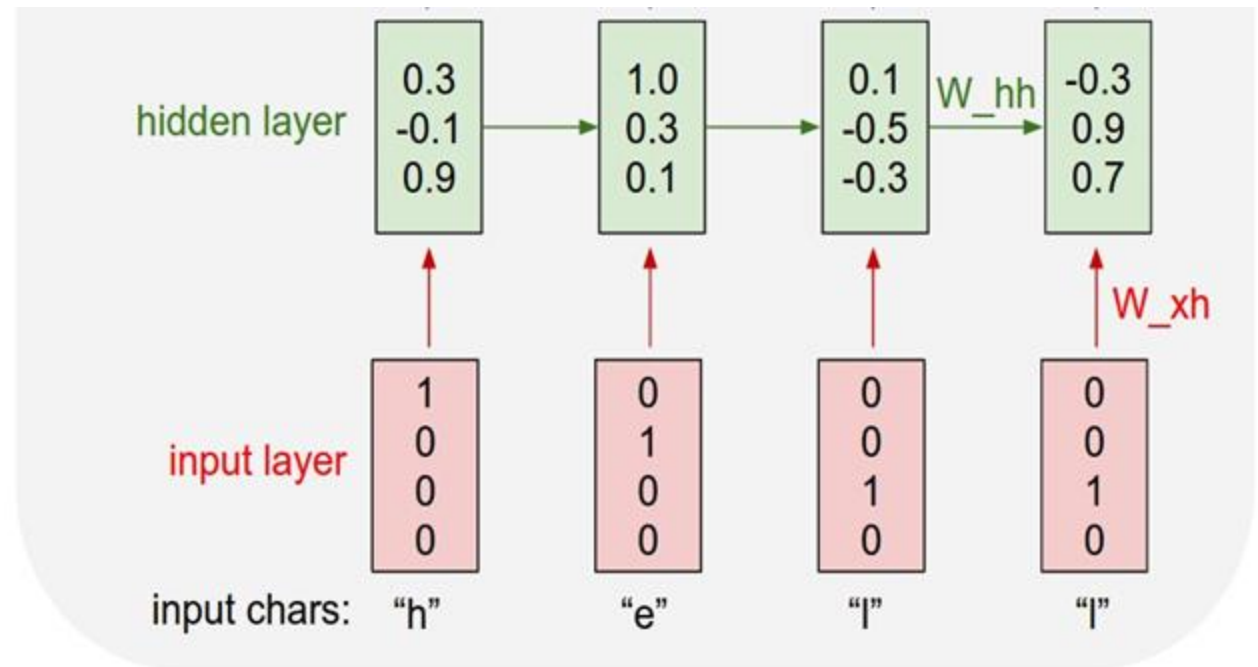
- Modelo baseado em caracteres
- Vocabulário:
[h, e, l o]
- Exemplo de treinamento:
“hello”



Exemplo de Modelo de Linguagem (Simples)

- Modelo baseado em caracteres
- Vocabulário:
[h, e, l o]
- Exemplo de treinamento:
“hello”

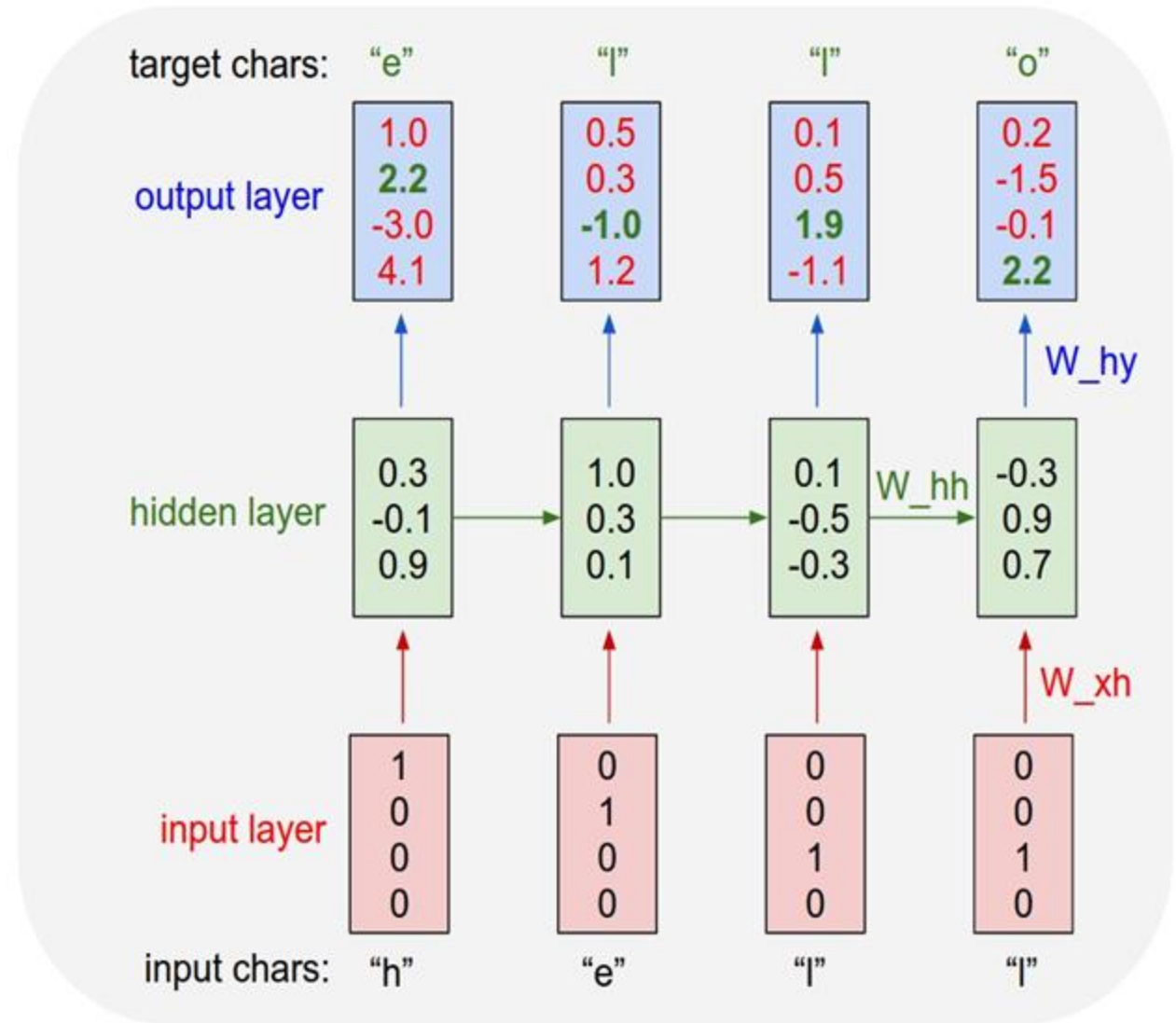
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



Exemplo de Modelo de Linguagem (Simples)

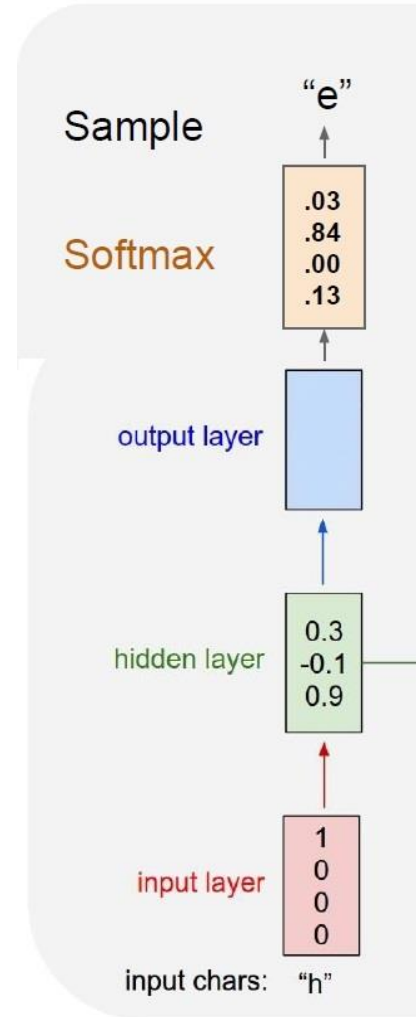
- Modelo baseado em caracteres
- Vocabulário:
[h, e, l o]
- Exemplo de treinamento:
“hello”

$$y_t = W_{hy}h_t$$



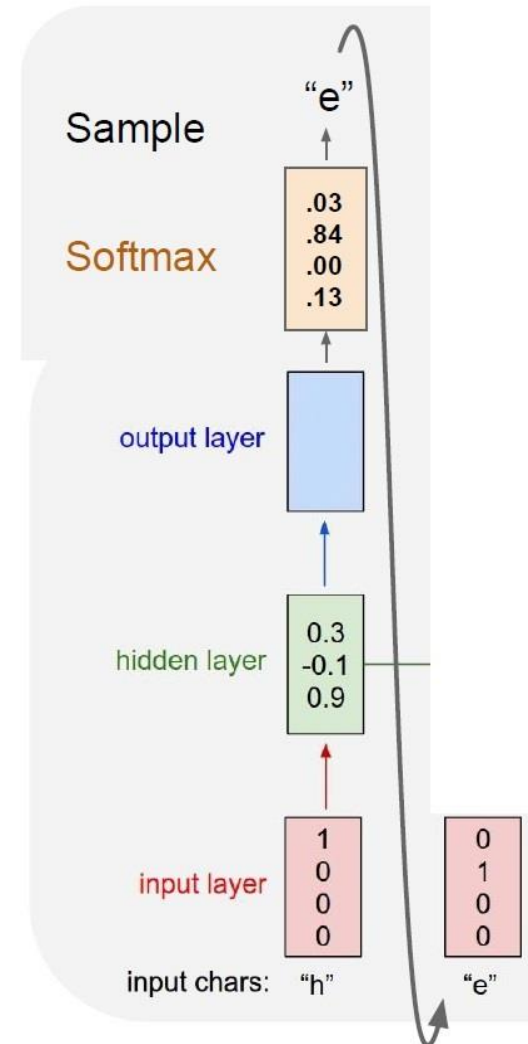
Exemplo de Modelo de Linguagem (Simples)

- Modelo baseado em caracteres
- Vocabulário:
[h, e, l o]
- Durante o teste/operação, um caractere é amostrado por vez e realimentado no modelo



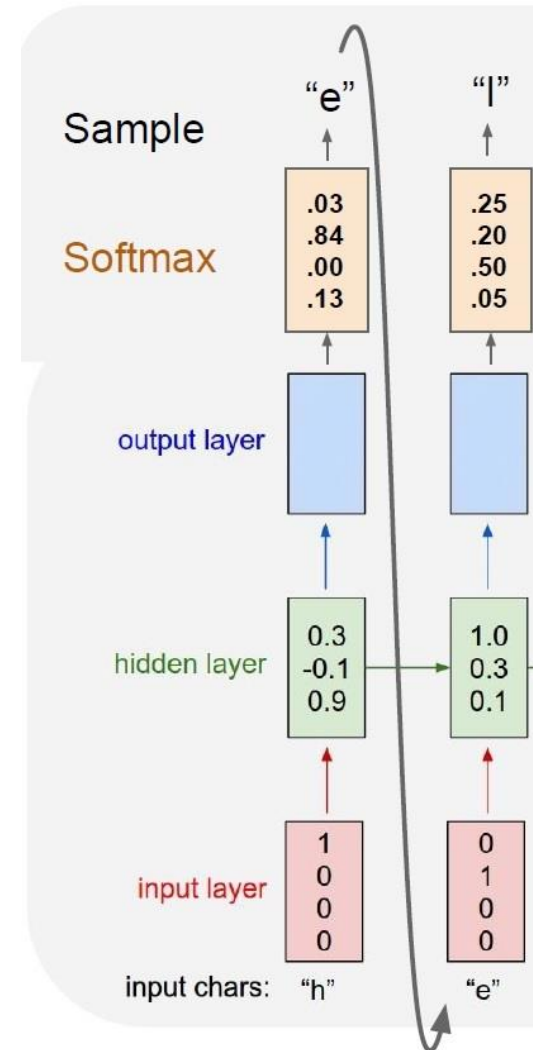
Exemplo de Modelo de Linguagem (Simples)

- Modelo baseado em caracteres
- Vocabulário:
[h, e, l o]
- Durante o teste/operação, um caractere é amostrado por vez e realimentado no modelo



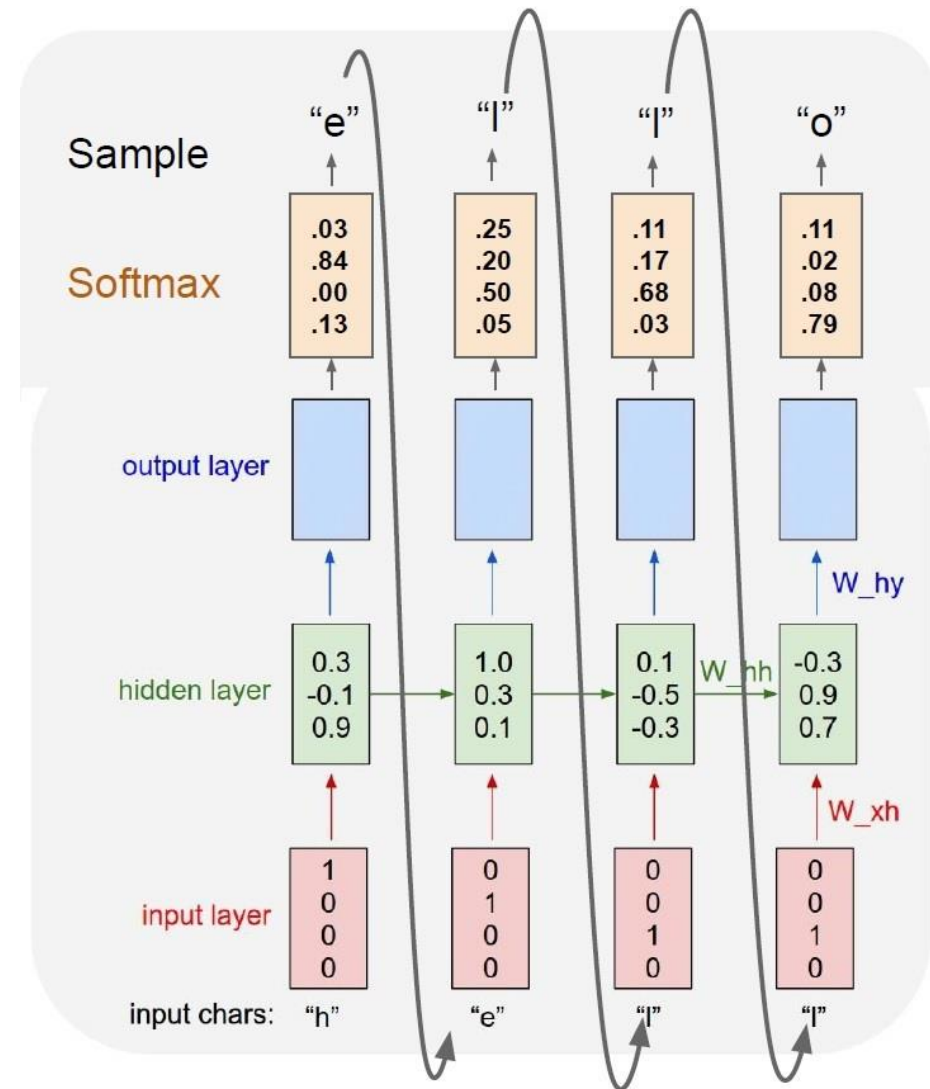
Exemplo de Modelo de Linguagem (Simples)

- Modelo baseado em caracteres
- Vocabulário:
[h, e, l o]
- Durante o teste/operação, um caractere é amostrado por vez e realimentado no modelo



Exemplo de Modelo de Linguagem (Simples)

- Modelo baseado em caracteres
- Vocabulário:
[h, e, l o]
- Durante o teste/operação, um caractere é amostrado por vez e realimentado no modelo



Exemplo de Modelo de Linguagem (Simples)

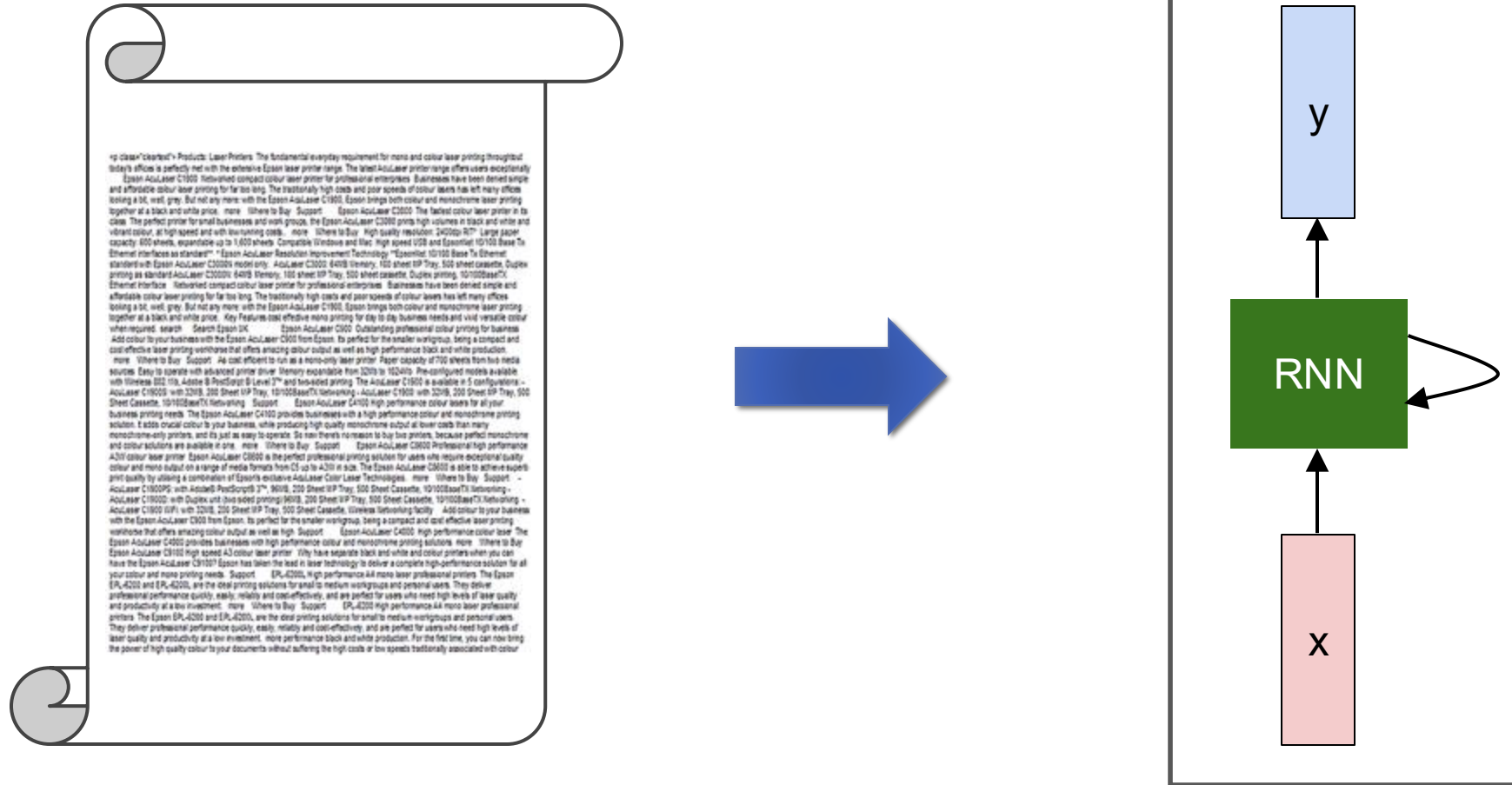
```
1  """
2  Minimal Character-level Vanilla RNN model. Written by Andrej Karpathy (@karpathy)
3  BSD license
4  """
5  import numpy as np
6
7  # data I/O
8  data = open('input.txt', 'r').read() # should be simple plain text file
9  chars = list(set(data))
10 data_size, vocab_size = len(data), len(chars)
11 print 'data has %d characters, %d unique.' % (data_size, vocab_size)
12 char_to_ix = { ch:i for i,ch in enumerate(chars) }
13 ix_to_char = { i:ch for i,ch in enumerate(chars) }
14
15 # hyperparameters
16 hidden_size = 100 # size of hidden layer of neurons
17 seq_length = 25 # number of steps to unroll the RNN for
18 learning_rate = 1e-3
19
20 # model parameters
21 wkh = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
22 whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
23 why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
24 bh = np.zeros((hidden_size, 1)) # hidden bias
25 by = np.zeros((vocab_size, 1)) # output bias
26
27 def lossFun(inputs, targets, hprev):
28     """
29     inputs, targets are both list of integers.
30     hprev is Nx1 array of initial hidden state
31     returns the loss, gradients on model parameters, and last hidden state
32     """
33     xs, hs, ys, ps = {}, {}, {}, {}
34     hs[-1] = np.copy(hprev)
35     loss = 0
36     # forward pass
37     for t in xrange(len(inputs)):
38         xs[t] = np.zeros((vocab_size,1)) # encode in 1-of-k representation
39         xs[t][inputs[t]] = 1
40         hs[t] = np.tanh(np.dot(wkh, xs[t]) + np.dot(whh, hs[t-1]) + bh) # hidden state
41         ys[t] = np.dot(why, hs[t]) + by # unnormalized log probabilities for next chars
42         ps[t] = np.exp(ys[t]) / np.sum(np.exp(ys[t])) # probabilities for next chars
43         loss += -np.log(ps[t][targets[t],0]) # softmax (cross-entropy loss)
44     # backward pass: compute gradients going backwards
45     dwhh, dwkh, dwhy = np.zeros_like(whh), np.zeros_like(wkh), np.zeros_like(why)
46     dbh, dby = np.zeros_like(bh), np.zeros_like(by)
47     dhnext = np.zeros_like(hs[0])
48     for t in reversed(xrange(len(inputs))):
49         dy = np.copy(ps[t])
50         dy[targets[t]] -= 1 # backprop into y
51         dwhy += np.dot(dy, hs[t].T)
52         dby += dy
53         dh = np.dot(why.T, dy) + dhnext # backprop into h
54         dhraw = (1 - hs[t]**2) * dh # backprop through tanh nonlinearity
55         dwkh += dhrw
56         dwhh += np.dot(dhraw, hs[t-1].T)
57         dbh += np.dot(dhraw, hs[t].T)
58         dhnext = np.dot(whh.T, dhraw)
59     for dparam in [dwkh, dwhh, dwhy, dbh, dby]:
60         np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients
61     return loss, dwkh, dwhh, dwhy, dbh, dby, hs[len(inputs)-1]
```

```
62 def sample(h, seed_ix, n):
63     """
64     sample a sequence of integers from the model
65     h is memory state, seed_ix is seed letter for first time step
66     """
67     x = np.zeros((vocab_size, 1))
68     x[seed_ix] = 1
69     ixes = []
70     for t in xrange(n):
71         h = np.tanh(np.dot(wkh, x) + np.dot(whh, h) + bh)
72         y = np.dot(why, h) + by
73         p = np.exp(y) / np.sum(np.exp(y))
74         ix = np.random.choice(range(vocab_size), p=p.ravel())
75         x = np.zeros((vocab_size, 1))
76         x[ix] = 1
77         ixes.append(ix)
78     return ixes
79
80 n, p = 0, 0
81 wkh, whh, why = np.zeros_like(wkh), np.zeros_like(whh), np.zeros_like(why)
82 mbh, mby = np.zeros_like(bh), np.zeros_like(by) # memory variables for Adagrad
83 smooth_loss = -np.log(1.0/vocab_size)*seq_length # loss at iteration 0
84 while True:
85     # prepare inputs (we're sweeping from left to right in steps seq_length long)
86     if p+seq_length > len(data) or n == 0:
87         hprev = np.zeros((hidden_size,1)) # reset RNN memory
88         p = 0 # go from start of data
89     inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
90     targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
91
92     # sample from the model now and then
93     if n % 100 == 0:
94         sample_ix = sample(hprev, inputs[0], 200)
95         txt = ''.join(ix_to_char[ix] for ix in sample_ix)
96         print '....\n %s \n....' % (txt, )
97
98     # forward seq_length characters through the net and fetch gradient
99     loss, dwkh, dwhh, dwhy, dbh, dby, hprev = lossFun(inputs, targets, hprev)
100    smooth_loss = smooth_loss * 0.999 + loss * 0.001
101    if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
102
103    # perform parameter update with Adagrad
104    for param, dparam, mem in zip([wkh, whh, why, bh, by],
105                                  [dwkh, dwhh, dwhy, dbh, dby],
106                                  [dwhh, mwhh, mwhy, mbh, mby]):
107        mem += dparam * dparam
108        param += -learning_rate * dparam / np.sqrt(mem + 1e-8) # adagrad update
109
110    p += seq_length # move data pointer
111    n += 1 # iteration counter
```

min-char-rnn.py: 112 linhas

(<https://gist.github.com/karpathy/d4dee566867f8291f086>)

Resultados – Modelo de Linguagem (Simples)



<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Resultados – Modelo de Linguagem (Simples)

Treinamento com poesia

Sonnet 116 – Let me not ...

by William Shakespeare

Let me not to the marriage of true minds
Admit impediments. Love is not love
Which alters when it alteration finds,
Or bends with the remover to remove:
O no! it is an ever-fixed mark
That looks on tempests and is never shaken;
It is the star to every wandering bark,
Whose worth's unknown, although his height be taken.
Love's not Time's fool, though rosy lips and cheeks
Within his bending sickle's compass come:
Love alters not with his brief hours and weeks,
But bears it out even to the edge of doom.
If this be error and upon me proved,
I never writ, nor no man ever loved.

Resultados – Modelo de Linguagem (Simples)

Treinamento com poesia

Sonnet 116 – Let me not ...

by William Shakespeare

Let me not to the marriage of true minds
Admit impediments. Love is not love
Which alters when it alteration finds,
Or bends with the remover to remove:
O no! it is an ever-fixed mark
That looks on tempests and is never shaken;
It is the star to every wandering bark,
Whose worth's unknown, although his height be taken.
Love's not Time's fool, though rosy lips and cheeks
Within his bending sickle's compass come:
Love alters not with his brief hours and weeks,
But bears it out even to the edge of doom.
If this be error and upon me proved,
I never writ, nor no man ever loved.

tyntd-iafhatawiaoirdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhtnee e
plia tkrgd t o idoe ns,smtt h ne etie h,hregtrs niglike,aoaenns lng

Resultados – Modelo de Linguagem (Simples)

Treinamento com poesia

Sonnet 116 – Let me not ...

by William Shakespeare

Let me not to the marriage of true minds
Admit impediments. Love is not love
Which alters when it alteration finds,
Or bends with the remover to remove:
O no! it is an ever-fixed mark
That looks on tempests and is never shaken;
It is the star to every wandering bark,
Whose worth's unknown, although his height be taken.
Love's not Time's fool, though rosy lips and cheeks
Within his bending sickle's compass come:
Love alters not with his brief hours and weeks,
But bears it out even to the edge of doom.
If this be error and upon me proved,
I never writ, nor no man ever loved.

tyntd-iafhatawiaoirdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhtnee e
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng



Treinando mais

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

Resultados – Modelo de Linguagem (Simples)

Treinamento com poesia

Sonnet 116 – Let me not ...

by William Shakespeare

Let me not to the marriage of true minds
Admit impediments. Love is not love
Which alters when it alteration finds,
Or bends with the remover to remove:
O no! it is an ever-fixed mark
That looks on tempests and is never shaken;
It is the star to every wandering bark,
Whose worth's unknown, although his height be taken.
Love's not Time's fool, though rosy lips and cheeks
Within his bending sickle's compass come:
Love alters not with his brief hours and weeks,
But bears it out even to the edge of doom.
If this be error and upon me proved,
I never writ, nor no man ever loved.

tyntd-iafhatawiaoirdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs niglike,aoaenns lng



Treinando mais

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwy fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."



Treinando mais

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and offer.

Resultados – Modelo de Linguagem (Simples)

Treinamento com poesia

Sonnet 116 – Let me not ...

by William Shakespeare

Let me not to the marriage of true minds
Admit impediments. Love is not love
Which alters when it alteration finds,
Or bends with the remover to remove:
O no! it is an ever-fixed mark
That looks on tempests and is never shaken;
It is the star to every wandering bark,
Whose worth's unknown, although his height be taken.
Love's not Time's fool, though rosy lips and cheeks
Within his bending sickle's compass come:
Love alters not with his brief hours and weeks,
But bears it out even to the edge of doom.
If this be error and upon me proved,
I never writ, nor no man ever loved.

tyntd-iafhatawiaoirdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs niglike,aoaenns lng



Treinando mais

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."



Treinando mais

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and offer.



Treinando mais

"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

Resultados – Modelo de Linguagem (Simples)

Ao final:

PANDARUS:

Alas, I think he shall be come approached and the day
When little strain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.

Second Senator:

They are away this miseries, produced upon my soul,
Breaking and strongly should be buried, when I perish
The earth and thoughts of many states.

DUKE VINCENTIO:

Well, your wit is in the care of side and that.

Second Lord:

They would be ruled after this chamber, and
my fair nuns begun out of the fact, to be conveyed,
Whose noble souls I'll have the heart of the wars.

Clown:

Come, sir, I will make did behold your worship.

VIOLA:

I'll drink it.

VIOLA:

Why, Salisbury must find his flesh and thought
That which I am not apt, not a man and in fire,
To show the reining of the raven and the wars
To grace my hand reproach within, and not a fair are hand,
That Caesar and my goodly father's world;
When I was heaven of presence and our fleets,
We spare with hours, but cut thy council I am great,
Murdered and by thy master's ready there
My power to give thee but so much as hell:
Some service in the noble bondman here,
Would show him to her wine.

KING LEAR:

O, if you were a feeble sight, the courtesy of your law,
Your sight and several breath, will wear the gods
With his heads, and my hands are wonder'd at the deeds,
So drop upon your lordship's head, and your opinion
Shall be against your honour.

Resultados – Modelo de Linguagem (Simples)

Ao final:

PANDARUS:

Alas, I think he shall be come approached and the day
When little strain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.

Second Senator:

They are away this miseries, produced upon my soul,
Breaking and strongly should be buried, when I perish
The earth and thoughts of many states.

DUKE VINCENTIO:

Well, your wit is in the care of side and that.

Second Lord:

They would be ruled after this chamber, and
my fair nudes begun out of the fact, to be conveyed,
Whose noble souls I'll have the heart of the wars.

Clown:

Come, sir, I will make did behold your worship.

VIOLA:

I'll drink it.

VIOLA:

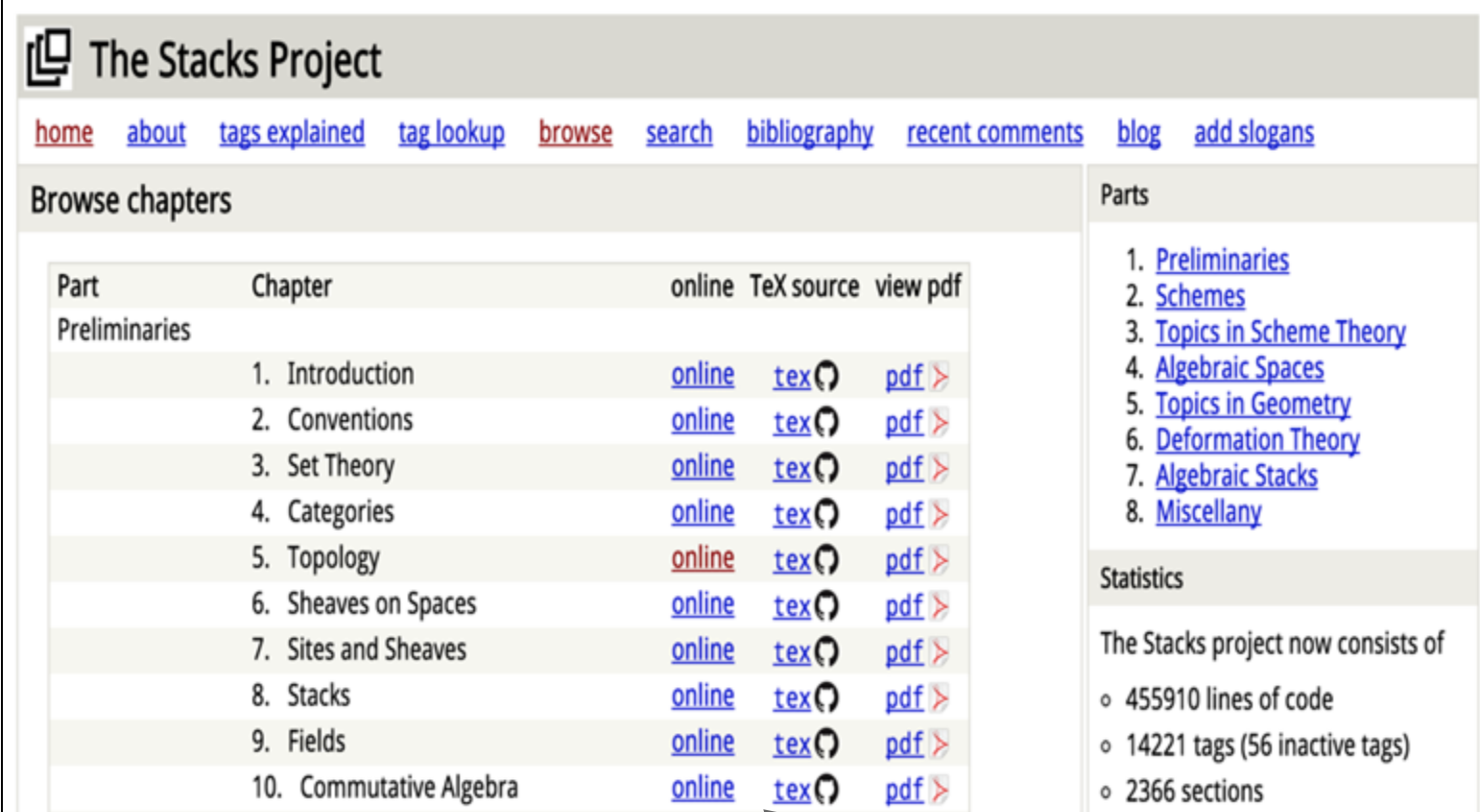
Why, Salisbury must find his flesh and thought
That which I am not apt, not a man and in fire,
To show the reining of the raven and the wars
To grace my hand reproach within, and not a fair are hand,
That Caesar and my goodly father's world;
When I was heaven of presence and our fleets,
We spare with hours, but cut thy council I am great,
Murdered and by thy master's ready there
My power to give thee but so much as hell:
Some service in the noble bondman here,
Would show him to her wine.

KING LEAR:

O, if you were a feeble sight, the courtesy of your law,
Your sight and several breath, will wear the gods
With his heads, and my hands are wonder'd at the deeds,
So drop upon your lordship's head, and your opinion
Shall be against your honour.

Resultados – Modelo de Linguagem (Simples)

Treinamento com livro “open source” sobre geometria algébrica



The Stacks Project

[home](#) [about](#) [tags explained](#) [tag lookup](#) [browse](#) [search](#) [bibliography](#) [recent comments](#) [blog](#) [add slogans](#)

Browse chapters

Part	Chapter	online	TeX source	view pdf
Preliminaries				
	1. Introduction	online	tex 	pdf 
	2. Conventions	online	tex 	pdf 
	3. Set Theory	online	tex 	pdf 
	4. Categories	online	tex 	pdf 
	5. Topology	online	tex 	pdf 
	6. Sheaves on Spaces	online	tex 	pdf 
	7. Sites and Sheaves	online	tex 	pdf 
	8. Stacks	online	tex 	pdf 
	9. Fields	online	tex 	pdf 
	10. Commutative Algebra	online	tex 	pdf 

Parts

1. [Preliminaries](#)
2. [Schemes](#)
3. [Topics in Scheme Theory](#)
4. [Algebraic Spaces](#)
5. [Topics in Geometry](#)
6. [Deformation Theory](#)
7. [Algebraic Stacks](#)
8. [Miscellany](#)

Statistics

The Stacks project now consists of

- 455910 lines of code
- 14221 tags (56 inactive tags)
- 2366 sections

Fonte LaTeX

Resultados – Modelo de Linguagem (Simple)

For $\bigoplus_{n=1, \dots, m}$ where $\mathcal{L}_{m, \bullet} = 0$, hence we can find a closed subset $\mathcal{H}$ in $\mathcal{H}$ and any sets $\mathcal{F}$ on X , U is a closed immersion of S , then $U \rightarrow T$ is a separated algebraic space.

Proof. Proof of (1). It also start we get

$$S = \text{Spec}(R) = U \times_X U \times_X U$$

and the comparicoly in the fibre product covering we have to prove the lemma generated by $\coprod Z \times_U U \rightarrow V$. Consider the maps M along the set of points Sch_{fppf} and $U \rightarrow U$ is the fibre category of S in U in Section, ?? and the fact that any U affine, see Morphisms, Lemma ???. Hence we obtain a scheme S and any open subset $W \subset U$ in $\text{Sh}(G)$ such that $\text{Spec}(R') \rightarrow S$ is smooth or an

$$U = \bigcup U_i \times_{S_i} U_i$$

which has a nonzero morphism we may assume that f_i is of finite presentation over S . We claim that $\mathcal{O}_{X, x}$ is a scheme where $x, x', s'' \in S'$ such that $\mathcal{O}_{X, x'} \rightarrow \mathcal{O}'_{X', x'}$ is separated. By Algebra, Lemma ?? we can define a map of complexes $\text{GL}_{S'}(x'/S'')$ and we win. $\square$

To prove study we see that $\mathcal{F}|_U$ is a covering of $\mathcal{X}'$, and $\mathcal{T}_i$ is an object of $\mathcal{F}_{X/S}$ for $i > 0$ and $\mathcal{F}_p$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$ -modules on $\mathcal{C}$ as a $\mathcal{F}$ -module. In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\widetilde{M}^\bullet = \mathcal{I}^\bullet \otimes_{\text{Spec}(k)} \mathcal{O}_{S, s} - i_X^{-1} \mathcal{F}$$

is a unique morphism of algebraic stacks. Note that

$$\text{Arrows} = (\text{Sch}/S)_{fppf}^{opp}, (\text{Sch}/S)_{fppf}$$

and

$$V = \Gamma(S, \mathcal{O}) \mapsto (U, \text{Spec}(A))$$

is an open subset of X . Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S .

Proof. See discussion of sheaves of sets. $\square$

The result to prove any open covering follows from the less of Example ??. It may replace S by $X_{spaces, \acute{e}tale}$ which gives an open subspace of X and T equal to S_{Zar} , see Descent, Lemma ??. Namely, by Lemma ?? we see that R is geometrically regular over S .

Lemma 0.1. Assume (3) and (3) by the construction in the description.

Suppose $X = \lim |X|$ (by the formal open covering X and a single map $\text{Proj}_X(\mathcal{A}) = \text{Spec}(B)$ over U compatible with the complex

$$\text{Set}(\mathcal{A}) = \Gamma(X, \mathcal{O}_{X, \mathcal{O}_X}).$$

When in this case of to show that $\mathcal{Q} \rightarrow \mathcal{C}_{Z/X}$ is stable under the following result in the second conditions of (1), and (3). This finishes the proof. By Definition ?? (without element is when the closed subschemes are catenary. If T is surjective we may assume that T is connected with residue fields of S . Moreover there exists a closed subspace $Z \subset X$ of X where U in X' is proper (some defining as a closed subset of the uniqueness it suffices to check the fact that the following theorem

(1) f is locally of finite type. Since $S = \text{Spec}(R)$ and $Y = \text{Spec}(R)$.

Proof. This is form all sheaves of sheaves on X . But given a scheme U and a surjective étale morphism $U \rightarrow X$. Let $U \cap U = \coprod_{i=1, \dots, n} U_i$ be the scheme X over S at the schemes $X_i \rightarrow X$ and $U = \lim_i X_i$. $\square$

The following lemma surjective restrocomposes of this implies that $\mathcal{F}_{x_0} = \mathcal{F}_{x_0} = \mathcal{F}_{X, \dots, 0}$.

Lemma 0.2. Let X be a locally Noetherian scheme over S , $E = \mathcal{F}_{X/S}$. Set $\mathcal{I} = \mathcal{I}_1 \subset \mathcal{I}'_n$. Since $\mathcal{I}^n \subset \mathcal{I}^n$ are nonzero over $i_0 \leq \mathfrak{p}$ is a subset of $\mathcal{I}_{n,0} \circ \bar{A}_2$ works.

Lemma 0.3. In Situation ??. Hence we may assume $q' = 0$.

Proof. We will use the property we see that $\mathfrak{p}$ is the next functor (?). On the other hand, by Lemma ?? we see that

$$D(\mathcal{O}_{X'}) = \mathcal{O}_X(D)$$

where K is an F -algebra where δ_{n+1} is a scheme over S . $\square$

Resultados – Modelo de Linguagem (Simple)

For $\bigoplus_{n=1, \dots, m}$ where $\mathcal{L}_{m, \bullet} = 0$, hence we can find a closed subset $\mathcal{H}$ in $\mathcal{H}$ and any sets $\mathcal{F}$ on X , U is a closed immersion of S , then $U \rightarrow T$ is a separated algebraic space.

Proof. Proof of (1). It also start we get

$$S = \text{Spec}(R) = U \times_X U \times_X U$$

and the comparicoly in the fibre product covering we have to prove the lemma generated by $\coprod Z \times_U U \rightarrow V$. Consider the maps M along the set of points Sch_{fppf} and $U \rightarrow U$ is the fibre category of S in U in Section, ?? and the fact that any U affine, see Morphisms, Lemma ???. Hence we obtain a scheme S and any open subset $W \subset U$ in $\text{Sh}(G)$ such that $\text{Spec}(R') \rightarrow S$ is smooth or an

$$U = \bigcup U_i \times_{S_i} U_i$$

which has a nonzero morphism we may assume that f_i is of finite presentation over S . We claim that $\mathcal{O}_{X, x}$ is a scheme where $x, x', s'' \in S'$ such that $\mathcal{O}_{X, x'} \rightarrow \mathcal{O}'_{X', x'}$ is separated. By Algebra, Lemma ?? we can define a map of complexes $\text{GL}_{S'}(x' / \mathcal{O}'')$ and we win. $\square$

To prove study we see that $\mathcal{F}|_U$ is a covering of $\mathcal{X}'$, and $\mathcal{T}_i$ is an object of $\mathcal{F}_{X/S}$ for $i > 0$ and $\mathcal{F}_p$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$ -modules on $\mathcal{C}$ as a $\mathcal{F}$ -module. In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\widetilde{M}^\bullet = \mathcal{I}^\bullet \otimes_{\text{Spec}(k)} \mathcal{O}_{S, s} - i_X^{-1} \mathcal{F}$$

is a unique morphism of algebraic stacks. Note that

$$\text{Arrows} = (\text{Sch}/S)_{fppf}^{opp}, (\text{Sch}/S)_{fppf}$$

and

$$V = \Gamma(S, \mathcal{O}) \mapsto (U, \text{Spec}(A))$$

is an open subset of X . Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S .

Proof. See discussion of sheaves of sets. $\square$

The result for prove any open covering follows from the less of Example ???. It may replace S by $X_{spaces, \acute{e}tale}$ which gives an open subspace of X and T equal to S_{Zar} , see Descent, Lemma ???. Namely, by Lemma ?? we see that R is geometrically regular over S .

Lemma 0.1. Assume (3) and (3) by the construction in the description.

Suppose $X = \lim |X|$ (by the formal open covering X and a single map $\text{Proj}_X(\mathcal{A}) = \text{Spec}(B)$ over U compatible with the complex

$$\text{Set}(\mathcal{A}) = \Gamma(X, \mathcal{O}_{X, \mathcal{O}_X}).$$

When in this case of to show that $\mathcal{Q} \rightarrow \mathcal{C}_{Z/X}$ is stable under the following result in the second conditions of (1), and (3). This finishes the proof. By Definition ?? (without element is when the closed subschemes are catenary. If T is surjective we may assume that T is connected with residue fields of S . Moreover there exists a closed subspace $Z \subset X$ of X where U in X' is proper (some defining as a closed subset of the uniqueness it suffices to check the fact that the following theorem

(1) f is locally of finite type. Since $S = \text{Spec}(R)$ and $Y = \text{Spec}(R)$.

Proof. This is form all sheaves of sheaves on X . But given a scheme U and a surjective étale morphism $U \rightarrow X$. Let $U \cap U = \coprod_{i=1, \dots, n} U_i$ be the scheme X over S at the schemes $X_i \rightarrow X$ and $U = \lim_i X_i$. $\square$

The following lemma surjective restrocomposes of this implies that $\mathcal{F}_{x_0} = \mathcal{F}_{x_0} = \mathcal{F}_{X, \dots, 0}$.

Lemma 0.2. Let X be a locally Noetherian scheme over S , $E = \mathcal{F}_{X/S}$. Set $\mathcal{I} = \mathcal{J}_1 \subset \mathcal{I}_n$. Since $\mathcal{I}^n \subset \mathcal{I}^n$ are nonzero over $i_0 \leq \mathfrak{p}$ is a subset of $\mathcal{J}_{n,0} \circ \bar{A}_2$ works.

Lemma 0.3. In Situation ???. Hence we may assume $q' = 0$.

Proof. We will use the property we see that $\mathfrak{p}$ is the next functor (??). On the other hand, by Lemma ?? we see that

$$D(\mathcal{O}_{X'}) = \mathcal{O}_X(D)$$

where K is an F -algebra where δ_{n+1} is a scheme over S . $\square$

Resultados – Modelo de Linguagem (Simple)

Proof. Omitted. □

Lemma 0.1. *Let $\mathcal{C}$ be a set of the construction.*

Let $\mathcal{C}$ be a gerber covering. Let $\mathcal{F}$ be a quasi-coherent sheaves of $\mathcal{O}$ -modules. We have to show that

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

Proof. This is an algebraic space with the composition of sheaves $\mathcal{F}$ on $X_{\text{étale}}$ we have

$$\mathcal{O}_X(\mathcal{F}) = \{\text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where $\mathcal{G}$ defines an isomorphism $\mathcal{F} \rightarrow \mathcal{F}$ of $\mathcal{O}$ -modules. □

Lemma 0.2. *This is an integer Z is injective.*

Proof. See Spaces, Lemma ?? □

Lemma 0.3. *Let S be a scheme. Let X be a scheme and X is an affine open covering. Let $\mathcal{U} \subset \mathcal{X}$ be a canonical and locally of finite type. Let X be a scheme. Let X be a scheme which is equal to the formal complex.*

The following to the construction of the lemma follows.

Let X be a scheme. Let X be a scheme covering. Let

$$b : X \rightarrow Y' \rightarrow Y \rightarrow Y \rightarrow Y' \times_X Y \rightarrow X.$$

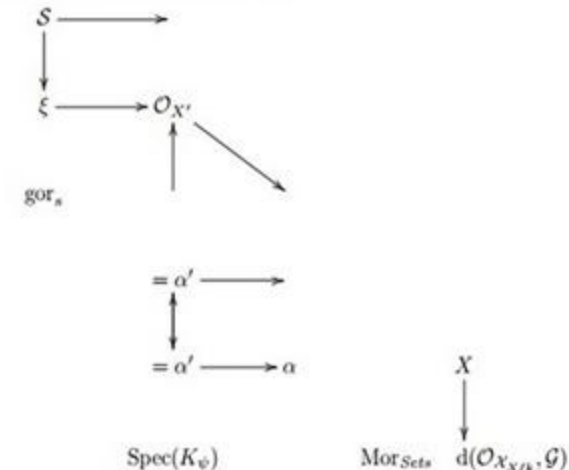
be a morphism of algebraic spaces over S and Y .

Proof. Let X be a nonzero scheme of X . Let X be an algebraic space. Let $\mathcal{F}$ be a quasi-coherent sheaf of $\mathcal{O}_X$ -modules. The following are equivalent

- (1) $\mathcal{F}$ is an algebraic space over S .
- (2) If X is an affine open covering.

Consider a common structure on X and X the functor $\mathcal{O}_X(U)$ which is locally of finite type. □

This since $\mathcal{F} \in \mathcal{F}$ and $x \in \mathcal{G}$ the diagram



is a limit. Then $\mathcal{G}$ is a finite type and assume S is a flat and $\mathcal{F}$ and $\mathcal{G}$ is a finite type f_* . This is of finite type diagrams, and

- the composition of $\mathcal{G}$ is a regular sequence,
- $\mathcal{O}_{X'}$ is a sheaf of rings.

□

Proof. We have see that $X = \text{Spec}(R)$ and $\mathcal{F}$ is a finite type representable by algebraic space. The property $\mathcal{F}$ is a finite morphism of algebraic stacks. Then the cohomology of X is an open neighbourhood of U . □

Proof. This is clear that $\mathcal{G}$ is a finite presentation, see Lemmas ??.

A reduced above we conclude that U is an open covering of $\mathcal{C}$. The functor $\mathcal{F}$ is a "field

$$\mathcal{O}_{X,x} \longrightarrow \mathcal{F}_x \longrightarrow \mathcal{O}_{X_{\text{étale}}}^{-1} \longrightarrow \mathcal{O}_{X_{\lambda}}^{-1} \mathcal{O}_{X_{\lambda}}(\mathcal{O}_{X_{\eta}}^{\#})$$

is an isomorphism of covering of $\mathcal{O}_{X_i}$. If $\mathcal{F}$ is the unique element of $\mathcal{F}$ such that X is an isomorphism.

The property $\mathcal{F}$ is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme $\mathcal{O}_X$ -algebra with $\mathcal{F}$ are opens of finite type over S .

If $\mathcal{F}$ is a scheme theoretic image points. □

If $\mathcal{F}$ is a finite direct sum $\mathcal{O}_{X_{\lambda}}$ is a closed immersion, see Lemma ???. This is a sequence of $\mathcal{F}$ is a similar morphism.

Resultados – Modelo de Linguagem (Simple)

Proof. Omitted.

Lemma 0.1. *Let $\mathcal{C}$ be a set of the construction.*

Let $\mathcal{C}$ be a gerber covering. Let $\mathcal{F}$ be a quasi-coherent sheaves of $\mathcal{O}$ -modules. We have to show that

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

Proof. This is an algebraic space with the composition of sheaves $\mathcal{F}$ on $X_{\text{étale}}$ we have

$$\mathcal{O}_X(\mathcal{F}) = \{\text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where $\mathcal{G}$ defines an isomorphism $\mathcal{F} \rightarrow \mathcal{F}$ of $\mathcal{O}$ -modules.

Lemma 0.2. *This is an integer Z is injective.*

Proof. See Spaces, Lemma ??.

Lemma 0.3. *Let S be a scheme. Let X be a scheme and X is an affine open covering. Let $\mathcal{U} \subset \mathcal{X}$ be a canonical and locally of finite type. Let X be a scheme. Let X be a scheme which is equal to the formal complex.*

The following to the construction of the lemma follows.

Let X be a scheme. Let X be a scheme covering. Let

$$b : X \rightarrow Y' \rightarrow Y \rightarrow Y \rightarrow Y' \times_X Y \rightarrow X.$$

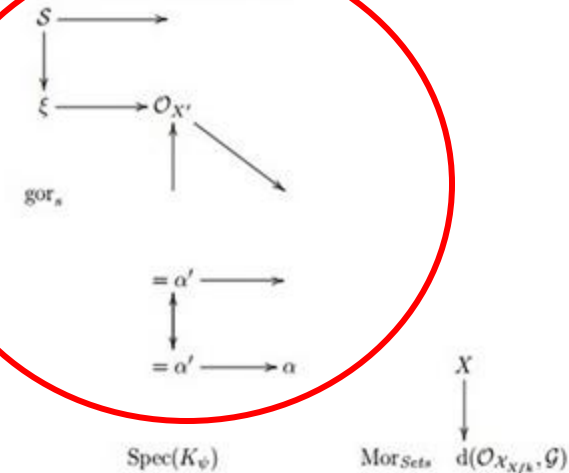
be a morphism of algebraic spaces over S and Y .

Proof. Let X be a nonzero scheme of X . Let X be an algebraic space. Let $\mathcal{F}$ be a quasi-coherent sheaf of $\mathcal{O}_X$ -modules. The following are equivalent

- (1) $\mathcal{F}$ is an algebraic space over S .
- (2) If X is an affine open covering.

Consider a common structure on X and X the functor $\mathcal{O}_X(U)$ which is locally of finite type.

This since $\mathcal{F} \in \mathcal{F}$ and $x \in \mathcal{G}$ the diagram



is a limit. Then $\mathcal{G}$ is a finite type and assume S is a flat and $\mathcal{F}$ and $\mathcal{G}$ is a finite type f_* . This is of finite type diagrams, and

- the composition of $\mathcal{G}$ is a regular sequence,
- $\mathcal{O}_{X'}$ is a sheaf of rings.

Proof. We have see that $X = \text{Spec}(R)$ and $\mathcal{F}$ is a finite type representable by algebraic space. The property $\mathcal{F}$ is a finite morphism of algebraic stacks. Then the cohomology of X is an open neighbourhood of U .

Proof. This is clear that $\mathcal{G}$ is a finite presentation, see Lemmas ??.

A reduced above we conclude that U is an open covering of $\mathcal{C}$. The functor $\mathcal{F}$ is a "field

$$\mathcal{O}_{X,x} \rightarrow \mathcal{F}_x \rightarrow \mathcal{O}_{X_{\text{étale}}}^{-1} \rightarrow \mathcal{O}_{X_{\text{étale}}}^{-1} \mathcal{O}_{X_{\text{étale}}}(\mathcal{O}_{X_{\text{étale}}}^{\text{étale}})$$

is an isomorphism of covering of $\mathcal{O}_{X_i}$. If $\mathcal{F}$ is the unique element of $\mathcal{F}$ such that X is an isomorphism.

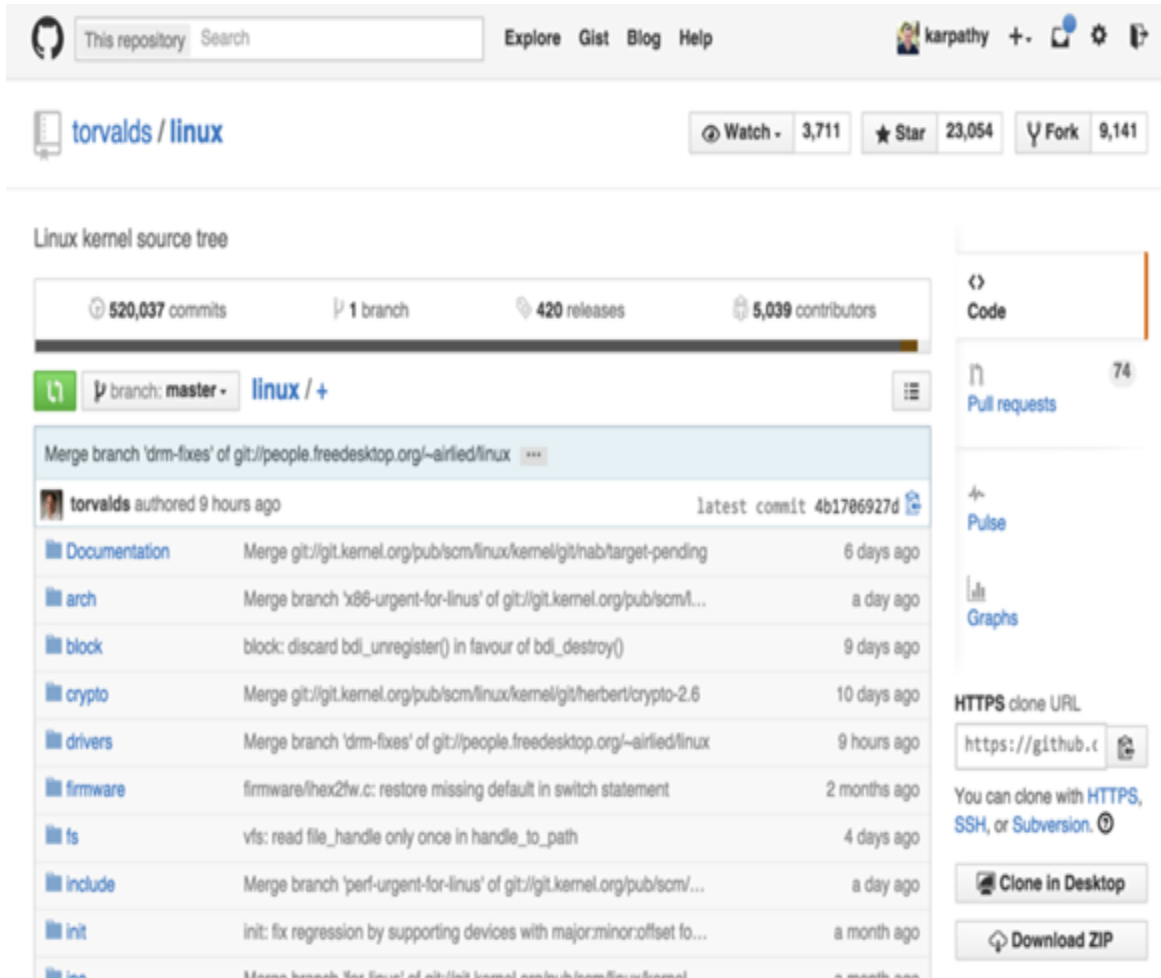
The property $\mathcal{F}$ is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme $\mathcal{O}_X$ -algebra with $\mathcal{F}$ are opens of finite type over S .

If $\mathcal{F}$ is a scheme theoretic image points.

If $\mathcal{F}$ is a finite direct sum $\mathcal{O}_{X_{\text{étale}}}$ is a closed immersion, see Lemma ??.

Resultados – Modelo de Linguagem (Simples)

Treinamento com código do Linux



The screenshot shows the GitHub repository for the Linux kernel source tree. The repository is owned by 'torvalds' and has 520,037 commits, 1 branch, 420 releases, and 5,039 contributors. The current branch is 'master' and the repository is named 'linux'. The repository is watched by 3,711 users, starred by 23,054 users, and forked by 9,141 users. The repository is a merge of the 'drm-fixes' branch of 'git://people.freedesktop.org/~airlied/linux'. The latest commit is 4b1786927d, authored by 'torvalds' 9 hours ago. The repository contains several subdirectories: Documentation, arch, block, crypto, drivers, firmware, fs, include, init, and io. The 'Documentation' directory is highlighted. The repository is also available as a ZIP file.

Directory	Description	Time
Documentation	Merge git://git.kernel.org/pub/scm/linux/kernel/git/hab/target-pending	6 days ago
arch	Merge branch 'x86-urgent-for-linus' of git://git.kernel.org/pub/scm/lin...	a day ago
block	block: discard bdi_unregister() in favour of bdi_destroy()	9 days ago
crypto	Merge git://git.kernel.org/pub/scm/linux/kernel/git/herbert/crypto-2.6	10 days ago
drivers	Merge branch 'drm-fixes' of git://people.freedesktop.org/~airlied/linux	9 hours ago
firmware	firmware/ihex2fw.c: restore missing default in switch statement	2 months ago
fs	vfs: read file_handle only once in handle_to_path	4 days ago
include	Merge branch 'perf-urgent-for-linus' of git://git.kernel.org/pub/scm/lin...	a day ago
init	init: fix regression by supporting devices with major:minor:offset fo...	a month ago
io	Linux kernel source tree	a month ago

```
static void do_command(struct seq_file *m, void *v)
{
    int column = 32 << (cmd[2] & 0x80);
    if (state)
        cmd = (int)(int_state ^ (in_8(&ch->ch_flags) & Cmd) ? 2 : 1);
    else
        seq = 1;
    for (i = 0; i < 16; i++) {
        if (k & (1 << i))
            pipe = (in_use & UMXTHREAD_UNCCA) +
                ((count & 0x00000000ffffff8) & 0x000000f) << 8;
        if (count == 0)
            sub(pid, ppc_md.kexec_handle, 0x20000000);
        pipe_set_bytes(i, 0);
    }
    /* Free our user pages pointer to place camera if all dash */
    subsystem_info = &of_changes[PAGE_SIZE];
    rek_controls(offset, idx, &soffset);
    /* Now we want to deliberately put it to device */
    control_check_polarity(&context, val, 0);
    for (i = 0; i < COUNTER; i++)
        seq_puts(s, "policy ");
}
```

Resultados – Modelo de Linguagem (Simples)

```
/*
 * Copyright (c) 2006-2010, Intel Mobile Communications. All rights reserved.
 *
 * This program is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 as published by
 * the Free Software Foundation.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 *
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software Foundation,
 * Inc., 675 Mass Ave, Cambridge, MA 02139, USA.
 */

#include <linux/kexec.h>
#include <linux/errno.h>
#include <linux/io.h>
#include <linux/platform_device.h>
#include <linux/multi.h>
#include <linux/ckevent.h>

#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>
```

```
#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>

#define REG_PG    vesa_slot_addr_pack
#define PFM_NOCOMP AFSR(0, load)
#define STACK_DDR(type)      (func)

#define SWAP_ALLOCATE(nr)      (e)
#define emulate_sigs()  arch_get_unaligned_child()
#define access_rw(TST)  asm volatile("movd %esp, %0, %3" : : "r" (0)); \
    if (__type & DO_READ)

static void stat_PC_SEC __read_mostly offsetof(struct seq_argsqueue, \
    pc>[1]);

static void
os_prefix(unsigned long sys)
{
#ifdef CONFIG_PREEMPT
    PUT_PARAM_RAID(2, sel) = get_state_state();
    set_pid_sum((unsigned long)state, current_state_str(),
        (unsigned long)-1->lr_full; low;
}
}
```

Resultados – Modelo de Linguagem (Simples)

```
/*
 * Copyright (c) 2006-2010, Intel Mobile Communications. All rights reserved.
 *
 * This program is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 as published by
 * the Free Software Foundation.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 *
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software Foundation,
 * Inc., 675 Mass Ave, Cambridge, MA 02139, USA.
 */
```

```
#include <linux/kexec.h>
#include <linux/errno.h>
#include <linux/io.h>
#include <linux/platform_device.h>
#include <linux/multi.h>
#include <linux/ckevent.h>

#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>
```

```
#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>
```

```
#define REG_PG    vesa_slot_addr_pack
#define PFM_NOCOMP AFSR(0, load)
#define STACK_DDR(type)      (func)

#define SWAP_ALLOCATE(nr)      (e)
#define emulate_sigs() arch_get_unaligned_child()
#define access_rw(TST) asm volatile("movd %0, %1" : : "r" (0)); \
    if (__type & DO_READ)
```

```
static void stat_PC_SEC __read_mostly offsetof(struct seq_argsqueue, \
    pc>[1]);
```

```
static void
os_prefix(unsigned long sys)
{
#ifdef CONFIG_PREEMPT
    PUT_PARAM_RAID(2, sel) = get_state_state();
    set_pid_sum((unsigned long)state, current_state_str(),
        (unsigned long)-1->lr_full; low;
}
```

RNN – *Prós* × *Cons*

Vantagens:

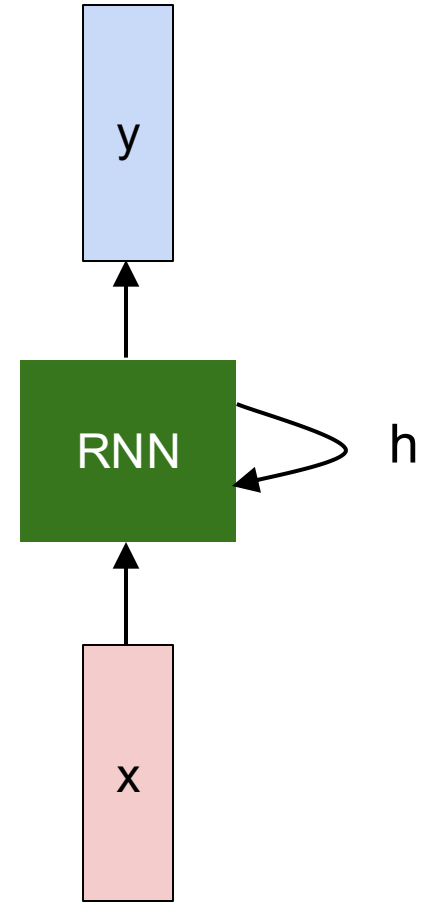
- Pode processar entrada de qualquer tamanho
- Computação para o passo t pode (em teoria) usar informação de muitos passos anteriores
- Tamanho do modelo não cresce para entradas mais longas
- Os mesmos pesos são usados em cada passo, gerando uma simetria na forma com as entradas são processadas

Desvantagens:

- Computação recorrente é lenta
- Na prática, existe dificuldade em acessar informações de muitos passos anteriores

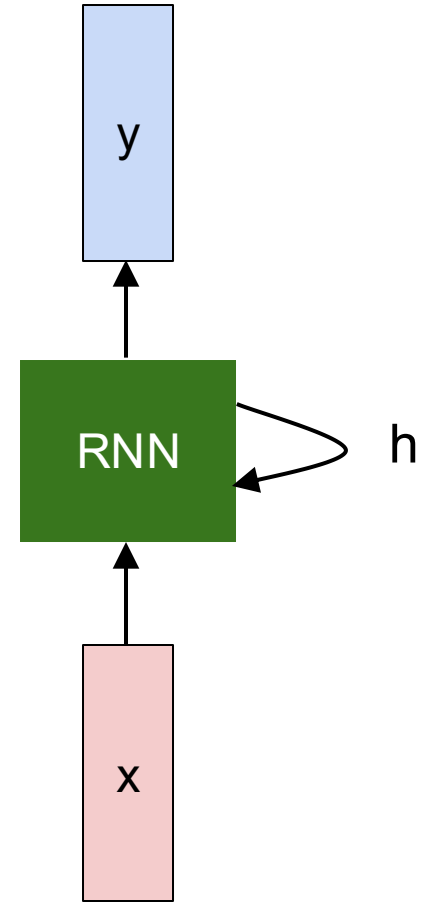
Problema – Rede Neural Recorrente (RNN)

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



Problema – Rede Neural Recorrente (RNN)

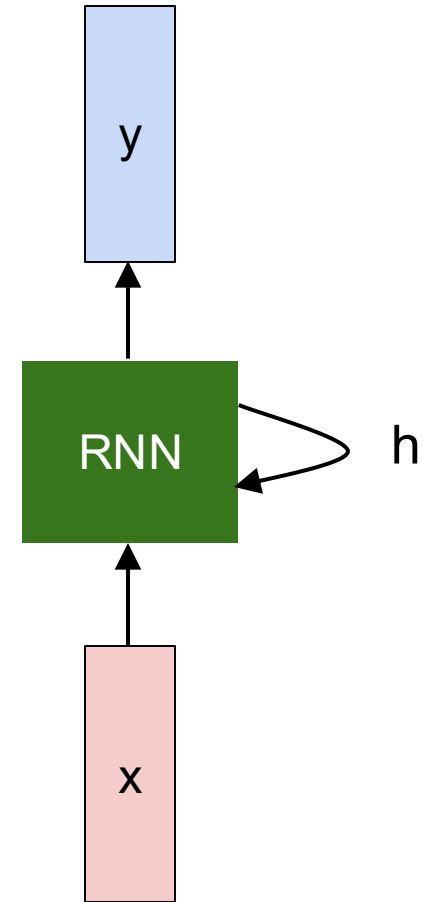
$$h_t = \tanh(\overbrace{W_{hh}h_{t-1} + W_{xh}x_t}^{\hat{h}})$$



Problema – Rede Neural Recorrente (RNN)

$$h_t = \tanh(\overbrace{W_{hh}h_{t-1} + W_{xh}x_t}^{\hat{h}})$$

No **backprop** será necessário $\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial \hat{h}} \times \frac{\partial \hat{h}}{\partial h_{t-1}}$

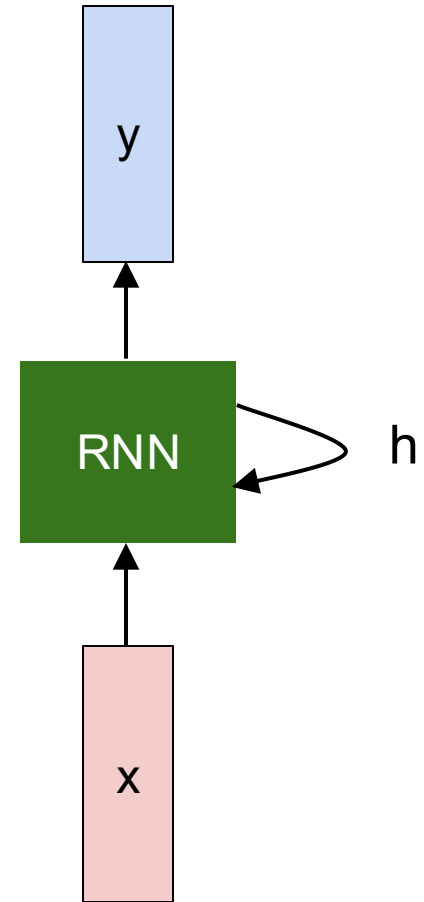


Problema – Rede Neural Recorrente (RNN)

$$h_t = \tanh(\overbrace{W_{hh}h_{t-1} + W_{xh}x_t}^{\hat{h}})$$

No **backprop** será necessário $\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial \hat{h}} \times \frac{\partial \hat{h}}{\partial h_{t-1}}$

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{sech}^2 \hat{h} \times W_{hh}$$

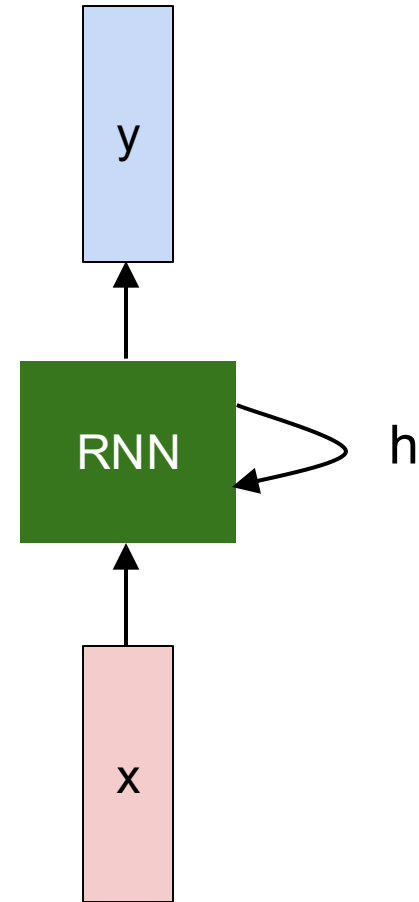


Problema – Rede Neural Recorrente (RNN)

$$h_t = \tanh(\overbrace{W_{hh}h_{t-1} + W_{xh}x_t}^{\hat{h}})$$

No **backprop** será necessário $\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial \hat{h}} \times \frac{\partial \hat{h}}{\partial h_{t-1}}$

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{sech}^2 \hat{h} \times W_{hh}$$

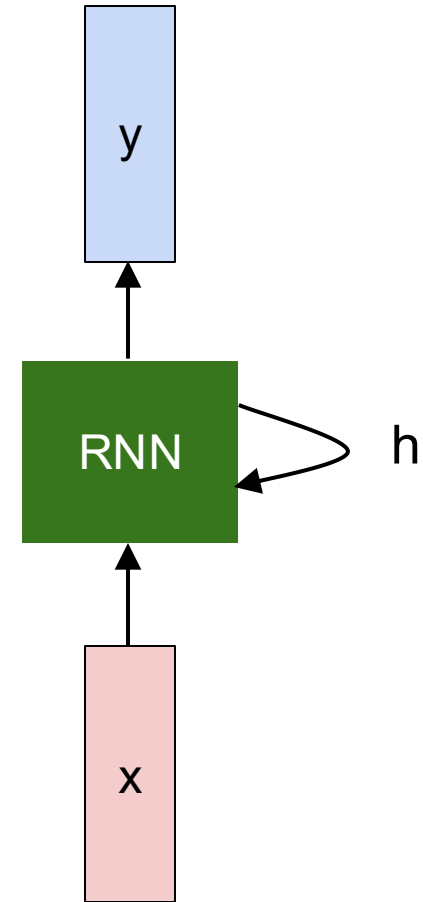


Problema – Rede Neural Recorrente (RNN)

$$h_t = \tanh(\overbrace{W_{hh}h_{t-1} + W_{xh}x_t}^{\hat{h}})$$

No **backprop** será necessário $\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial \hat{h}} \times \frac{\partial \hat{h}}{\partial h_{t-1}}$

$$\frac{\partial h_t}{\partial h_{t-1}} = \underbrace{\text{sech}^2 \hat{h}}_{\leq 1} \times W_{hh}$$



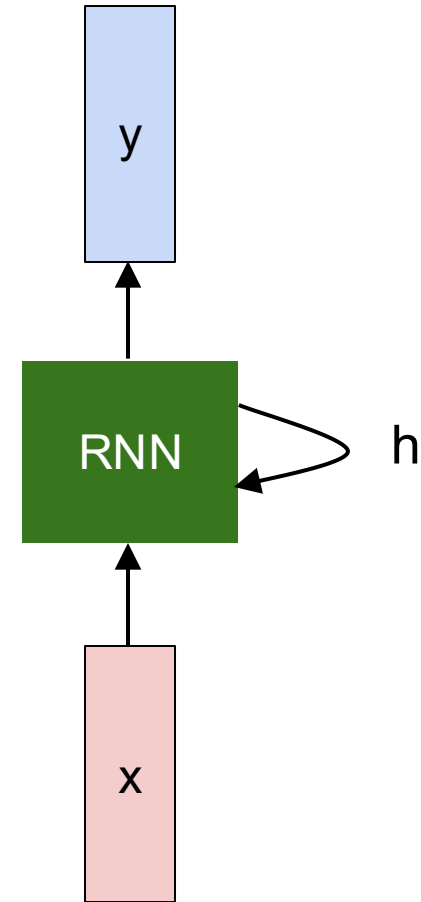
Problema – Rede Neural Recorrente (RNN)

$$h_t = \tanh(\overbrace{W_{hh}h_{t-1} + W_{xh}x_t}^{\hat{h}})$$

No **backprop** será necessário $\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial \hat{h}} \times \frac{\partial \hat{h}}{\partial h_{t-1}}$

$$\frac{\partial h_t}{\partial h_{t-1}} = \underbrace{\text{sech}^2 \hat{h}}_{\leq 1} \times W_{hh}$$

W_{hh} é multiplicado pelo gradiente no próximo passo, logo, o gradiente ao longo do tempo é aproximadamente uma potência de W_{hh}



Problema – Rede Neural Recorrente (RNN)

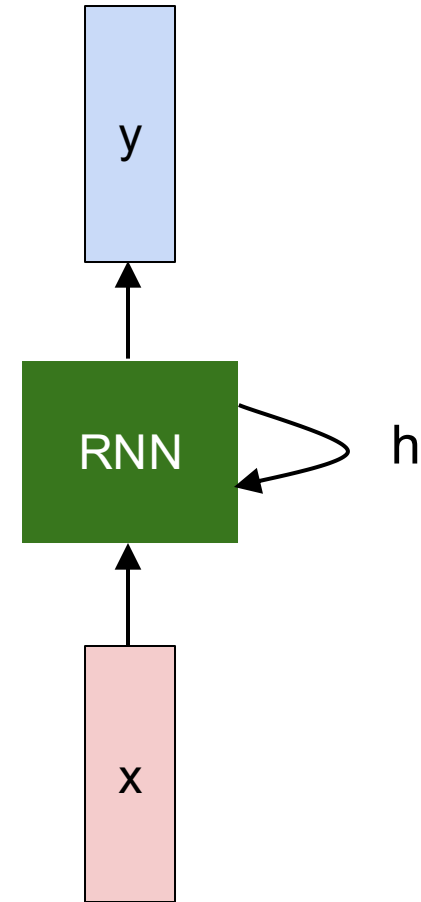
$$h_t = \tanh(\overbrace{W_{hh}h_{t-1} + W_{xh}x_t}^{\hat{h}})$$

No **backprop** será necessário $\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial \hat{h}} \times \frac{\partial \hat{h}}{\partial h_{t-1}}$

$$\frac{\partial h_t}{\partial h_{t-1}} = \underbrace{\text{sech}^2 \hat{h}}_{\leq 1} \times W_{hh}$$

W_{hh} é multiplicado pelo gradiente no próximo passo, logo, o gradiente ao longo do tempo é aproximadamente uma potência de W_{hh}

Se o maior autovalor de $W_{hh} > 1$, os gradientes crescerão com o tempo (explodindo)



Problema – Rede Neural Recorrente (RNN)

$$h_t = \tanh(\overbrace{W_{hh}h_{t-1} + W_{xh}x_t}^{\hat{h}})$$

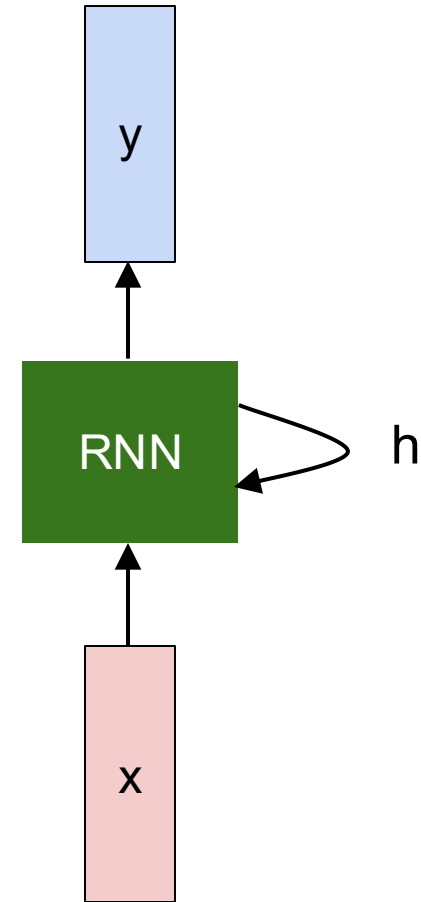
No **backprop** será necessário $\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial \hat{h}} \times \frac{\partial \hat{h}}{\partial h_{t-1}}$

$$\frac{\partial h_t}{\partial h_{t-1}} = \underbrace{\text{sech}^2 \hat{h}}_{\leq 1} \times W_{hh}$$

W_{hh} é multiplicado pelo gradiente no próximo passo, logo, o gradiente ao longo do tempo é aproximadamente uma potência de W_{hh}

Se o maior autovalor de $W_{hh} > 1$, os gradientes crescerão com o tempo (explodindo)

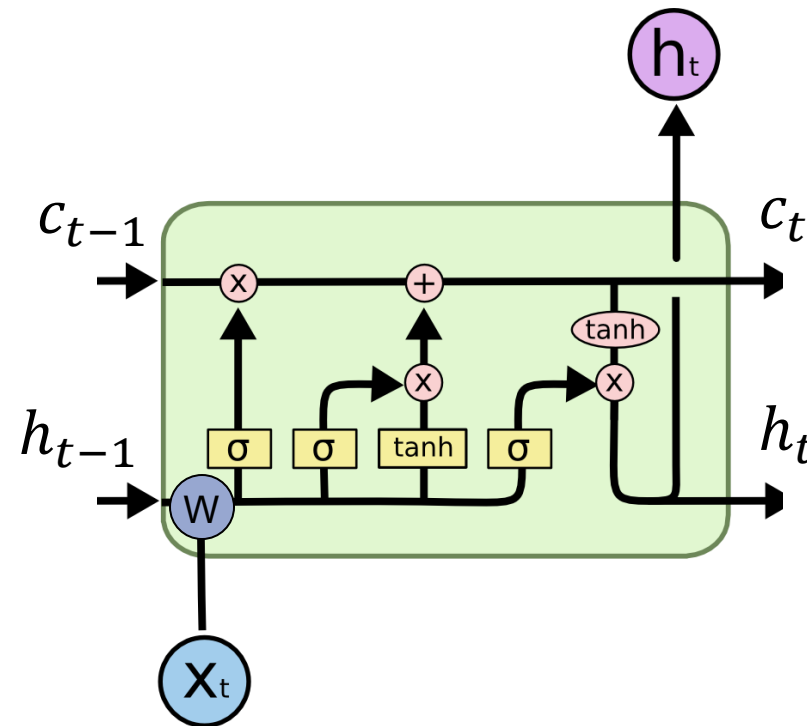
Se o maior autovalor de $W_{hh} < 1$, os gradientes reduzirão com o tempo (desaparecendo)



LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

Unidades LSTM (*Long Short-Term Memory*) possuem uma memória de célula c_i que é repassada de um instante de tempo para o próximo (sem transformação não linear)

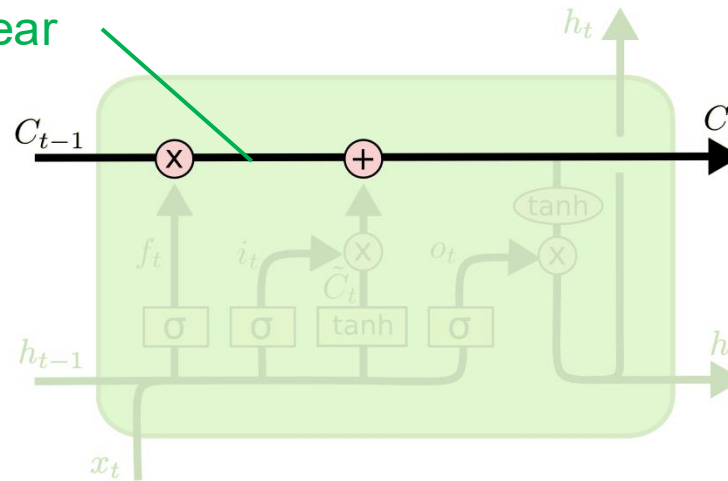


LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

Unidades LSTM (*Long Short-Term Memory*) possuem uma memória de célula c_i que é repassada de um instante de tempo para o próximo (sem transformação não linear)

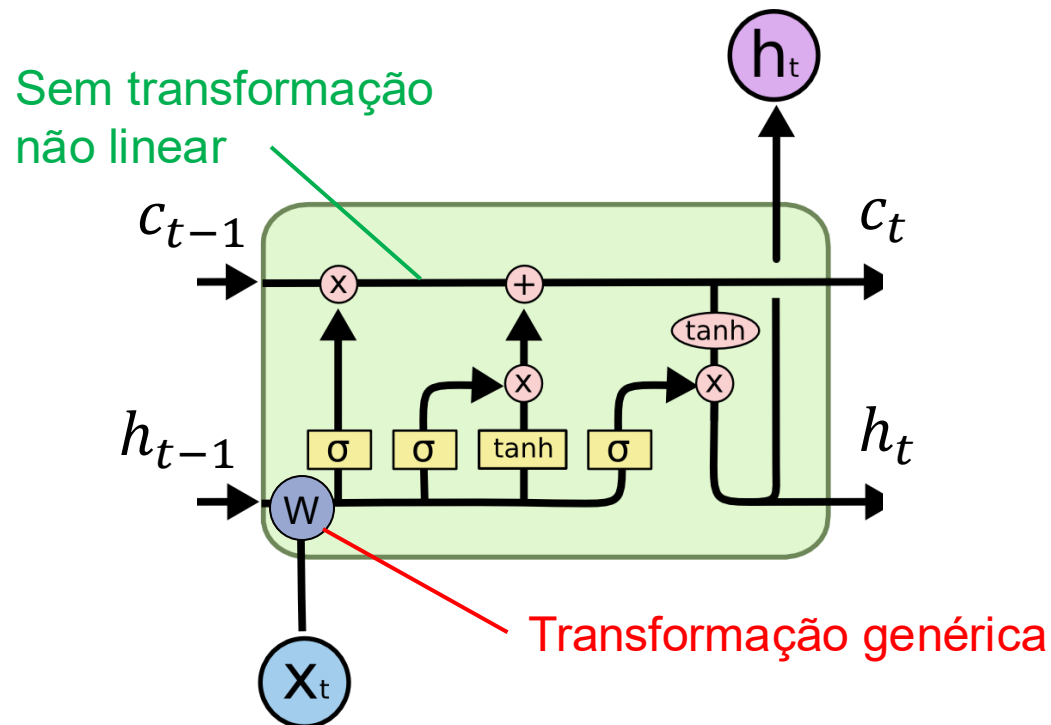
Sem transformação
não linear



LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

Unidades LSTM (*Long Short-Term Memory*) possuem uma memória de célula c_i que é repassada de um instante de tempo para o próximo (sem transformação não linear)



Elas também possuem informações de estado “escondidas” que passam por transformações não lineares como uma RNN tradicional

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

- Existem dois elementos recorrentes c_i e h_i

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

- Existem dois elementos recorrentes c_i e h_i
- h_i assume o papel da saída de uma RNN tradicional

LSTM – *Long Short Term Memory*

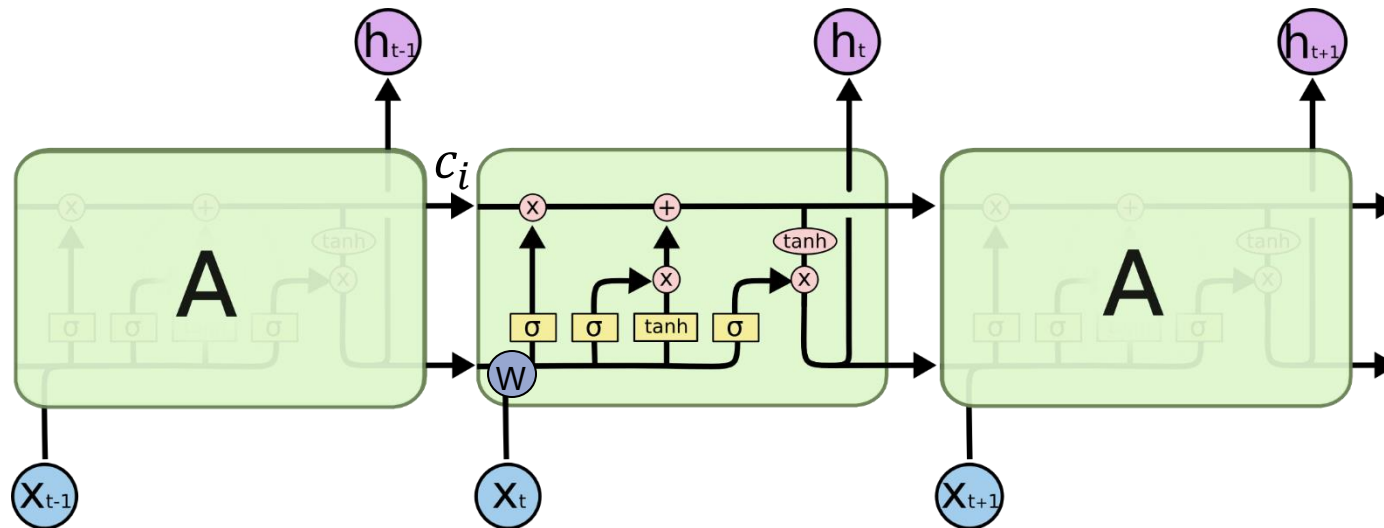
[Hochreiter et al., 1997]

- Existem dois elementos recorrentes c_i e h_i
- h_i assume o papel da saída de uma RNN tradicional
- O estado da célula c_i representa a memória da mesma e não passa por transformação

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

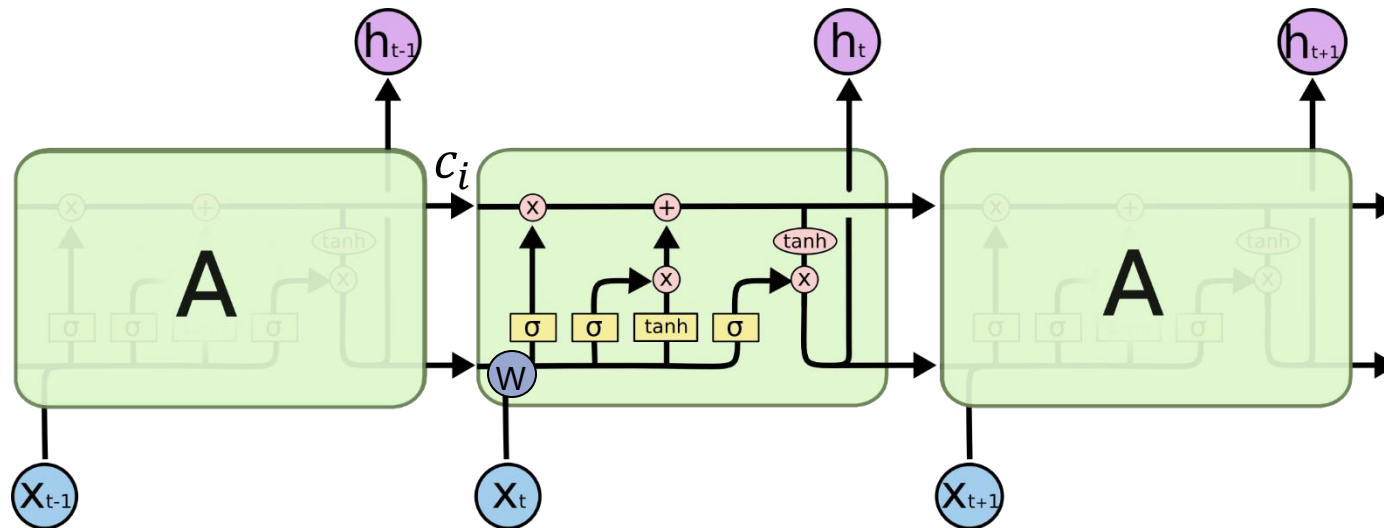
- Existem dois elementos recorrentes c_i e h_i
- h_i assume o papel da saída de uma RNN tradicional
- O estado da célula c_i representa a memória da mesma e não passa por transformação
- LSTM podem ser estendida no tempo (como uma RNN)



LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

- Existem dois elementos recorrentes c_i e h_i
- h_i assume o papel da saída de uma RNN tradicional
- O estado da célula c_i representa a memória da mesma e não passa por transformação
- LSTM podem ser estendida no tempo (como uma RNN)



- Para várias camadas, as saídas h_i s de uma camada tornam-se as entradas x_i s da próxima

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

$W^l [n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

The diagram shows the equation $h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$. A red arrow points from the text $h \in \mathbb{R}^n$ to the input vector $\begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$. Another red arrow points from the weight matrix W^l to the same input vector. Two blue arrows point from the text annotations to the input vector: one from 'Entrada da camada inferior $\equiv x$ ' to the top element h_t^{l-1} , and another from 'Entrada do passo anterior' to the bottom element h_{t-1}^l .

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

$W^l [n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

$W^l [n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

$W^l [n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$W^l [4n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

$W^l [n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$W^l [4n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

$W^l [n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$W^l [4n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM pode simular uma simples RNN com valores adequados para i , f e o

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

$W^l [n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$W^l [4n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM pode simular uma simples RNN com valores adequados para i , f e o

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

RNN:

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$

$W^l [n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

LSTM:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$W^l [4n \times 2n]$

Entrada da camada inferior $\equiv x$

Entrada do passo anterior

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM pode simular uma simples RNN com valores adequados para i , f e o

QUASE!

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

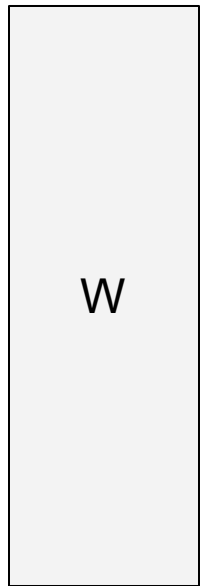
LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]



$4n \times 2n$

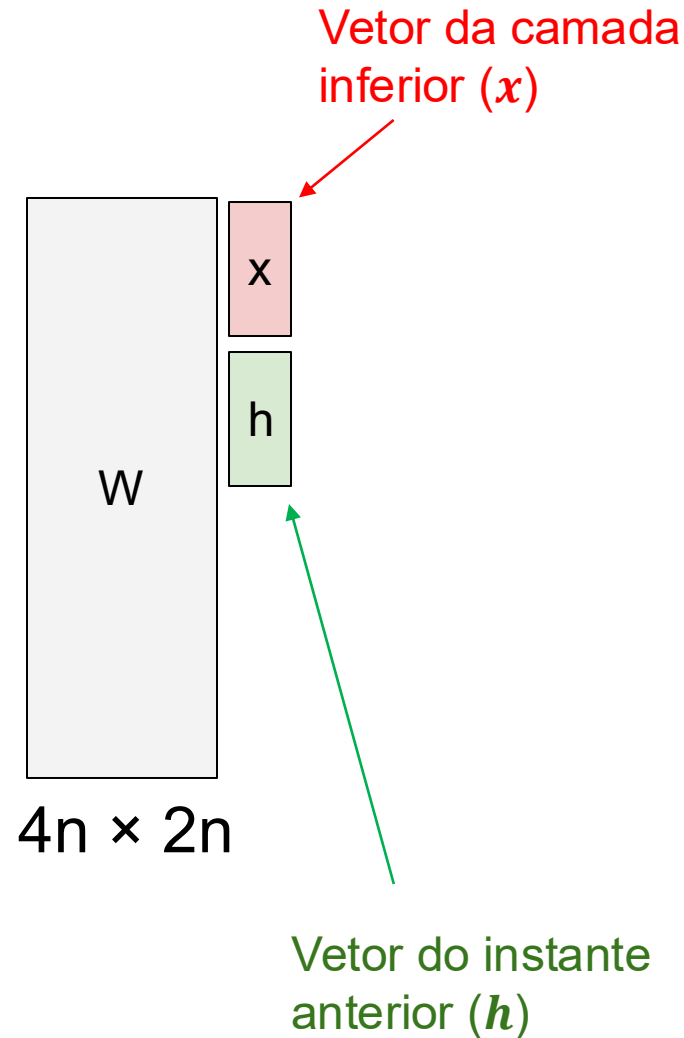
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

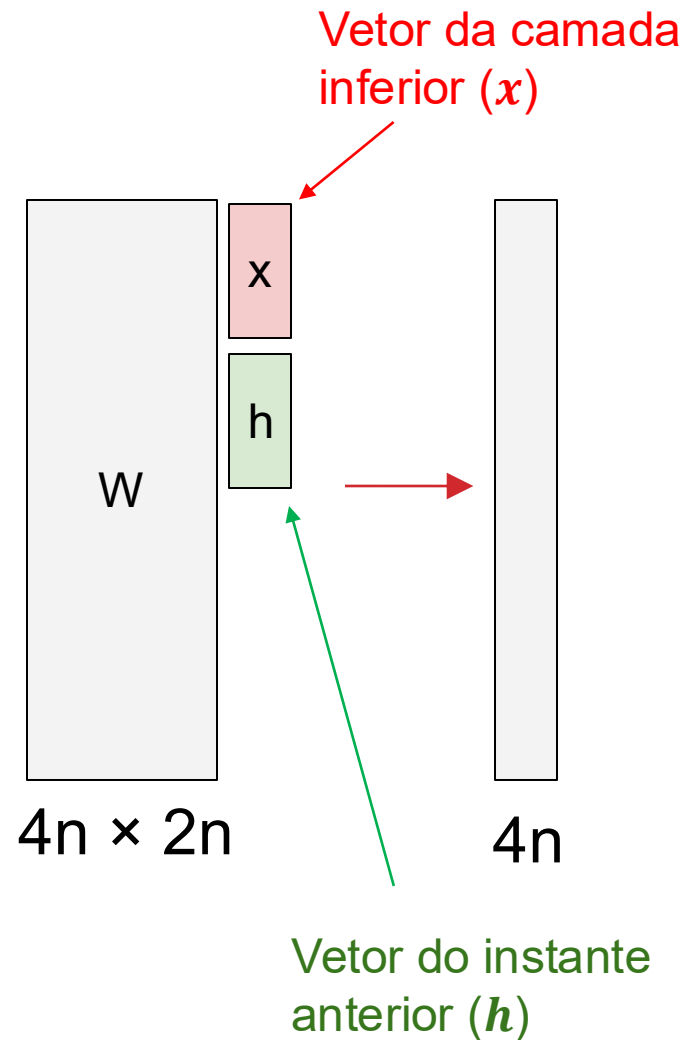
[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

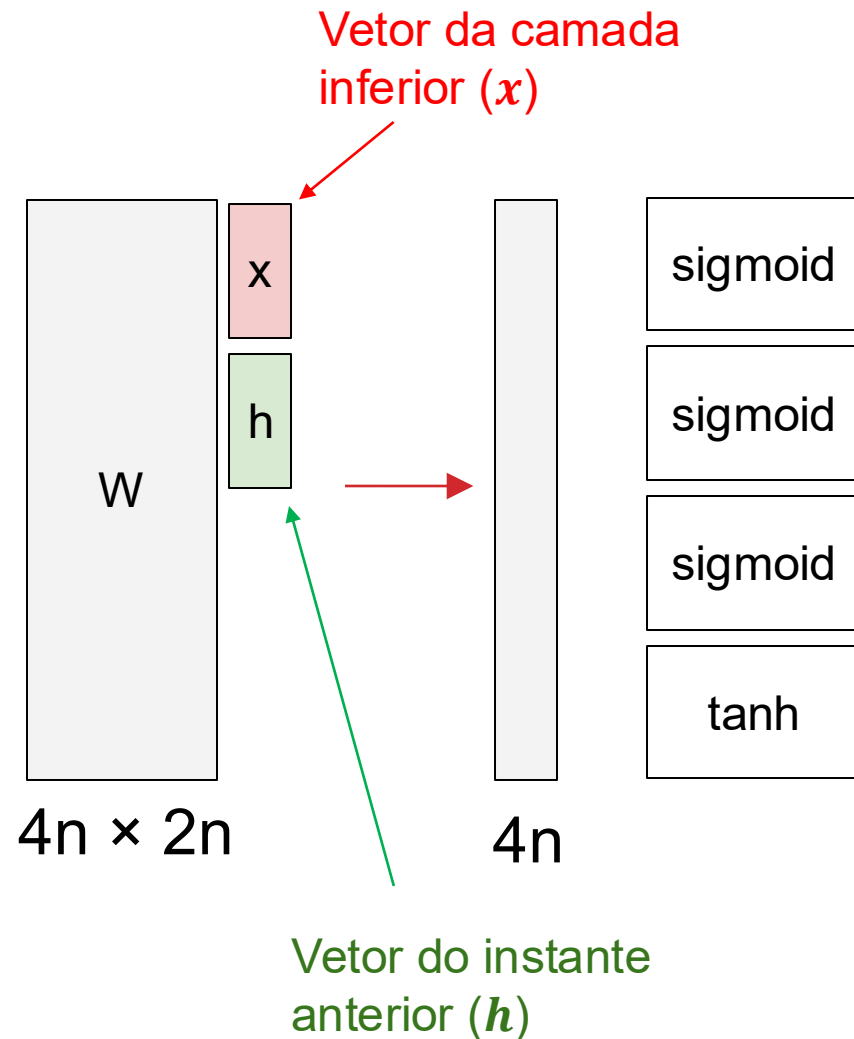
[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

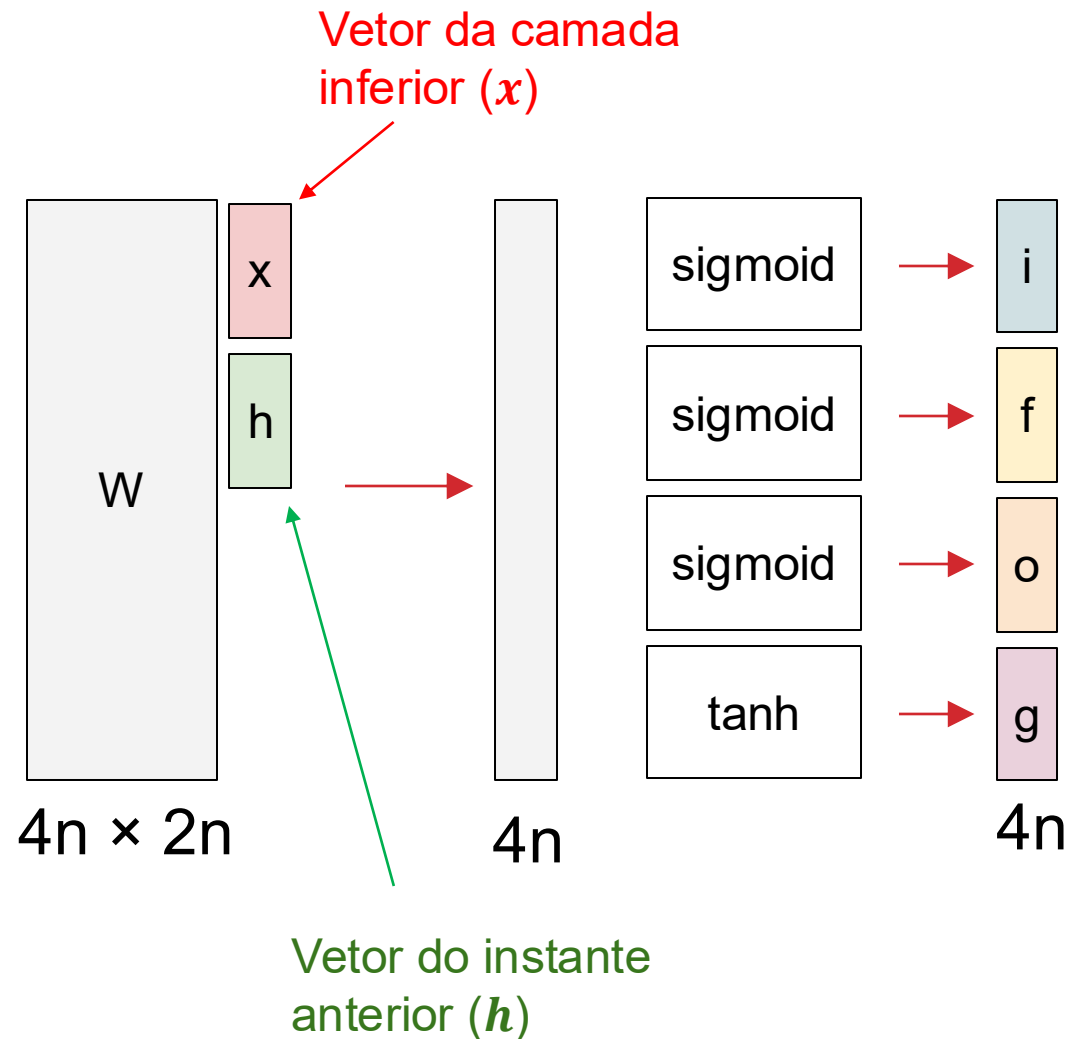
[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

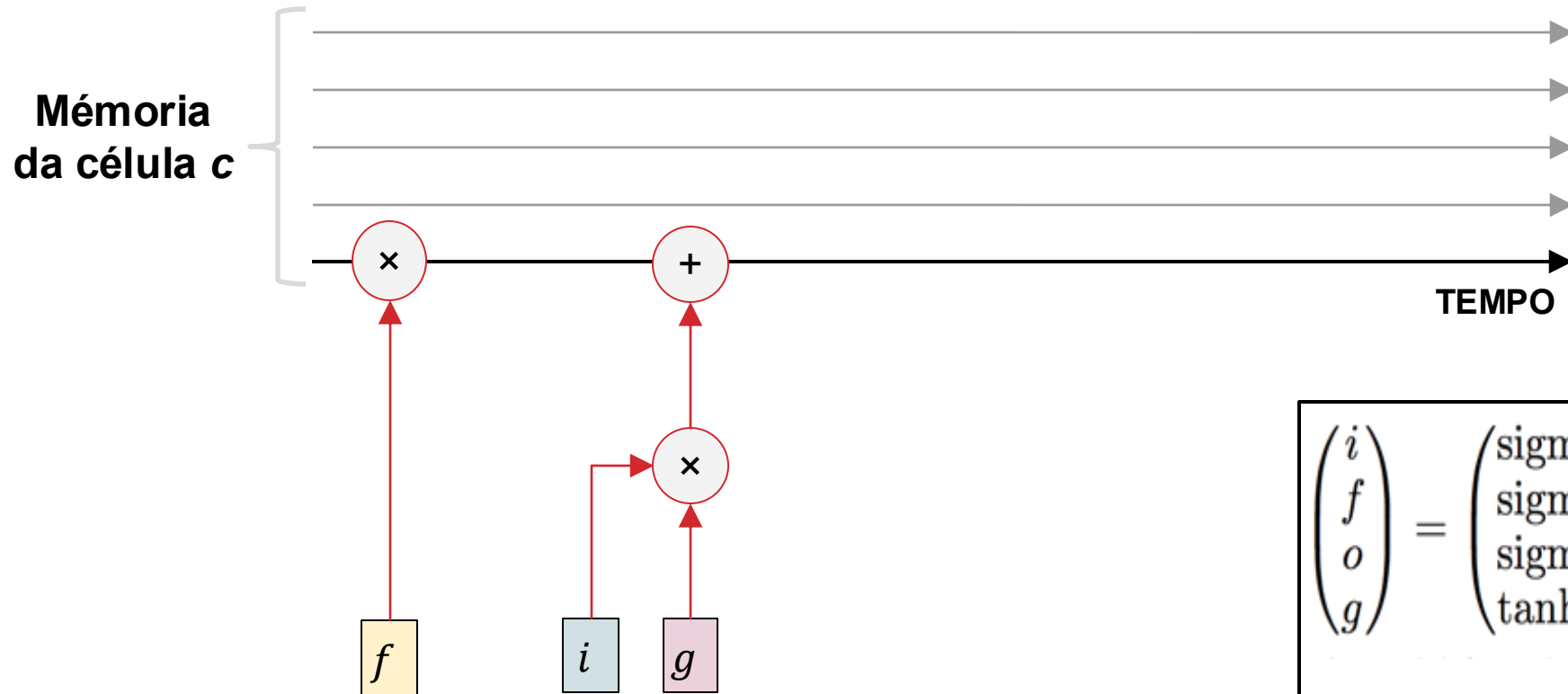
[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

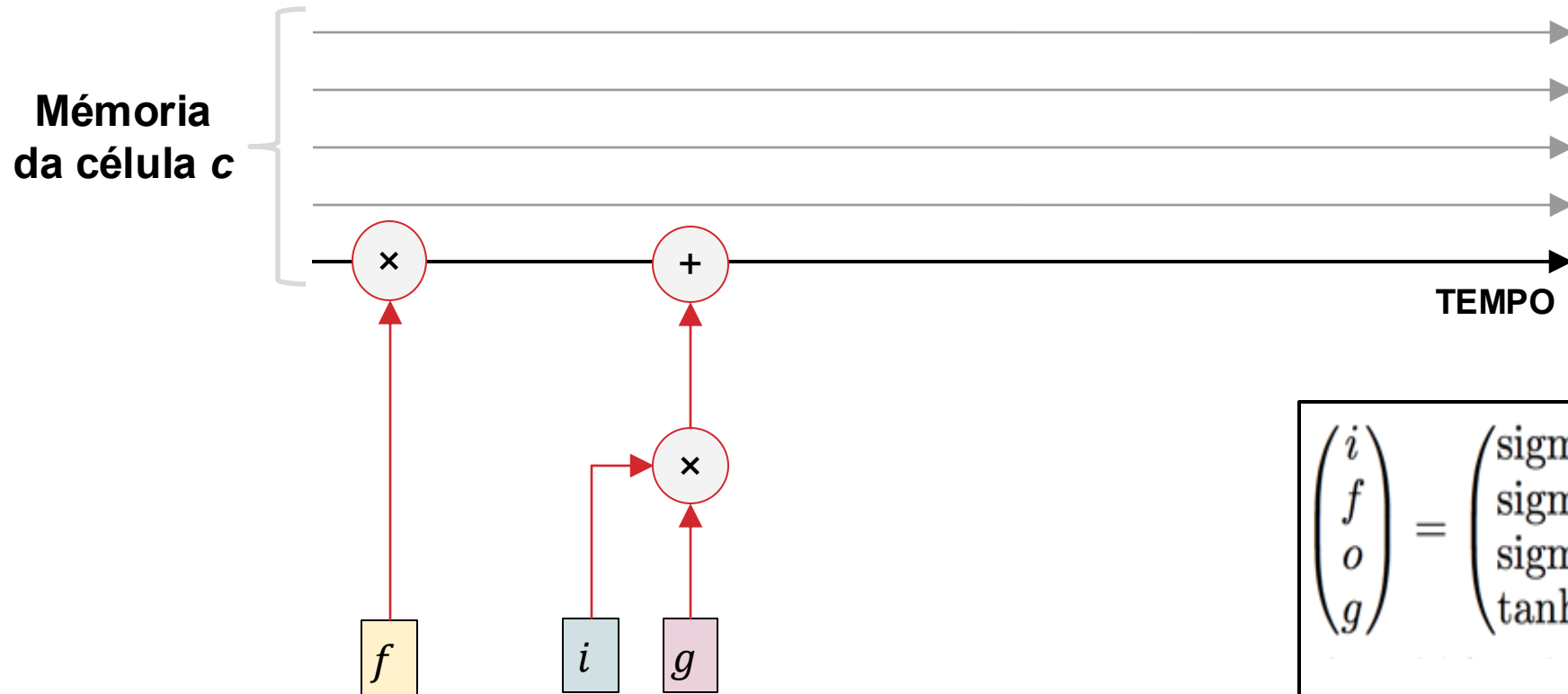
[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

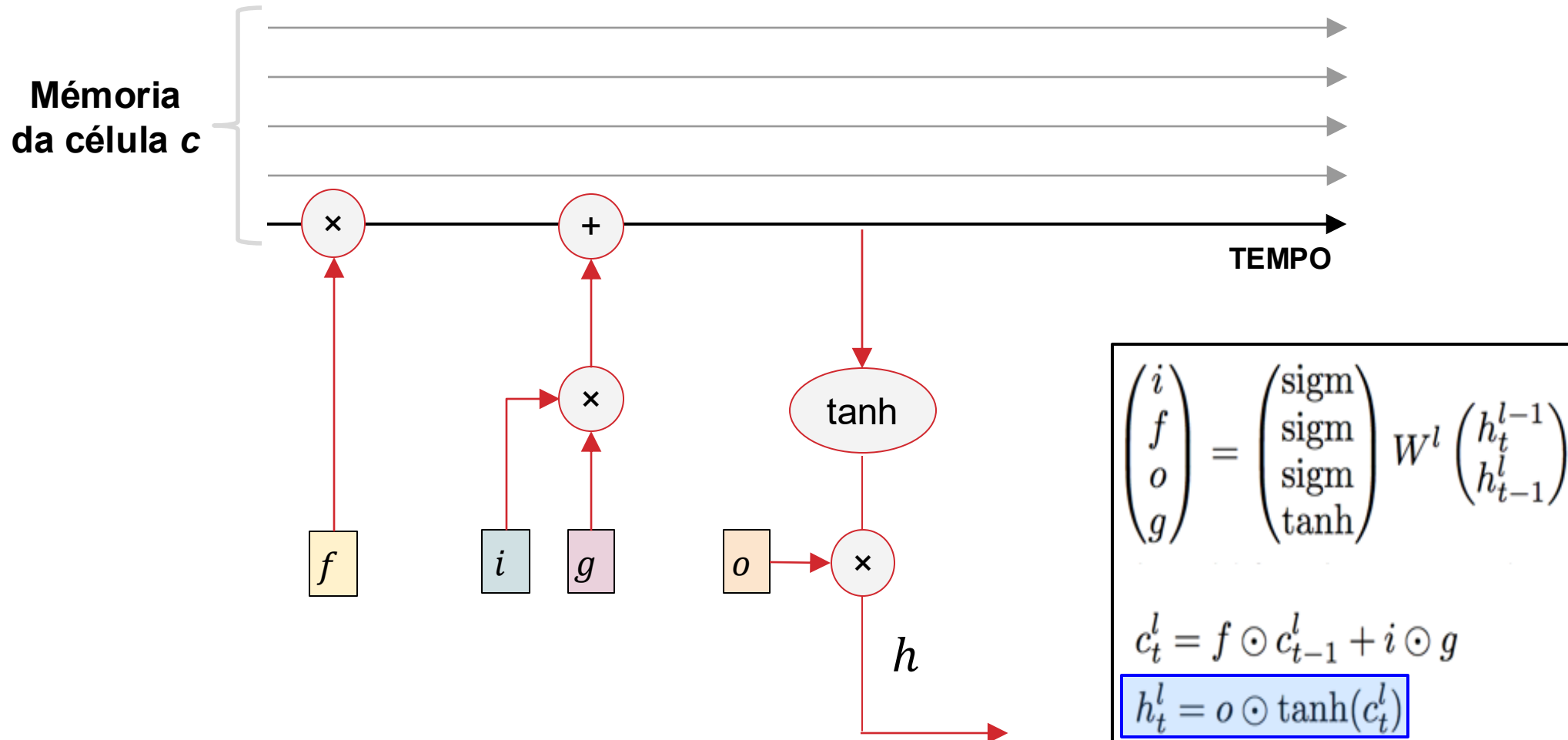
[Hochreiter et al., 1997]



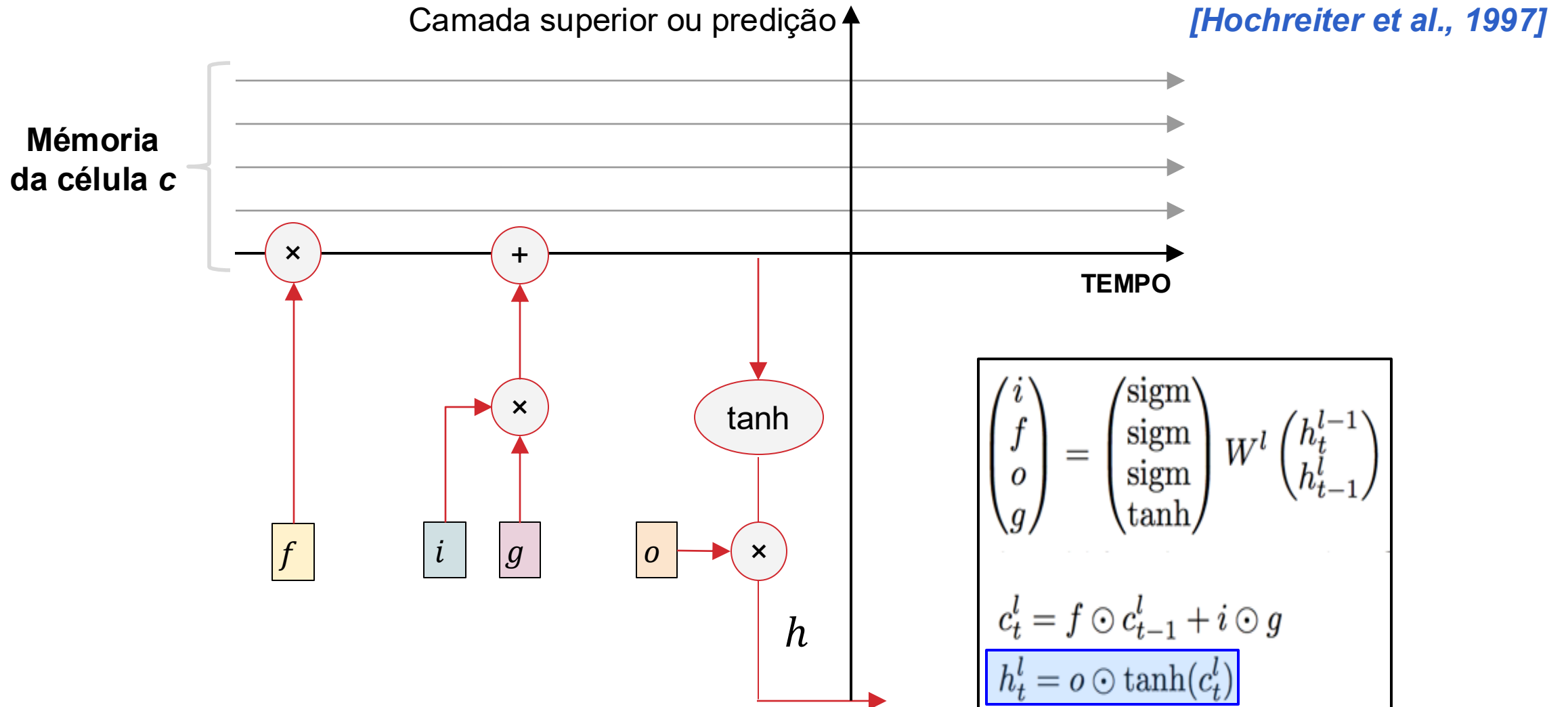
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$
$$h_t^l = o \odot \tanh(c_t^l)$$

LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

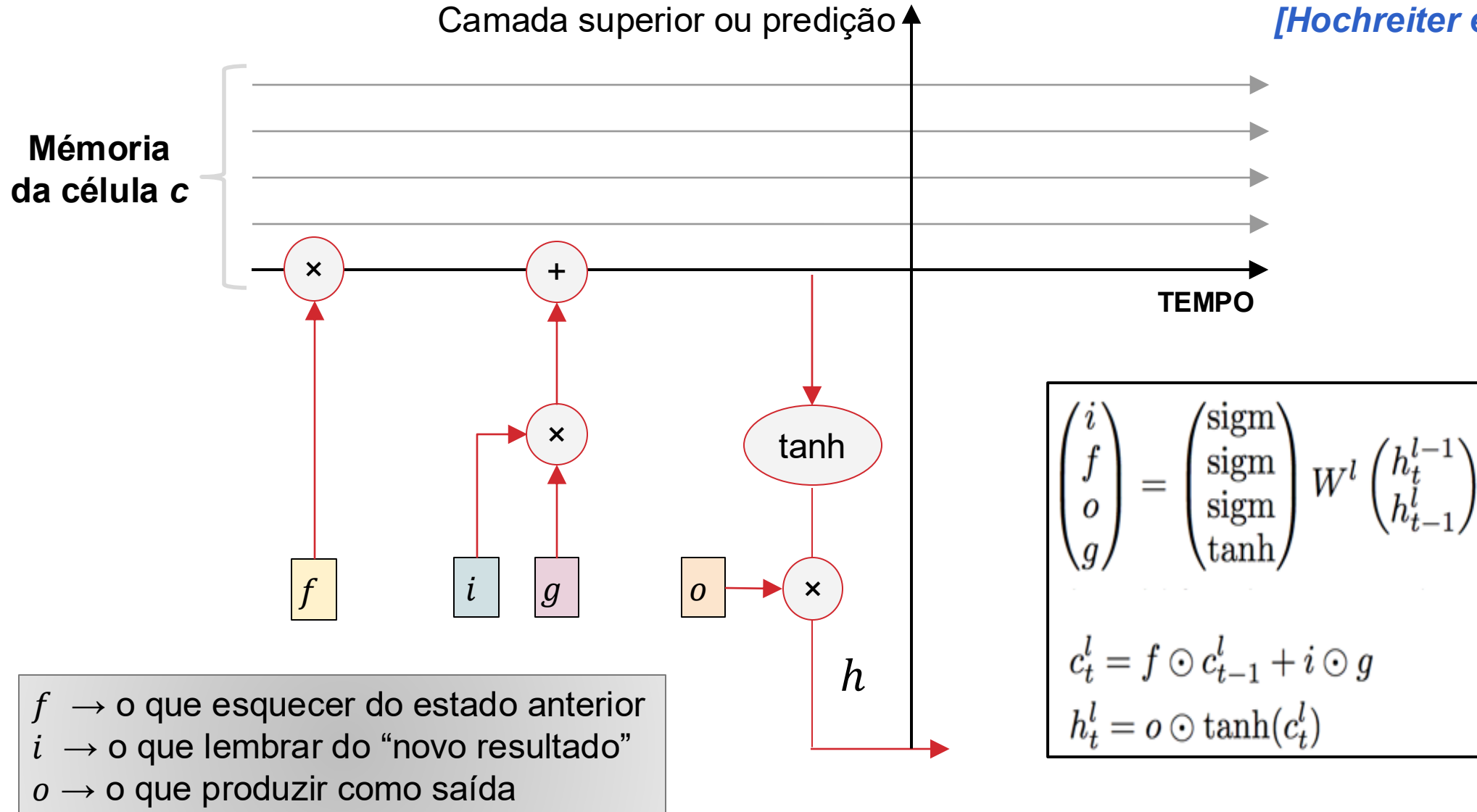


LSTM – Long Short Term Memory

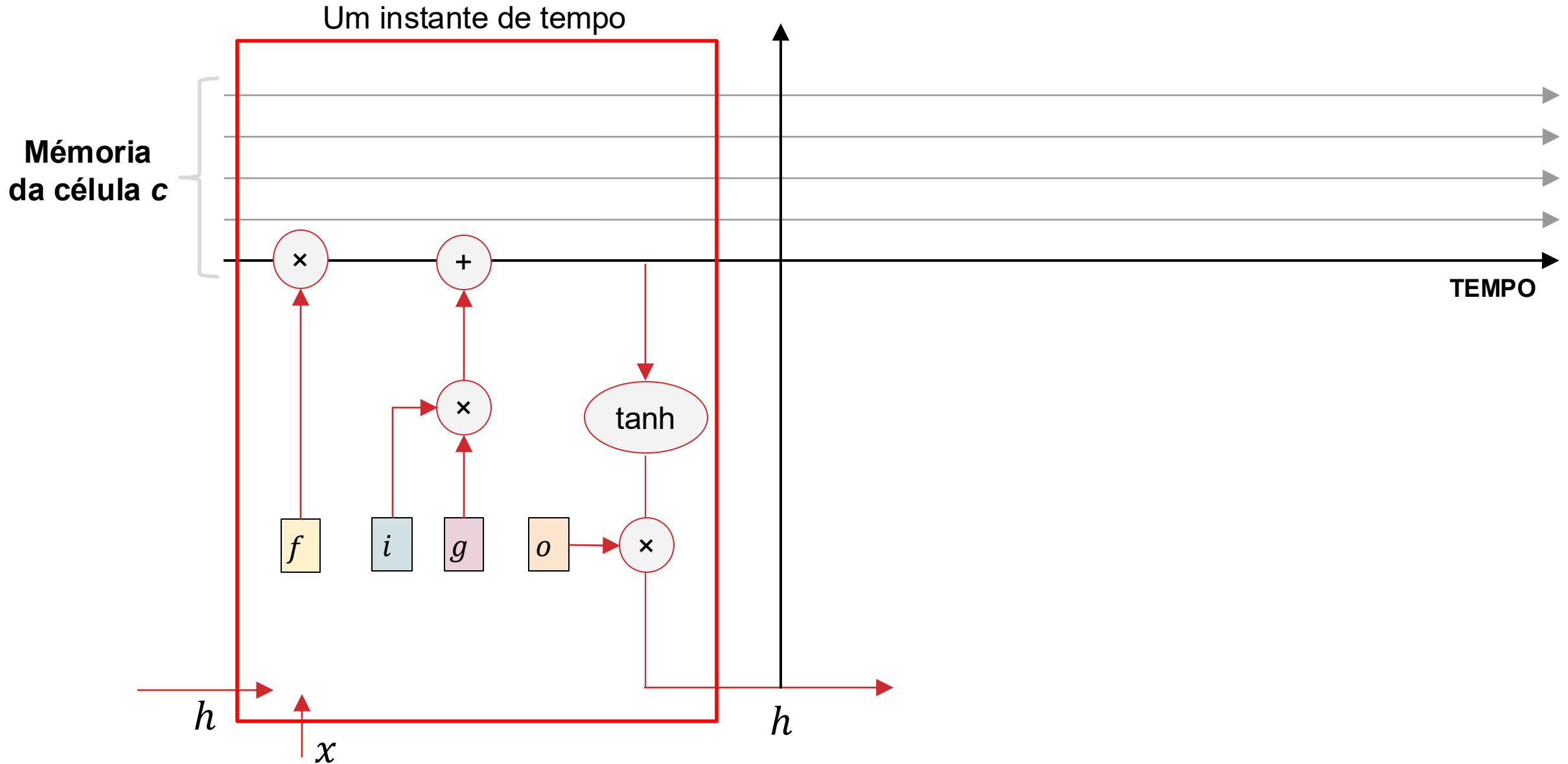


LSTM – Long Short Term Memory

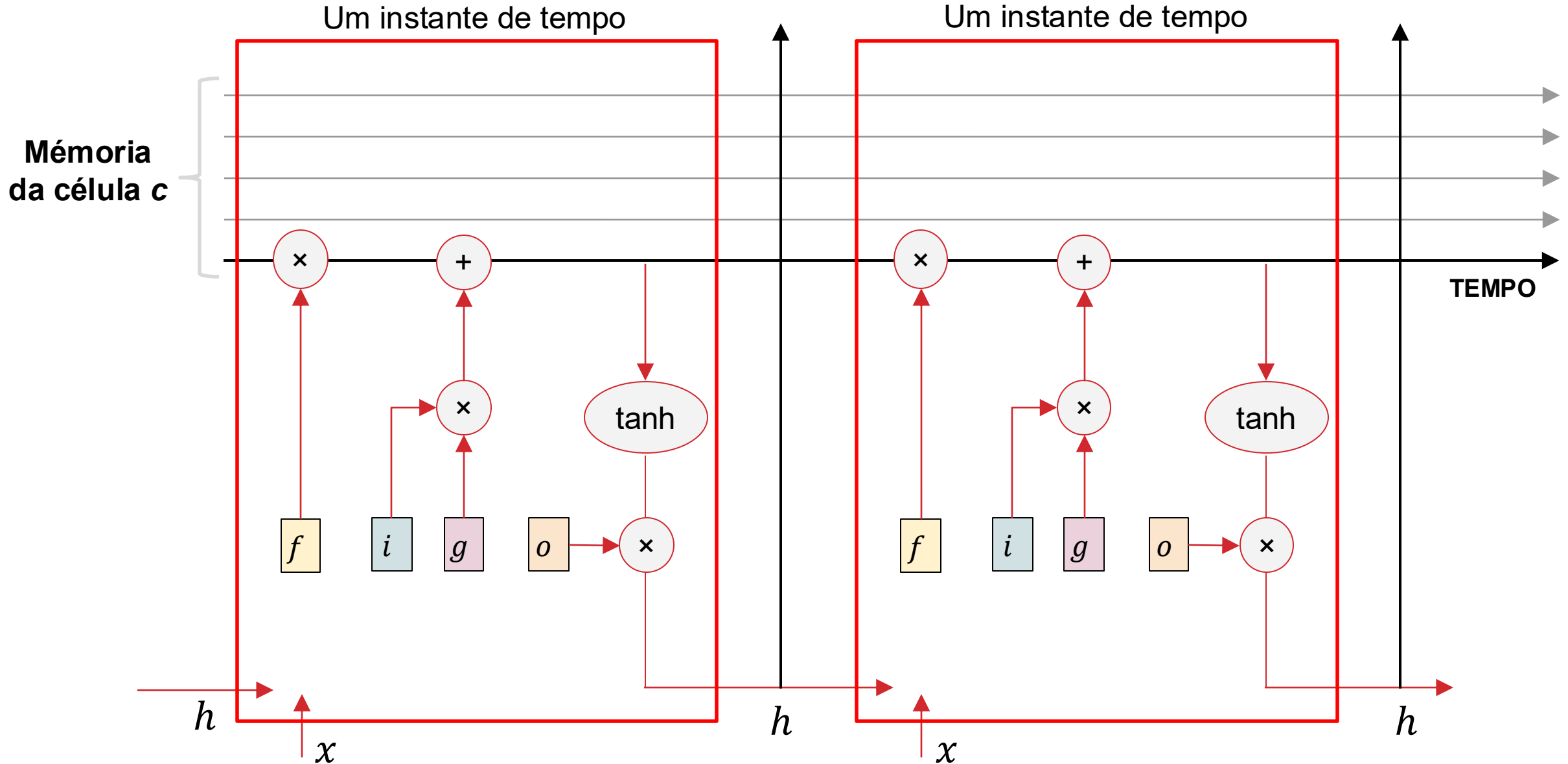
[Hochreiter et al., 1997]



LSTM – *Long Short Term Memory*



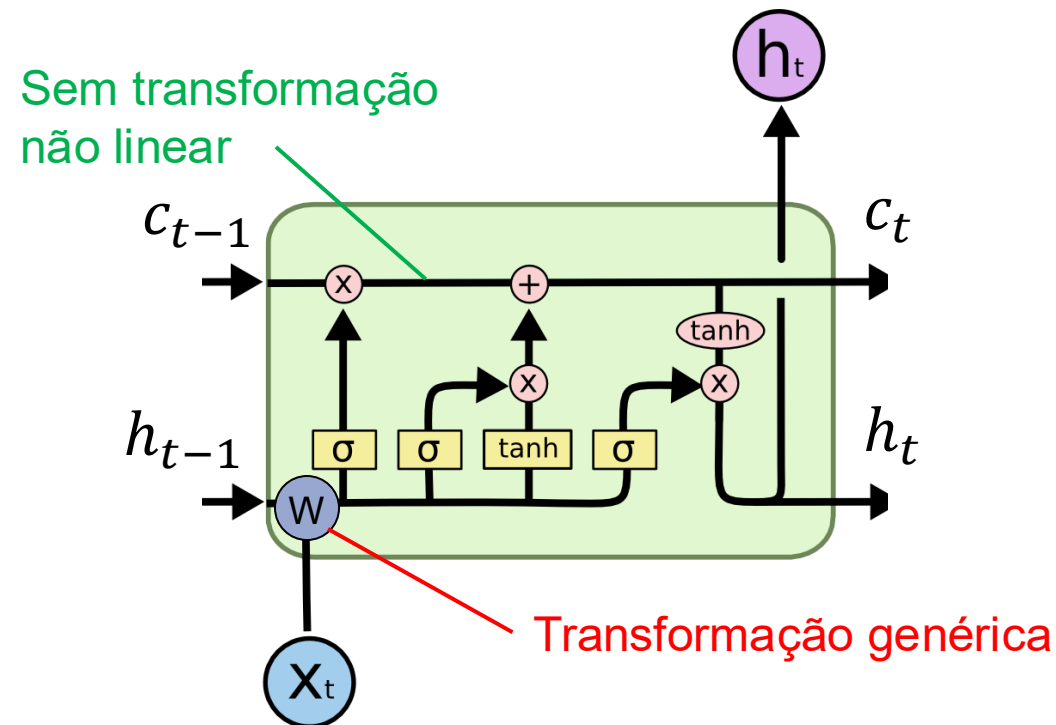
LSTM – *Long Short Term Memory*



LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

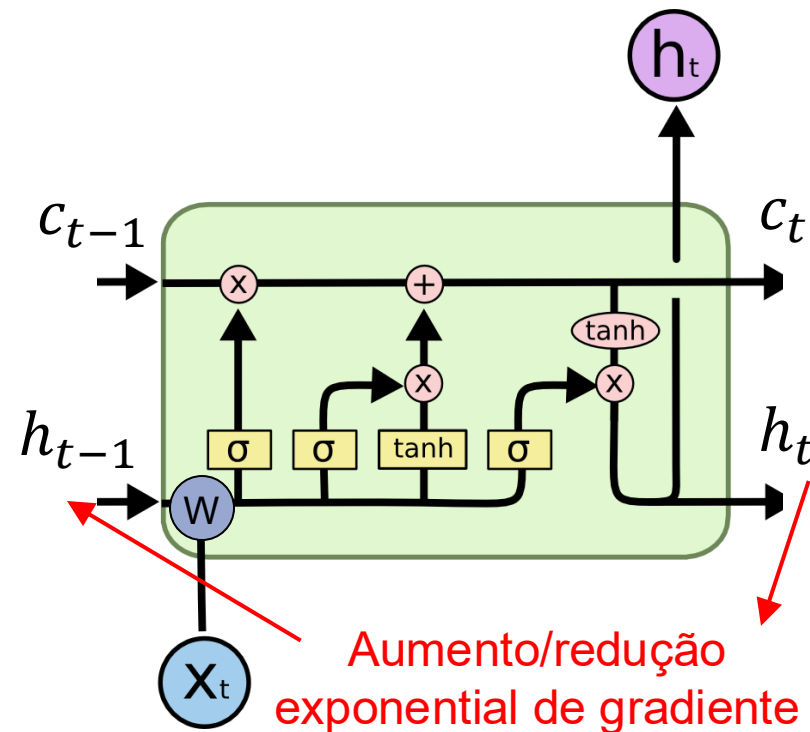
Lembre-se de que h passa por uma transformação que depende de W_{hh} em cada unidade



LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

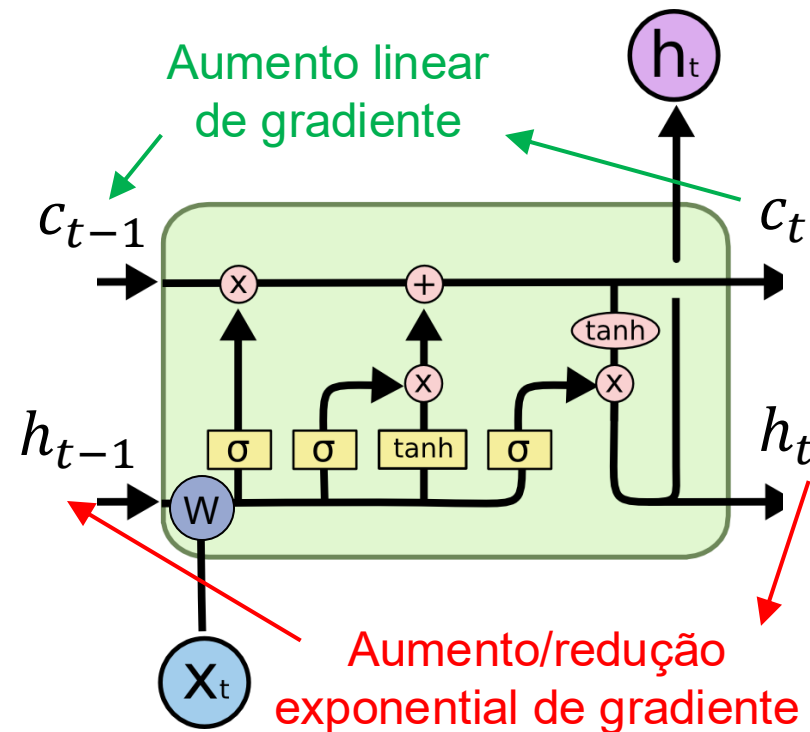
Lembre-se de que h passa por uma transformação que depende de W_{hh} em cada unidade



LSTM – *Long Short Term Memory*

[Hochreiter et al., 1997]

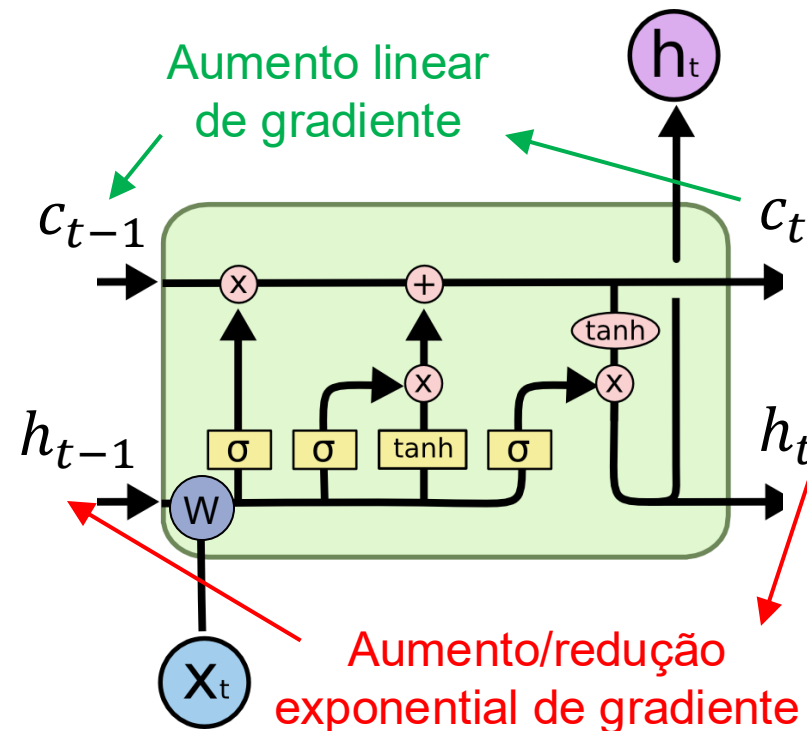
Lembre-se de que h passa por uma transformação que depende de W_{hh} em cada unidade



LSTM – Long Short Term Memory

[Hochreiter et al., 1997]

Lembre-se de que h passa por uma transformação que depende de W_{hh} em cada unidade



Gradientes são bem comportados no caminho de c mas não do de h , porém LSTMs aprendem a “confiar” principalmente em c para a memória de longo prazo

LSTM – Variações

GRU – *Gated Recurrent Unit* [Cho et al. 2014]

$$\begin{aligned}r_t &= \text{sigm}(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\z_t &= \text{sigm}(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \\\tilde{h}_t &= \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h) \\h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t\end{aligned}$$

Veja refs sobre equivalência:

- [*LSTM: A Search Space Odyssey*, Greff et al., 2015]
- [*An Empirical Exploration of Recurrent Network Architectures*, Jozefowicz et al., 2015]

LSTM – Variações

[*An Empirical Exploration of Recurrent Network Architectures*, Jozefowicz et al., 2015]

[*LSTM: A Search Space Odyssey*, Greff et al., 2015]

GRU [*Learning phrase representations using RNN encoder-decoder for statistical machine translation*, Cho et al. 2014]

$$\begin{aligned}r_t &= \text{sigm}(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\z_t &= \text{sigm}(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \\\tilde{h}_t &= \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h) \\h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t\end{aligned}$$

MUT1:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + b_z) \\r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + \tanh(x_t) + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

MUT2:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + W_{hz}h_t + b_z) \\r &= \text{sigm}(x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

MUT3:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + W_{hz} \tanh(h_t) + b_z) \\r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

LSTM – Variações

Estes projetos combinam as funções de c e h , e (similar a ResNet) sempre incluem um caminho identidade para o h anterior (*memory path*).

GRU:

$$\begin{aligned}r_t &= \text{sigm}(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\z_t &= \text{sigm}(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \\ \tilde{h}_t &= \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h) \\h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t\end{aligned}$$

MUT1:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + b_z) \\r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + \tanh(x_t) + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

MUT2:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + W_{hz}h_t + b_z) \\r &= \text{sigm}(x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

MUT3:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + W_{hz} \tanh(h_t) + b_z) \\r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

LSTM – Variações

Eles também combinam as funções de f (*forgetting gate*) e i (*input*) em um único z .

E quanto mais se adicionar do estado atual (isto é, z grande), menos $(1 - z)$ será lembrado do estado anterior h , e vice-versa.

GRU:

$$\begin{aligned}r_t &= \text{sigm}(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\z_t &= \text{sigm}(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \\\tilde{h}_t &= \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h) \\h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t\end{aligned}$$

MUT1:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + b_z) \\r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + \tanh(x_t) + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

MUT2:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + W_{hz}h_t + b_z) \\r &= \text{sigm}(x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

MUT3:

$$\begin{aligned}z &= \text{sigm}(W_{xz}x_t + W_{hz} \tanh(h_t) + b_z) \\r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

Novos Paradigmas para Modelar Sequências

Vaswani et al. “Attention is all you need”, NIPS 2018

- “*Transformer*” → nova arquitetura
 - Não processa sequencialmente
 - Opera em paralelo por meio de mecanismos de atenção
- Tem produzido resultados “SOTA” em NLP (além de facilitar refinamento)
 - BERT, Devlin et al., 2018 (110~340 M Params)
 - OpenAI GPT-2, Radford et al., 2018 (1,5 B Params)
 - OpenAI GPT-3, Brown et al., 2020 (175 B Params)

